

University of Chicago Political Science Math Prefresher

Anton Strezhnev

9/12/2023

Table of contents

1 Overview	3
1.1 Introduction	3
1.2 Course Booklet	3
1.3 Schedule	3
1.4 Software	4
1.5 Acknowledgments	4
2 Sets, Operations, and Functions	5
2.1 Sets	5
Sets	7
Sets	8
2.2 Metric spaces	8
2.3 Operators; Sum and Product notation	9
Operators	12
Operators	13
2.4 Introduction to Functions	13
Functions	15
Functions	16
2.5 Logarithms and Exponents	16
Logarithms	19
2.6 Graphing Functions	19
2.7 Solving for Variables and Finding Roots	20
Solving	21
Roots	22
3 Limits	23
Example: The Central Limit Theorem	23
Example: The Law of Large Numbers	24

3.1 Sequences	25
Sequences	26
3.2 The Limit of a Sequence	27
Ratios	28
Limits	29
3.3 Limits of a Function	29
Limits of a function	30
Limits of a function	31
Limits of ratios	32
Limits of a function	33
3.4 Continuity	33
Continuity	35
Continuity	36
4 Calculus	38
Example: The Mean is a Type of Integral	38
4.1 Derivatives	39
Derivative	40
Properties of derivatives	41
Power Rule	42
Derivatives	44
4.2 Higher-Order Derivatives (Derivatives of Derivatives of Derivatives)	44
Succession of derivatives	46
4.3 Composite Functions and the Chain Rule	46
Composite functions	47
Composite Exponent	48
4.4 Derivatives of natural logs and the exponent	48
Derivative of Exponents/Logs	49
Derivatives of natural exponential function (e)	49

Derivatives of exponents	51
Derivatives of logarithms	51
Derivatives of logs	53
Outline of Proof	53
4.5 Partial Derivatives	54
Partial derivatives	55
Partial derivatives	56
4.6 Taylor Series Approximation	56
4.7 The Indefinite Integration	57
Antiderivative	58
Antiderivative	59
4.8 Indefinite Integral	59
Graphing	60
Common Rules of Integration	61
Common Integration	62
4.9 The Definite Integral: The Area under the Curve	62
The Definite Integral (Riemann)	64
First Fundamental Theorem of Calculus	65
Second Fundamental Theorem of Calculus	66
Definite Integral of a monomial	67
Indefinite integrals	68
Common Rules for Definite Integrals	68
Definite integrals	69
4.10 Integration by Substitution	69
Integration by Substitutiton II	71
4.11 Integration by Parts	71
Integration by parts	72
Integration by parts	73

5 Optimization	74
Example: Meltzer-Richard	74
5.1 Maxima and Minima	76
Plotting a maximum and minimum	78
Maxima and Minima by drawing	80
5.2 Concavity of a Function	80
Concave Function	81
Convex Function	82
Quasiconcave Function	83
Quasiconvex Function	84
Quadratic Forms	84
Definiteness of Quadratic Forms	85
5.3 FOC and SOC	85
First Order Conditions	86
Gradient	87
Critical Point	88
Second Order Conditions	89
Hessian	90
Max and min with two dimensions	91
Definiteness and Concavity	91
5.4 Global Maxima and Minima	92
Optimization	93
5.5 Constrained Optimization	94
Equality Constraints	96
Constrained optimization with two goods and a budget constraint	99
5.6 Inequality Constraints	100
Constrained optimization	103
5.7 Kuhn-Tucker Conditions	104
Kuhn-Tucker with two variables	106

Kuhn-Tucker with logs	108
5.8 Applications of Quadratic Forms	109
6 Probability Theory	111
6.1 Counting rules	111
Counting Possibilities	112
Counting	114
Counting	115
6.2 Probability	115
Probability Definitions: Formal and Informal	115
Probability	116
Axioms of Probability	117
Probability Operations	117
Probability	118
Probability	119
6.3 Conditional Probability and Bayes Rule	119
Conditional Probability 1	120
Conditional Probability 2	121
Multiplicative Law of Probability	122
Law of total probability	123
Bayes' Rule	125
Conditional Probability	126
6.4 Independence	126
Independence	127
6.5 Random Variables	128
Random Variable	130
6.6 Distributions	130
Distribution of a random variable	131

Total Number of Occurrences	132
Discrete Random Variables	132
Discrete Random Variable	133
Probability Mass Function	134
Continuous Random Variables	135
Continuous Random Variable	136
Probability density function	137
Continuous R.V.	138
6.7 Joint Distributions	138
Discrete, Joint Distributions	140
6.8 Expectation	140
Expectation of a discrete R.V.	141
Expectation of a discrete R.V.	142
Expectation of a continuous R.V.	143
Expected Value of a Function	143
Properties of Expected Values	143
6.9 Variance and Covariance	145
Covariance	146
Correlation	147
Expectation and Variance 1	148
Expectation and Variance 2	149
Expectation and Variance 3	150
6.10 Distributions	150
Binomial Distribution	151
Binomial distribution	152
Poisson Distribution	153
Poisson Distribution	154

Uniform Distribution	155
Uniform	156
Normal Distribution	157
6.11 Summarizing Observed Events (Data)	158
Sample Covariance and Correlation	160
6.12 Asymptotic Theory	160
6.12.1 CLT and LLN	160
Central Limit Theorem	161
Weak Law of Large Numbers (LLN)	162
6.12.2 Big $\mathcal{O}$ Notation	162
7 Linear Algebra	164
7.1 Working with Vectors	164
Vector Algebra	166
Vector Algebra	167
7.2 Linear Independence	167
Linear independence	168
Linear independence	169
7.3 Basics of Matrix Algebra	169
Matrix addition	171
Scalar Multiplication	172
Matrix multiplication	173
Matrix Multiplication	175
7.4 Systems of Linear Equations	175
Linear Equations	177
7.5 Systems of Equations as Matrices	177
7.6 Finding Solutions to Augmented Matrices and Systems of Equations	178
Solving systems of equations	180
Solving Systems of Equations	181
7.7 Rank — and Whether a System Has One, Infinite, or No Solutions	181

Rank of Matrices	183
7.8 The Inverse of a Matrix	183
Matrix Inverse	185
Matrix Inverse	186
7.9 Linear Systems and Inverses	186
Solve linear system using inverses	187
7.10 Determinants	187
8 Determinants	189
8.1 Getting Inverse of a Matrix using its Determinant	189
9 Calculate Inverse using Determinant Formula	191
10 Programming: Orientation and Reading in Data	192
Motivation: Data and You	192
Where are we? Where are we headed?	192
Check your understanding	192
10.1 General Orientation	193
10.2 But what is R	194
10.3 The Computer and You: Giving Instructions	195
10.4 Base-R vs. tidyverse	195
Dataframe subsetting	196
Read data	197
Visualization	197
10.5 A is for Athens	198
10.5.1 Locating the Data	198
10.5.2 Reading in Data	199
10.5.3 Inspecting	199
10.5.4 Finding observations	201
Exercises	202
1	202
2	202
3	202
4	202
5	203
11 Programming: Manipulating Vectors and Matrices	204
Motivation	204
Where are we? Where are we headed?	204
11.1 Read Data	205
11.2 data.frame vs. matrices	207

11.3 Handling matrices in R	208
11.4 Variable Transformations	211
11.5 Linear Combinations	212
11.6 Matrix Basics	215
Checkpoint	223
1	223
2	223
3	223
Exercises	224
1	224
2	224
3	225
4	225
5	225
12 Objects, Functions, Loops	228
Where are we? Where are we headed?	228
12.1 What is an object?	228
12.1.1 Lists	229
12.2 Making your own objects	231
12.2.1 Seeing R through objects	233
12.2.2 Parsing an object by <code>str()</code> s	234
12.3 Types of variables	235
12.3.1 scalars	235
12.3.2 numeric vectors	236
12.3.3 characters (aka strings)	236
12.4 What is a function?	238
12.4.1 Write your own function	238
Checkpoint	240
1	240
2	240
3	240
12.5 What is a package?	240
12.6 Conditionals	241
12.7 For-loops	242
12.8 Nested Loops	244
Exercises	245
Exercise 1: Write your own function	245
Exercise 2: Using Loops	245
Exercise 3: Storing information derived within loops in a global dataframe	245
13 Joins and Merges, Wide and Long	248
Motivation	248

Where are we? Where are we headed?	248
13.1 Setting up	249
13.2 Create a project directory	249
13.3 Data Sources	249
13.4 Example with 2 Datasets	250
13.5 Loops	251
13.6 Merging	252
13.7 Main Project	254
Task 1: Data Input and Standardization	254
Task 2: Data Merging	255
Task 3: Tabulations and Visualization	255
14 Functionals and Optimization	256
14.1 More Tidyverse/Data Cleaning	256
Functionals	260
14.1.1 Replacing for loops	261
14.2 Excerise: Optimization	263
15 Simulation	267
Motivation: Simulation as an Analytical Tool	267
Where are we? Where are we headed?	268
Check your Understanding	268
15.1 Pick a sample, any sample	268
15.2 The <code>sample()</code> function	268
15.2.1 Sampling rows from a dataframe	270
15.3 Random numbers from specific distributions	273
<code>rbinom()</code>	273
<code>runif()</code>	274
<code>rnorm()</code>	274
15.4 <code>r</code> , <code>p</code> , and <code>d</code>	275
15.5 <code>set.seed()</code>	276
15.6 Calculating an expectation using Monte Carlo	277
Exercise	279
Baccarat	279
15.6.1 Part 1 (-)	281
15.6.2 Part 2 (-)	282
15.6.3 Part 3 (-)	282
15.6.4 Part 4 (-)	283
15.6.5 Part 5 (-)	283
15.6.6 Challenge problem (-)	283

1 Overview

1.1 Introduction

The 2023 UChicago Math Prefresher for incoming Political Science graduate students will be held from September 12-15 and September 18-20. The course is designed as a brief review of math fundamentals – calculus, optimization, probability theory and linear algebra among other topics – as well as an introduction to programming in the R statistical computing language. The course is entirely optional and there are no grades or assignments but we encourage all incoming graduate students to attend if they are able.

1.2 Course Booklet

The course notes for the math and programming sections as well as all practice problems are made available on this website and can be accessed by navigating the menus in the sidebar.

1.3 Schedule

The prefresher will run for a total of seven days September 12-15, and September 18-20 in *Pick Hall 506*. Each day will run from around 9am to 4pm with many breaks in between.

The morning will focus on math instruction. We will have two one hour sessions from 9:30am - 10:30am and 10:45am-11:45am, with a ~15 minute break in between. These sessions will involve a combination of lectures and working through practice problems.

We will break for lunch from 12:00pm-1:00pm.

The afternoon will focus on coding instruction with lecture/demonstration from 1:00pm-2:45pm. After a short break you will work together on a variety of coding exercises from 3:00-3:30pm. In the last 30 minutes we will regroup to wrap up and discuss any questions on the material.

1.4 Software

As the afternoons of the prefresher will involve instruction in coding, you should be sure to bring a laptop and a charge cable. In addition, prior to the start of the prefresher, please make sure to have installed the following on your computer.

- [R](#) (version 4.2.1 or higher)
- [RStudio Desktop Open Source License](#) (this is the primary IDE or integrated development environment in which we will be working)
- LaTeX: This is primarily to allow you to generate PDF documents using RMarkdown. We will use the TinyTeX LaTeX distribution which is designed to be minimalist and tailored specifically for R users. After installing R and RStudio, open up an instance of R, install the ‘tinytex’ package and run the `install_tinytex()` command

```
install.packages('tinytex')
tinytex::install_tinytex()
```

We will also spend some time discussing document preparation and typesetting using LaTeX and Markdown. For the former, we will be using the popular cloud platform [Overleaf](#), which allows for collaborative document editing and streamlines a lot of the irritating parts of typesetting in LaTeX. You should register for an account using your university e-mail as all University of Chicago students and faculty [have access](#) to an Overleaf Pro account for free.

You are also welcome to install a LaTeX editor on your local machine to work alongside the TinyTeX distribution or any other TeX distribution that you prefer such as [TexMaker](#)

1.5 Acknowledgments

This prefresher draws heavily on the wonderful materials that have been developed by over 20 years of instructors at the [Harvard Government Math Prefresher](#) that have been so generously distributed under the GPL 3.0 License. Special thanks to Shiro Kuriwaki, Yon Soo Park, and Connor Jerzak for their efforts in converting the original prefresher materials into the easily distributed Markdown format.

2 Sets, Operations, and Functions

2.1 Sets

Sets are the fundamental building blocks of mathematics. Events are not inherently numerical: the onset of war or the stock market crashing is not inherently a number. Sets can define such events, and we wrap math around so that we have a transparent language to communicate about those events. Combining sets with operations, relations, metrics, measures, etc... allows us to define useful mathematical structures. For example, the set of *real numbers* ($\mathbb{R}$) has a notion of *order* as well as defined *operations* of addition and multiplication.

Set : A set is any well defined collection of elements. If x is an element of S , $x \in S$.

Examples:

1. The set of choices available to a player in Rock-Paper-Scissors $\{\text{Rock, Paper, Scissors}\}$
2. The set of possible outcomes of a roll of a six-sided die $\{1, 2, 3, 4, 5, 6\}$
3. The set of all natural numbers $\mathbb{N}$
4. The set of all real numbers $\mathbb{R}$

Common mathematical notation relevant to sets:

- $\in$ = “is an element of”; $\notin$ = “is not an element of”
- $\forall$ = “for all” (universal quantifier)
- $\exists$ = “there exists” (existential quantifier)
- $:$ = “such that”

Subset: If every element of set A is also in set B , then A is a *subset* of B . $A \subseteq B$. If, in addition to being a subset of B , A is not equal to B , A is a *proper subset* $A \subset B$.

Empty Set: a set with no elements. $S = \{\}$. It is denoted by the symbol $\emptyset$.

Cardinality: The cardinality of a set S , typically written $|S|$ is the number of members of S .

Many sets are infinite. For example, $\mathbb{N}$ the set of natural numbers $\mathbb{N} = \{0, 1, 2, 3, 4, \dots\}$

- Sets with cardinality less than $|\mathbb{N}|$ are *countable*
- Sets with the same cardinality as $|\mathbb{N}|$ are *countably infinite*
- Sets with greater cardinality than $|\mathbb{N}|$ are *uncountably infinite* (e.g. the real numbers).

Set operations:

1. **Union:** The union of two sets A and B , $A \cup B$, is the set containing all of the elements in A or B . $A_1 \cup A_2 \cup \dots \cup A_n = \bigcup_{i=1}^n A_i$
2. **Intersection:** The intersection of sets A and B , $A \cap B$, is the set containing all of the elements in both A and B . $A_1 \cap A_2 \cap \dots \cap A_n = \bigcap_{i=1}^n A_i$
3. **Complement:** If set A is a subset of S , then the complement of A , denoted A^C , is the set containing all of the elements in S that are not in A .

Properties of set operations:

- **Commutative:** $A \cup B = B \cup A$; $A \cap B = B \cap A$
- **Associative:** $A \cup (B \cup C) = (A \cup B) \cup C$; $A \cap (B \cap C) = (A \cap B) \cap C$
- **Distributive:** $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$; $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$
- **de Morgan's laws:** $(A \cup B)^C = A^C \cap B^C$; $(A \cap B)^C = A^C \cup B^C$
- **Disjointness:** Sets are disjoint when they do not intersect, such that $A \cap B = \emptyset$. A collection of sets is pairwise disjoint (**mutually exclusive**) if, for all $i \neq j$, $A_i \cap A_j = \emptyset$. A collection of sets form a partition of set S if they are pairwise disjoint and they cover set S , such that $\bigcup_{i=1}^k A_i = S$.

Example 2.1.

Sets

Let set A be $\{1, 2, 3, 4\}$, B be $\{3, 4, 5, 6\}$, and C be $\{5, 6, 7, 8\}$. Sets A , B , and C are all subsets of the S which is $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

Write out the following sets:

1. $A \cup B$
2. $C \cap B$
3. B^c
4. $A \cap (B \cup C)$

Exercise 2.1.

Sets

Suppose you had a pair of four-sided dice. You sum the results from a single toss.

What is the set of possible outcomes?

Consider subsets $A = \{2, 8\}$ and $B = \{2, 3, 7\}$ of the sample space you found. What is

1. A^c
2. $(A \cup B)^c$

2.2 Metric spaces

A *metric space* is a set that has a notion of *distance* - called a “metric” - defined between any two elements (sometimes referred to as “points”).

The distance function $d(x, y)$ defines the distance between element x and element y

- The real numbers $\mathbb{R}$ have a single distance function: $d(x, y) = |x - y|$
- In higher-dimensional real space (e.g. $\mathbb{R}^2$), we can define multiple distance metrics between $x = (x_1, x_2)$ and $y = (y_1, y_2)$
 - “Euclidean” distance: $d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$
 - “Taxicab” distance: $d(x, y) = |x_1 - y_1| + |x_2 - y_2|$
 - Chebyshev distance: $d(x, y) = \max\{|x_1 - y_1|, |x_2 - y_2|\}$
- All of these generalize to $\mathbb{R}^n$

A metric is a function that satisfies the following axioms

1. A distance between a point and itself is zero $d(x, x) = 0$
2. The distance between two points is strictly positive $d(x, y) > 0 \forall x \neq y$
3. Distance from x to y is the same as the distance from y to x ($d(x, y) = d(y, x)$)
4. The “triangle inequality” holds: $d(x, z) \leq d(x, y) + d(y, z)$

Once we have a metric space, we can define some additional useful concepts

Ball: A ball of radius r centered at x_0 is a set that contains all points with a distance less than r from x_0 .

Sphere: A sphere of radius r centered at x_0 is the set that contains all points with a distance exactly r from x_0 .

Interior Point: The point x is an interior point of the set S if x is in S and if there is some ϵ -ball around x that contains only points in S . The **interior** of S is the collection of all interior points in S . The interior can also be defined as the union of all open sets in S .

- If the set S is circular, the interior points are everything inside of the circle, but not on the circle's rim.
- Example: The interior of the set $\{(x, y) : x^2 + y^2 \leq 4\}$ is $\{(x, y) : x^2 + y^2 < 4\}$.

Boundary Point: The point $\mathbf{x}$ is a boundary point of the set S if every ϵ -ball around $\mathbf{x}$ contains both points that are in S and points that are outside S . The **boundary** is the collection of all boundary points.

- If the set S is circular, the boundary points are everything on the circle's rim.
- Example: The boundary of $\{(x, y) : x^2 + y^2 \leq 4\}$ is $\{(x, y) : x^2 + y^2 = 4\}$.

Open: A set S is open if for each point $\mathbf{x}$ in S , there exists an open ϵ -ball around $\mathbf{x}$ completely contained in S .

- If the set S is circular and open, the points contained within the set get infinitely close to the circle's rim, but do not touch it.
- Example: $\{(x, y) : x^2 + y^2 < 4\}$

Closed: A set S is closed if it contains all of its boundary points.

- Alternatively: A set is closed if its complement is open.
- If the set S is circular and closed, the set contains all points within the rim as well as the rim itself.
- Example: $\{(x, y) : x^2 + y^2 \leq 4\}$
- Note: a set may be neither open nor closed. Example: $\{(x, y) : 2 < x^2 + y^2 \leq 4\}$

2.3 Operators; Sum and Product notation

Addition (+), Subtraction (-), multiplication and division are basic operations of arithmetic. In statistics or calculus, we will often want to add a *sequence* of numbers that can be expressed as a pattern without needing to write down all its components. For example, how would we express the sum of all numbers from 1 to 100 without writing a hundred numbers?

For this we use the summation operator $\sum$ and the product operator $\prod$.

Summation:

$$\sum_{i=1}^{100} x_i = x_1 + x_2 + x_3 + \cdots + x_{100}$$

The bottom of the $\sum$ symbol indicates an index (here, i), and its start value 1. At the top is where the index ends. The notion of “addition” is part of the $\sum$ symbol. The content to the right of the summation is the meat of what we add. While you can pick your favorite index, start, and end values, the content must also have the index.

A few important features of sums:

- $\sum_{i=1}^n cx_i = c \sum_{i=1}^n x_i$
- $\sum_{i=1}^n (x_i + y_i) = \sum_{i=1}^n x_i + \sum_{i=1}^n y_i$
- $\sum_{i=1}^n c = nc$

Product:

$$\prod_{i=1}^n x_i = x_1 x_2 x_3 \cdots x_n$$

Properties:

- $\prod_{i=1}^n cx_i = c^n \prod_{i=1}^n x_i$
- $\prod_{i=k}^n cx_i = c^{n-k+1} \prod_{i=k}^n x_i$
- $\prod_{i=1}^n (x_i + y_i) = \text{a total mess}$
- $\prod_{i=1}^n c = c^n$

Other Useful Operations

Factorials!:

$$x! = x \cdot (x-1) \cdot (x-2) \cdots (1)$$

Modulo: Tells you the remainder when you divide the first number by the second.

- $17 \bmod 3 = 2$
- $100 \% 30 = 10$

Example 2.2.

Operators

1. $\sum_{i=1}^5 i =$

2. $\prod_{i=1}^5 i =$

3. $14 \bmod 4 =$

4. $4! =$

Exercise 2.2.

Operators

Let $x_1 = 4, x_2 = 3, x_3 = 7, x_4 = 11, x_5 = 2$

1. $\sum_{i=1}^3 (7)x_i$

2. $\sum_{i=1}^5 2$

3. $\prod_{i=3}^5 (2)x_i$

2.4 Introduction to Functions

A **function** is a mapping, or transformation, that relates members of one set to members of another set. For instance, if you have two sets: set A and set B , a function from A to B maps every value a in set A such that $f(a) \in B$. Functions can be “many-to-one”, where many values or combinations of values from set A produce a single output in set B , or they can be “one-to-one”, where each value in set A corresponds to a single value in set B . A function by definition has a single function value for each element of its domain. This means, there cannot be “one-to-many” mapping.

Dimensionality: $\mathbf{R}^1$ is the set of all real numbers extending from $-\infty$ to $+\infty$ — i.e., the real number line. $\mathbf{R}^n$ is an n -dimensional space, where each of the n axes extends from $-\infty$ to $+\infty$.

- $\mathbf{R}^1$ is a one dimensional line.
- $\mathbf{R}^2$ is a two dimensional plane.
- $\mathbf{R}^3$ is a three dimensional space.

Points in $\mathbf{R}^n$ are ordered n -tuples (just means an combination of n elements where order matters), where each element of the n -tuple represents the coordinate along that dimension.

For example:

- $\mathbf{R}^1$: (3)
- $\mathbf{R}^2$: (-15, 5)

- $\mathbf{R}^3$: (86, 4, 0)

Examples of mapping notation:

Function of one variable: $f : \mathbf{R}^1 \rightarrow \mathbf{R}^1$

- $f(x) = x + 1$. For each x in $\mathbf{R}^1$, $f(x)$ assigns the number $x + 1$.

Function of two variables: $f : \mathbf{R}^2 \rightarrow \mathbf{R}^1$.

- $f(x, y) = x^2 + y^2$. For each ordered pair (x, y) in $\mathbf{R}^2$, $f(x, y)$ assigns the number $x^2 + y^2$.

We often use variable x as input and another y as output, e.g. $y = x + 1$

Example 2.3.

Functions

For each of the following, state whether they are one-to-one or many-to-one functions.

1. For $x \in [0, \infty]$, $f : x \rightarrow x^2$ (this could also be written as $f(x) = x^2$).
2. For $x \in [-\infty, \infty]$, $f : x \rightarrow x^2$.

Exercise 2.3.

Functions

For each of the following, state whether they are one-to-one or many-to-one functions.

1. For $x \in [-3, \infty]$, $f : x \rightarrow x^2$.
2. For $x \in [0, \infty]$, $f : x \rightarrow \sqrt{x}$

Some functions are defined only on proper subsets of $\mathbf{R}^n$.

- **Domain:** the set of numbers in X at which $f(x)$ is defined.
- **Range:** elements of Y assigned by $f(x)$ to elements of X , or $f(X) = \{y : y = f(x), x \in X\}$ Most often used when talking about a function $f : \mathbf{R}^1 \rightarrow \mathbf{R}^1$.
- **Image:** same as range, but more often used when talking about a function $f : \mathbf{R}^n \rightarrow \mathbf{R}^1$.

Some General Types of Functions

Monomials: $f(x) = ax^k$

a is the coefficient. k is the degree.

Examples: $y = x^2$, $y = -\frac{1}{2}x^3$

Polynomials: sum of monomials.

Examples: $y = -\frac{1}{2}x^3 + x^2$, $y = 3x + 5$

The degree of a polynomial is the highest degree of its monomial terms. Also, it's often a good idea to write polynomials with terms in decreasing degree.

2.5 Logarithms and Exponents

Exponential Functions: Example: $y = 2^x$

Relationship of logarithmic and exponential functions:

$$y = \log_a(x) \iff a^y = x$$

The log function can be thought of as an inverse for exponential functions. a is referred to as the “base” of the logarithm.

Common Bases: The two most common logarithms are base 10 and base e .

1. Base 10: $y = \log_{10}(x) \iff 10^y = x$. The base 10 logarithm is often simply written as “ $\log(x)$ ” with no base denoted.
2. Base e : $y = \log_e(x) \iff e^y = x$. The base e logarithm is referred to as the “natural” logarithm and is written as “ $\ln(x)$ ”.

Properties of exponential functions:

- $a^x a^y = a^{x+y}$
- $a^{-x} = 1/a^x$
- $a^x / a^y = a^{x-y}$
- $(a^x)^y = a^{xy}$
- $a^0 = 1$

Properties of logarithmic functions (any base):

Generally, when statisticians or social scientists write $\log(x)$ they mean $\log_e(x)$. In other words: $\log_e(x) \equiv \ln(x) \equiv \log(x)$

$$\log_a(a^x) = x$$

and

$$a^{\log_a(x)} = x$$

- $\log(xy) = \log(x) + \log(y)$
- $\log(x^y) = y \log(x)$
- $\log(1/x) = \log(x^{-1}) = -\log(x)$
- $\log(x/y) = \log(x \cdot y^{-1}) = \log(x) + \log(y^{-1}) = \log(x) - \log(y)$
- $\log(1) = \log(e^0) = 0$

Change of Base Formula: Use the change of base formula to switch bases as necessary:

$$\log_b(x) = \frac{\log_a(x)}{\log_a(b)}$$

Example:

$$\log_{10}(x) = \frac{\ln(x)}{\ln(10)}$$

You can use logs to go between sum and product notation. This will be particularly important when you’re learning how to optimize likelihood functions.

$$\begin{aligned}
\log\left(\prod_{i=1}^n x_i\right) &= \log(x_1 \cdot x_2 \cdot x_3 \cdots x_n) \\
&= \log(x_1) + \log(x_2) + \log(x_3) + \cdots + \log(x_n) \\
&= \sum_{i=1}^n \log(x_i)
\end{aligned}$$

Therefore, you can see that the log of a product is equal to the sum of the logs. We can write this more generally by adding in a constant, c :

$$\begin{aligned}
\log\left(\prod_{i=1}^n cx_i\right) &= \log(cx_1 \cdot cx_2 \cdots cx_n) \\
&= \log(c^n \cdot x_1 \cdot x_2 \cdots x_n) \\
&= \log(c^n) + \log(x_1) + \log(x_2) + \cdots + \log(x_n) \\
&= n \log(c) + \sum_{i=1}^n \log(x_i)
\end{aligned}$$

Example 2.4.

Logarithms

Evaluate each of the following logarithms

1. $\log_4(16)$

2. $\log_2(16)$

Simplify the following logarithm. By “simplify”, we actually really mean - use as many of the logarithmic properties as you can.

3. $\log_4(x^3y^5)$

Exercise 2.4. Evaluate each of the following logarithms

1. $\log_{\frac{3}{2}}(\frac{27}{8})$

Simplify each of the following logarithms. By “simplify”, we actually really mean - use as many of the logarithmic properties as you can.

2. $\log(\frac{x^9y^5}{z^3})$

3. $\ln \sqrt{xy}$

2.6 Graphing Functions

What can a graph tell you about a function?

- Is the function increasing or decreasing? Over what part of the domain?
- How “fast” does it increase or decrease?
- Are there global or local maxima and minima? Where?
- Are there inflection points?
- Is the function continuous?
- Is the function differentiable?
- Does the function tend to some limit?
- Other questions related to the substance of the problem at hand.

2.7 Solving for Variables and Finding Roots

Sometimes we're given a function $y = f(x)$ and we want to find how x varies as a function of y . Use algebra to move x to the left hand side (LHS) of the equation and so that the right hand side (RHS) is only a function of y .

Example 2.5.

Solving

Solve for x:

1. $y = 3x + 2$

2. $y = e^x$

Solving for variables is especially important when we want to find the **roots** of an equation: those values of variables that cause an equation to equal zero. Especially important in finding equilibria and in doing maximum likelihood estimation.

Procedure: Given $y = f(x)$, set $f(x) = 0$. Solve for x .

Multiple Roots:

$$f(x) = x^2 - 9 \implies 0 = x^2 - 9 \implies 9 = x^2 \implies \pm\sqrt{9} = \sqrt{x^2} \implies \pm 3 = x$$

Quadratic Formula: For quadratic equations $ax^2 + bx + c = 0$, use the quadratic formula:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Exercise 2.5.

Roots

Solve for x:

1. $f(x) = 3x + 2 = 0$

2. $f(x) = x^2 + 3x - 4 = 0$

3. $f(x) = e^{-x} - 10 = 0$

3 Limits

Solving limits, i.e. finding out the value of functions as its input moves closer to some value, is important for the social scientist's mathematical toolkit for two related tasks. The first is for the study of calculus, which will be in turn useful to show where certain functions are maximized or minimized. The second is for the study of statistical inference, which is the study of inferring things about things you cannot see by using things you can see.

Example: The Central Limit Theorem

Perhaps the most important theorem in statistics is the Central Limit Theorem,

Theorem 3.1 (Central Limit Theorem). *For any series of independent and identically distributed random variables $X_1, X_2, \dots$, we know the distribution of its sum even if we do not know the distribution of X . The distribution of the sum is a Normal distribution.*

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \text{Normal}(0, 1)$$

where μ is the mean of X and σ is the standard deviation of X . The arrow is read as “converges in distribution to”. $\text{Normal}(0, 1)$ indicates a Normal Distribution with mean 0 and variance 1.

That is, the limit of the distribution of the lefthand side is the distribution of the righthand side.

The sign of a limit is the arrow “ $\rightarrow$ ”. Although we have not yet covered probability so we have not described what distributions and random variables are, it is worth foreshadowing the Central Limit Theorem. The Central Limit Theorem is powerful because it gives us a *guarantee* of what would happen if $n \rightarrow \infty$, which in this case means we collected more data.

Example: The Law of Large Numbers

A finding that perhaps rivals the Central Limit Theorem is the (Weak) Law of Large Numbers:

Theorem 3.2 ((Weak) Law of Large Numbers). *For any draw of identically distributed independent variables with mean μ , the sample average after n draws, $\bar{X}_n$, converges in probability to the true mean as $n \rightarrow \infty$:*

$$\lim_{n \rightarrow \infty} P(|\bar{X}_n - \mu| > \varepsilon) = 0$$

A shorthand of which is $\bar{X}_n \xrightarrow{p} \mu$, where the arrow is read as “converges in probability to”.

Intuitively, the more data, the more accurate is your guess. For example, Figure 3.1 shows how the sample average from many coin tosses converges to the true value : 0.5.

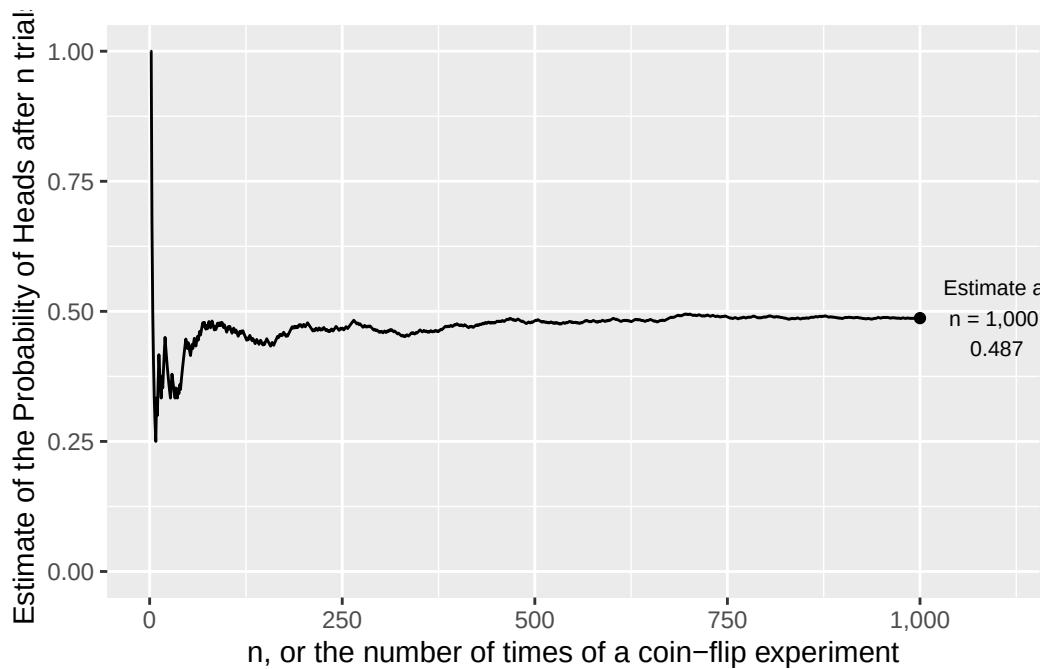


Figure 3.1: As the number of coin tosses goes to infinity, the average probability of heads converges to 0.5

3.1 Sequences

We need a couple of steps until we get to limit theorems in probability. First we will introduce a “sequence”, then we will think about the limit of a sequence, then we will think about the limit of a *function*.

A **sequence** $\{x_n\} = \{x_1, x_2, x_3, \dots, x_n\}$ is an ordered set of real numbers, where x_1 is the first term in the sequence and y_n is the n th term. Generally, a sequence is infinite, that is it extends to $n = \infty$. We can also write the sequence as $\{x_n\}_{n=1}^{\infty}$

where the subscript and superscript are read together as “from 1 to infinity.”

Example 3.1.

Sequences

How do these sequences behave?

1. $\{A_n\} = \{2 - \frac{1}{n^2}\}$
2. $\{B_n\} = \{\frac{n^2+1}{n}\}$
3. $\{C_n\} = \{(-1)^n (1 - \frac{1}{n})\}$

We find the sequence by simply “plugging in” the integers into each n . The important thing is to get a sense of how these numbers are going to change.

Graphing helps you make this point more clearly. See the sequence of $n = 1, \dots, 20$ for each of the three examples in Figure 3.2.

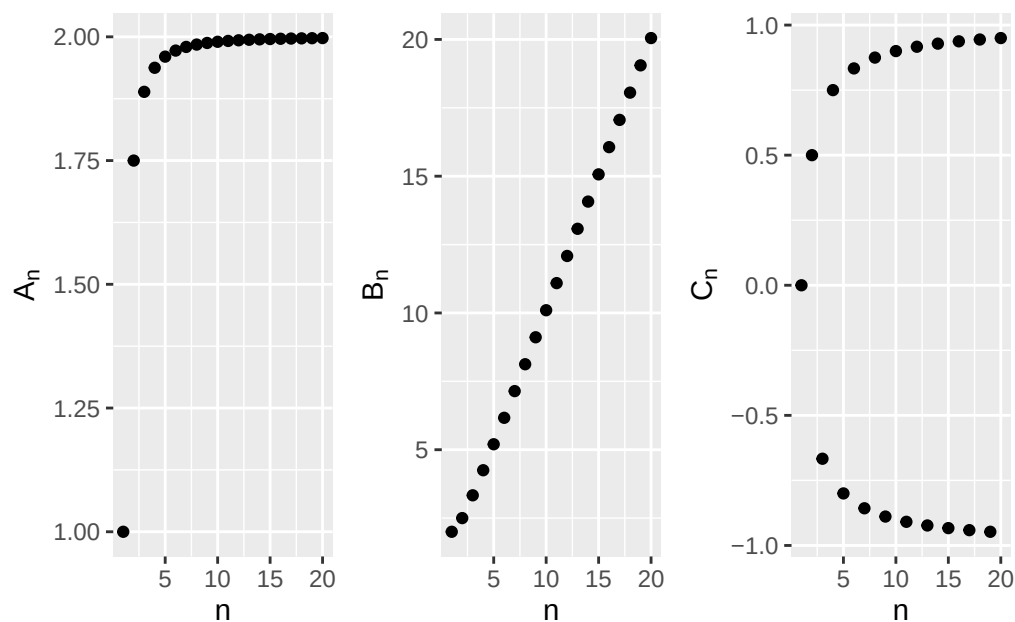


Figure 3.2: Behavior of Some Sequences

3.2 The Limit of a Sequence

The notion of “converging to a limit” is the behavior of the points in Example -@#exm-seqbehav. In some sense, that’s the counterfactual we want to know. What happens as $n \rightarrow \infty$?

1. Sequences like 1 above that converge to a limit.
2. Sequences like 2 above that increase without bound.
3. Sequences like 3 above that neither converge nor increase without bound — alternating over the number line.

Definition: Limit The sequence $\{y_n\}$ has the limit L , which we write as $\lim_{n \rightarrow \infty} y_n = L$, if for any $\epsilon > 0$ there is an integer N (which depends on ϵ) with the property that $|y_n - L| < \epsilon$ for each $n > N$. $\{y_n\}$ is said to converge to L . If the above does not hold, then $\{y_n\}$ diverges.

We can also express the behavior of a sequence as bounded or not:

1. Bounded: if $|y_n| \leq K$ for all n
2. Monotonically Increasing: $y_{n+1} > y_n$ for all n
3. Monotonically Decreasing: $y_{n+1} < y_n$ for all n

A limit is *unique*: If $\{y_n\}$ converges, then the limit L is unique.

If a sequence converges, then the sum of such sequences also converges. Let $\lim_{n \rightarrow \infty} y_n = y$ and $\lim_{n \rightarrow \infty} z_n = z$. Then

1. $\lim_{n \rightarrow \infty} [ky_n + \ell z_n] = ky + \ell z$
2. $\lim_{n \rightarrow \infty} y_n z_n = yz$
3. $\lim_{n \rightarrow \infty} \frac{y_n}{z_n} = \frac{y}{z}$, provided $z \neq 0$

This looks reasonable enough. The harder question, obviously is when the parts of the fraction *don't* converge. If $\lim_{n \rightarrow \infty} y_n = \infty$ and $\lim_{n \rightarrow \infty} z_n = \infty$, What is $\lim_{n \rightarrow \infty} y_n - z_n$? What is $\lim_{n \rightarrow \infty} \frac{y_n}{z_n}$?

It is nice for a sequence to converge in limit. We want to know if complex-looking sequences converge or not. The name of the game here is to break that complex sequence up into sums of simple fractions where n only appears in the denominator: $\frac{1}{n}$, $\frac{1}{n^2}$, and so on. Each of these will converge to 0, because the denominator gets larger and larger. Then, because of the properties above, we can then find the final sequence.

Example 3.2.

Ratios

Find the limit of $\lim_{n \rightarrow \infty} \frac{n+3}{n}$

Solution. At first glance, $n+3$ and n both grow to ∞ , so it looks like we need to divide infinity by infinity. However, we can express this fraction as a sum, then the limits apply separately:

$$\lim_{n \rightarrow \infty} \frac{n+3}{n} = \lim_{n \rightarrow \infty} \left(1 + \frac{3}{n} \right) = \underbrace{\lim_{n \rightarrow \infty} 1}_1 + \underbrace{\lim_{n \rightarrow \infty} \left(\frac{3}{n} \right)}_0$$

so, the limit is actually 1.

After some practice, the key to intuition is whether one part of the fraction grows “faster” than another. If the denominator grows faster to infinity than the numerator, then the fraction will converge to 0, even if the numerator will also increase to infinity. In a sense, limits show how not all infinities are the same.

Exercise 3.1.

Limits

Find the following limits of sequences, then explain in English the intuition for why that is the case.

1. $\lim_{n \rightarrow \infty} \frac{2n}{n^2+1}$
2. $\lim_{n \rightarrow \infty} (n^3 - 100n^2)$

3.3 Limits of a Function

We've now covered functions and just covered limits of sequences, so now is the time to combine the two.

A function f is a compact representation of some behavior we care about. Like for sequences, we often want to know if $f(x)$ approaches some number L as its independent variable x moves to some number c (which is usually 0 or $\pm\infty$). If it does, we say that the limit of $f(x)$, as x approaches c , is L : $\lim_{x \rightarrow c} f(x) = L$. Unlike a sequence, x is a continuous number, and we can move in decreasing order as well as increasing.

For a limit L to exist, the function $f(x)$ must approach L from both the left (increasing) and the right (decreasing).

Definition 3.1.

Limits of a function

Let $f(x)$ be defined at each point in some open interval containing the point c . Then L equals $\lim_{x \rightarrow c} f(x)$ if for any (small positive) number ϵ , there exists a corresponding number $\delta > 0$ such that if $0 < |x - c| < \delta$, then $|f(x) - L| < \epsilon$.

A neat, if subtle result is that $f(x)$ does not necessarily have to be defined at c for $\lim_{x \rightarrow c}$ to exist.

Properties: Let f and g be functions with $\lim_{x \rightarrow c} f(x) = k$ and $\lim_{x \rightarrow c} g(x) = \ell$.

1. $\lim_{x \rightarrow c} [f(x) + g(x)] = \lim_{x \rightarrow c} f(x) + \lim_{x \rightarrow c} g(x)$
2. $\lim_{x \rightarrow c} kf(x) = k \lim_{x \rightarrow c} f(x)$
3. $\lim_{x \rightarrow c} f(x)g(x) = \left[\lim_{x \rightarrow c} f(x) \right] \cdot \left[\lim_{x \rightarrow c} g(x) \right]$
4. $\lim_{x \rightarrow c} \frac{f(x)}{g(x)} = \frac{\lim_{x \rightarrow c} f(x)}{\lim_{x \rightarrow c} g(x)}$, provided $\lim_{x \rightarrow c} g(x) \neq 0$.

Simple limits of functions can be solved as we did limits of sequences. Just be careful which part of the function is changing.

Example 3.3.

Limits of a function

Find the limit of the following functions.

1. $\lim_{x \rightarrow c} k$
2. $\lim_{x \rightarrow c} x$
3. $\lim_{x \rightarrow 2} (2x - 3)$
4. $\lim_{x \rightarrow c} x^n$

Limits can get more complex in roughly two ways. First, the functions may become large polynomials with many moving pieces. Second, the functions may become discontinuous.

The function can be thought of as a more general or “smooth” version of sequences. For example,

Example 3.4.

Limits of ratios

Find the limit of

$$\lim_{x \rightarrow \infty} \frac{(x^4 + 3x - 99)(2 - x^5)}{(18x^7 + 9x^6 - 3x^2 - 1)(x + 1)}$$

Now, the functions will become a bit more complex:

Exercise 3.2.

Limits of a function

Solve the following limits of functions

1. $\lim_{x \rightarrow 0} |x|$
2. $\lim_{x \rightarrow 0} \left(1 + \frac{1}{x^2}\right)$

So there are a few more alternatives about what a limit of a function could be:

1. Right-hand limit: The value approached by $f(x)$ when you move from right to left.
2. Left-hand limit: The value approached by $f(x)$ when you move from left to right.
3. Infinity: The value approached by $f(x)$ as x grows infinitely large. Sometimes this may be a number; sometimes it might be ∞ or $-\infty$.
4. Negative infinity: The value approached by $f(x)$ as x grows infinitely negative. Sometimes this may be a number; sometimes it might be ∞ or $-\infty$.

The distinction between left and right becomes important when the function is not determined for some values of x . What are those cases in the examples below?

3.4 Continuity

To repeat a finding from the limits of functions: $f(x)$ does not necessarily have to be defined at c for $\lim_{x \rightarrow c}$ to exist. Functions that have breaks in their lines are called discontinuous. Functions that have no breaks are called continuous. Continuity is a concept that is more fundamental to, but related to that of “differentiability”, which we will cover next in calculus.

Definition 3.2.

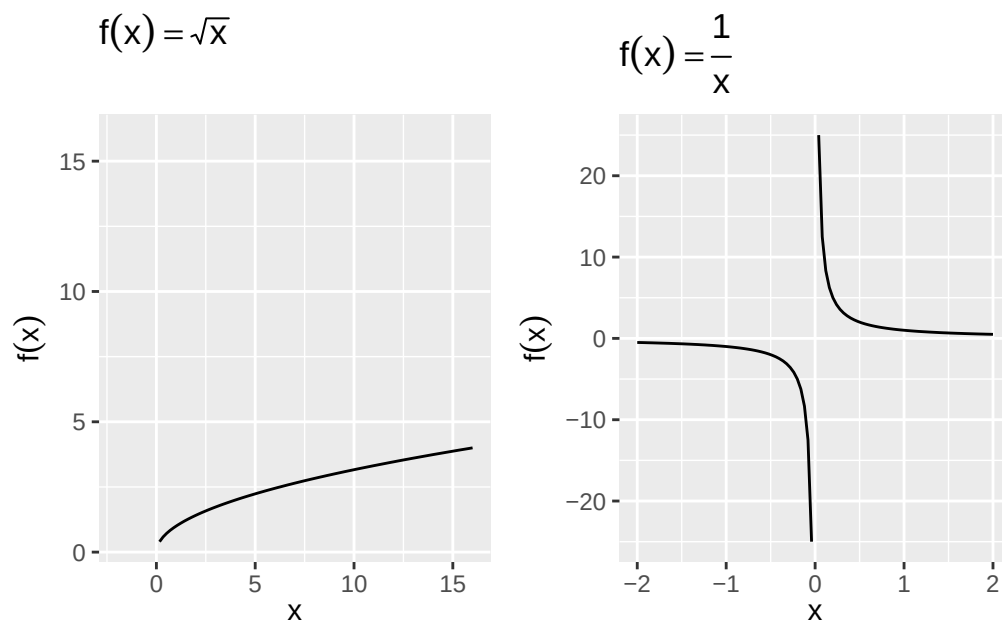


Figure 3.3: Functions which are not defined in some areas

Continuity

Suppose that the domain of the function f includes an open interval containing the point c . Then f is continuous at c if $\lim_{x \rightarrow c} f(x)$ exists and if $\lim_{x \rightarrow c} f(x) = f(c)$. Further, f is continuous on an open interval (a, b) if it is continuous at each point in the interval.

To prove that a function is continuous for all points is beyond this practical introduction to math, but the general intuition can be grasped by graphing.

Example 3.5.

Continuity

For each function, determine if it is continuous or discontinuous.

1. $f(x) = \sqrt{x}$
2. $f(x) = e^x$
3. $f(x) = 1 + \frac{1}{x^2}$
4. $f(x) = \text{floor}(x)$.

The floor is the smaller of the two integers bounding a number. So $\text{floor}(x = 2.999) = 2$, $\text{floor}(x = 2.0001) = 2$, and $\text{floor}(x = 2) = 2$.

Solution. In Figure 3.4, we can see that the first two functions are continuous, and the next two are discontinuous. $f(x) = 1 + \frac{1}{x^2}$ is discontinuous at $x = 0$, and $f(x) = \text{floor}(x)$ is discontinuous at each whole number.

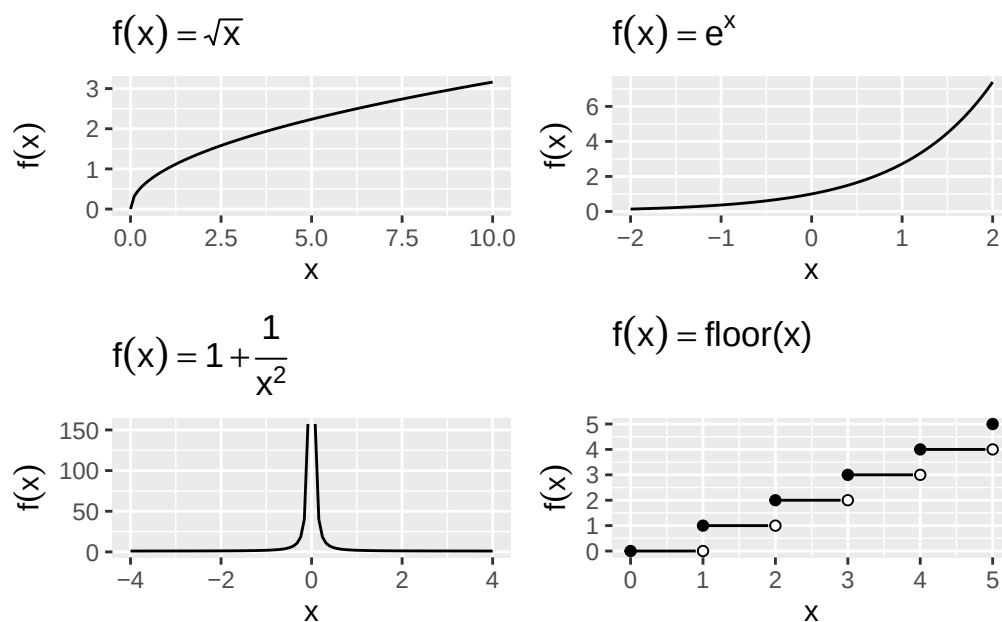


Figure 3.4: Continuous and Discontinuous Functions

Some properties of continuous functions:

1. If f and g are continuous at point c , then $f + g$, $f - g$, $f \cdot g$, $|f|$, and αf are continuous at point c also. f/g is continuous, provided $g(c) \neq 0$.
2. Boundedness: If f is continuous on the closed bounded interval $[a, b]$, then there is a number K such that $|f(x)| \leq K$ for each x in $[a, b]$.
3. Max/Min: If f is continuous on the closed bounded interval $[a, b]$, then f has a maximum and a minimum on $[a, b]$. They may be located at the end points.

Exercise

Let $f(x) = \frac{x^2+2x}{x}$.

1. Graph the function. Is it defined everywhere?
2. What is the functions limit at $x \rightarrow 0$?

4 Calculus

Calculus is a fundamental part of any type of statistics exercise. Although you may not be taking derivatives and integral in your daily work as an analyst, calculus undergirds many concepts we use: maximization, expectation, and cumulative probability.

Example: The Mean is a Type of Integral

The average of a quantity is a type of weighted mean, where the potential values are weighted by their likelihood, loosely speaking. The integral is actually a general way to describe this weighted average when there are conceptually an infinite number of potential values.

If X is a continuous random variable, its expected value $E(X)$ – the center of mass – is given by

$$E(X) = \int_{-\infty}^{\infty} xf(x)dx$$

where $f(x)$ is the probability density function of X .

This is a continuous version of the case where X is discrete, in which case

$$E(X) = \sum_{j=1}^{\infty} x_j P(X = x_j)$$

even more concretely, if the potential values of X are finite, then we can write out the expected value as a weighted mean, where the weights is the probability that the value occurs.

$$E(X) = \sum_x \left(\underbrace{x}_{\text{value}} \cdot \underbrace{P(X = x)}_{\text{weight, or PMF}} \right)$$

4.1 Derivatives

The derivative of f at x is its rate of change at x : how much $f(x)$ changes with a change in x . The rate of change is a fraction — rise over run — but because not all lines are straight and the rise over run formula will give us different values depending on the range we examine, we need to take a limit (Section -Chapter 3).

Definition 4.1.

Derivative

Let f be a function whose domain includes an open interval containing the point x . The derivative of f at x is given by

$$\frac{d}{dx}f(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{(x+h) - x} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

There are two main ways to denote a derivative:

- Leibniz Notation: $\frac{d}{dx}(f(x))$
- Prime or Lagrange Notation: $f'(x)$

If $f(x)$ is a straight line, the derivative is the slope. For a curve, the slope changes by the values of x , so the derivative is the slope of the line tangent to the curve at x . See, For example, Figure -Figure 4.1

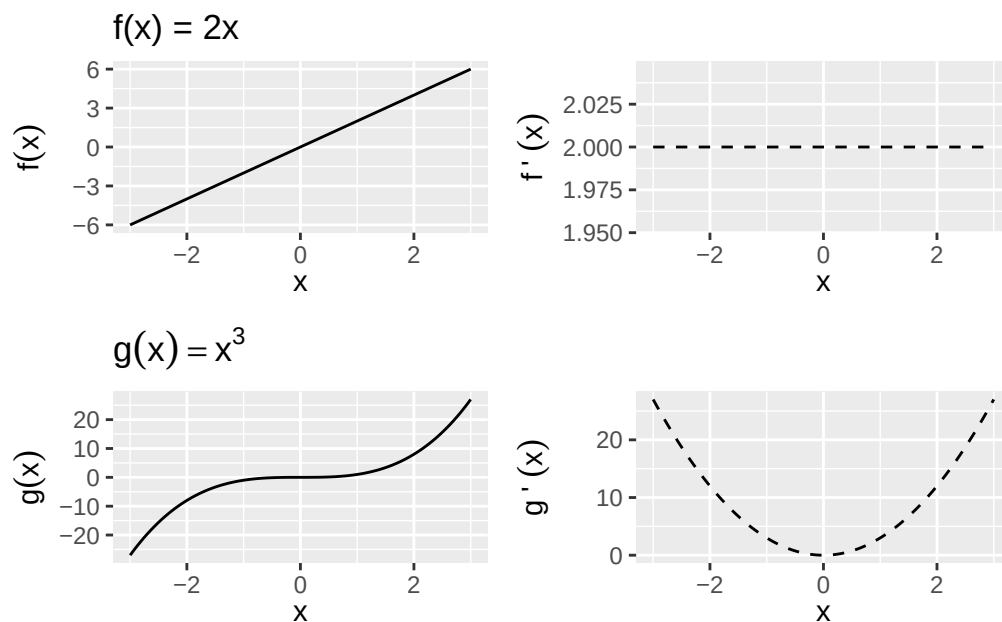


Figure 4.1: The Derivative as a Slope

If $f'(x)$ exists at a point x_0 , then f is said to be **differentiable** at x_0 . That also implies that $f(x)$ is continuous at x_0 .

Properties of derivatives

Suppose that f and g are differentiable at x and that α is a constant. Then the functions $f \pm g$, αf , fg , and f/g (provided $g(x) \neq 0$) are also differentiable at x . Additionally,

Constant rule:

$$[kf(x)]' = kf'(x)$$

Sum rule:

$$[f(x) \pm g(x)]' = f'(x) \pm g'(x)$$

With a bit more algebra, we can apply the definition of derivatives to get a formula for of the derivative of a product and a derivative of a quotient.

Product rule:

$$[f(x)g(x)]' = f'(x)g(x) + f(x)g'(x)$$

Quotient rule:

$$[f(x)/g(x)]' = \frac{f'(x)g(x) - f(x)g'(x)}{[g(x)]^2}, \quad g(x) \neq 0$$

Finally, one way to think of the power of derivatives is that it takes a function a notch down in complexity. The power rule applies to any higher-order function:

Power rule:

$$[x^k]' = kx^{k-1}$$

For any real number k (that is, both whole numbers and fractions). The power rule is proved **by induction**, a neat method of proof used in many fundamental applications to prove that a general statement holds for every possible case, even if there are countably infinite cases. We'll show a simple case where k is an integer here.

Proposition 4.1.

Power Rule

$$[x^k]' = kx^{k-1}$$

for any integer k .

Proof. First, consider the first case (the base case) of $k = 1$. We can show by the definition of derivatives (setting $f(x) = x^1 = x$) that

$$[x^1]' = \lim_{h \rightarrow 0} \frac{(x+h) - x}{(x+h) - x} = 1.$$

Because 1 is also expressed as $1x^{1-1}$, the statement we want to prove holds for the case $k = 1$.

Now, *assume* that the statement holds for some integer m . That is, assume

$$[x^m]' = mx^{m-1}$$

Then, for the case $m + 1$, using the product rule above, we can simplify

$$\begin{aligned} [x^{m+1}]' &= [x^m \cdot x]' \\ &= (x^m)' \cdot x + (x^m) \cdot (x)' \\ &= mx^{m-1} \cdot x + x^m \quad \text{by previous assumption} \\ &= mx^m + x^m \\ &= (m+1)x^m \\ &= (m+1)x^{(m+1)-1} \end{aligned}$$

Therefore, the rule holds for the case $k = m + 1$ once we have assumed it holds for $k = m$. Combined with the first case, this completes proof by induction – we have now proved that the statement holds for all integers $k = 1, 2, 3, \dots$.

To show that it holds for real fractions as well, we can prove expressing that exponent by a fraction of two integers.

□

These “rules” become apparent by applying the definition of the derivative above to each of the things to be “derived”, but these come up so frequently that it is best to repeat until it is muscle memory.

Exercise 4.1.

Derivatives

For each of the following functions, find the first-order derivative $f'(x)$.

1. $f(x) = c$
2. $f(x) = x$
3. $f(x) = x^2$
4. $f(x) = x^3$
5. $f(x) = \frac{1}{x^2}$
6. $f(x) = (x^3)(2x^4)$
7. $f(x) = x^4 - x^3 + x^2 - x + 1$
8. $f(x) = (x^2 + 1)(x^3 - 1)$
9. $f(x) = 3x^2 + 2x^{1/3}$
10. $f(x) = \frac{x^2+1}{x^2-1}$

4.2 Higher-Order Derivatives (Derivatives of Derivatives of Derivatives)

The first derivative is applying the definition of derivatives on the function, and it can be expressed as

$$f'(x), \quad y', \quad \frac{d}{dx}f(x), \quad \frac{dy}{dx}$$

We can keep applying the differentiation process to functions that are themselves derivatives. The derivative of $f'(x)$ with respect to x , would then be

$$f''(x) = \lim_{h \rightarrow 0} \frac{f'(x+h) - f'(x)}{h}$$

and we can therefore call it the **Second derivative**:

$$f''(x), \quad y'', \quad \frac{d^2}{dx^2}f(x), \quad \frac{d^2y}{dx^2}$$

Similarly, the derivative of $f''(x)$ would be called the third derivative and is denoted $f'''(x)$. And by extension, the **nth derivative** is expressed as $\frac{d^n}{dx^n}f(x)$, $\frac{d^ny}{dx^n}$.

Example 4.1.

Succession of derivatives

$$\begin{aligned}f(x) &= x^3 \\f'(x) &= 3x^2 \\f''(x) &= 6x \\f'''(x) &= 6 \\f''''(x) &= 0\end{aligned}$$

Earlier, in Section -Section 4.1, we said that if a function differentiable at a given point, then it must be continuous. Further, if $f'(x)$ is itself continuous, then $f(x)$ is called continuously differentiable. All of this matters because many of our findings about optimization (Section @ref(optim)) rely on differentiation, and so we want our function to be differentiable in as many layers. A function that is continuously differentiable infinitely is called “smooth”. Some examples: $f(x) = x^2$, $f(x) = e^x$.

4.3 Composite Functions and the Chain Rule

As useful as the above rules are, many functions you’ll see won’t fit neatly in each case immediately. Instead, they will be functions of functions. For example, the difference between $x^2 + 1^2$ and $(x^2 + 1)^2$ may look trivial, but the sum rule can be easily applied to the former, while it’s actually not obvious what to do with the latter.

Composite functions are formed by substituting one function into another and are denoted by

$$(f \circ g)(x) = f[g(x)].$$

To form $f[g(x)]$, the range of g must be contained (at least in part) within the domain of f . The domain of $f \circ g$ consists of all the points in the domain of g for which $g(x)$ is in the domain of f .

Example 4.2.

Composite functions

Let $f(x) = \log x$ for $0 < x < \infty$ and $g(x) = x^2$ for $-\infty < x < \infty$.

Then

$$(f \circ g)(x) = \log x^2, -\infty < x < \infty - \{0\}$$

Also

$$(g \circ f)(x) = [\log x]^2, 0 < x < \infty$$

Notice that $f \circ g$ and $g \circ f$ are not the same functions.

With the notation of composite functions in place, now we can introduce a helpful additional rule that will deal with a derivative of composite functions as a chain of concentric derivatives.

Chain Rule:

Let $y = (f \circ g)(x) = f[g(x)]$. The derivative of y with respect to x is

$$\frac{d}{dx}\{f[g(x)]\} = f'[g(x)]g'(x)$$

We can read this as: “the derivative of the composite function y is the derivative of f evaluated at $g(x)$, times the derivative of g .”

The chain rule can be thought of as the derivative of the “outside” times the derivative of the “inside”, remembering that the derivative of the outside function is evaluated at the value of the inside function.

- The chain rule can also be written as

$$\frac{dy}{dx} = \frac{dy}{dg(x)} \frac{dg(x)}{dx}$$

This expression does not imply that the $dg(x)$ ’s cancel out, as in fractions. They are part of the derivative notation and you can’t separate them out or cancel them.)

Example 4.3.

Composite Exponent

Find $f'(x)$ for $f(x) = (3x^2 + 5x - 7)^6$.

The direct use of a chain rule is when the exponent of is itself a function, so the power rule could not have applied generally:

Generalized Power Rule:

If $f(x) = [g(x)]^p$ for any rational number p ,

$$f'(x) = p[g(x)]^{p-1}g'(x)$$

4.4 Derivatives of natural logs and the exponent

Natural logs and exponents (they are inverses of each other; see Section @ref(logexponents)) crop up everywhere in statistics. Their derivative is a special case from the above, but quite elegant.

Theorem 4.1.

Derivative of Exponents/Logs

The functions e^x and the natural logarithm $\log(x)$ are continuous and differentiable in their domains, and their first derivative is

$$(e^x)' = e^x$$

$$\log(x)' = \frac{1}{x}$$

Also, when these are composite functions, it follows by the generalized power rule that

$$(e^{g(x)})' = e^{g(x)} \cdot g'(x)$$

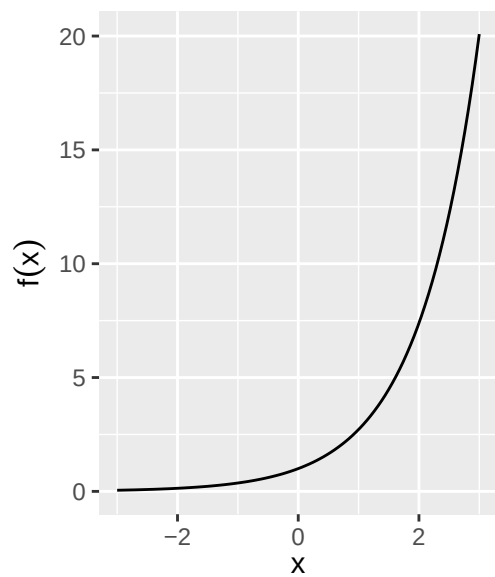
$$(\log g(x))' = \frac{g'(x)}{g(x)}, \quad \text{if } g(x) > 0$$

Derivatives of natural exponential function (e)

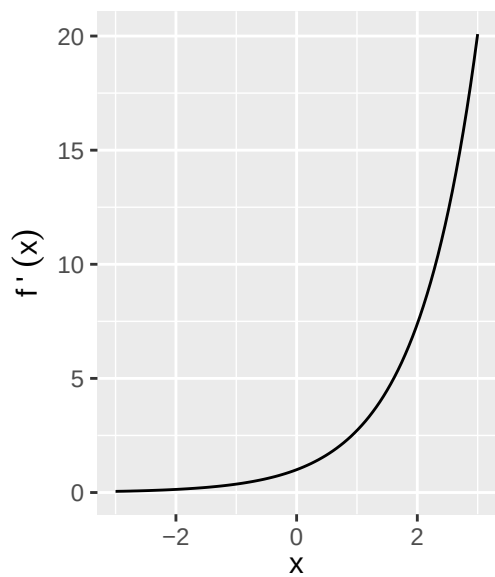
To repeat the main rule in Theorem @ref(thm:derivexplog), the intuition is that

1. Derivative of e^x is itself: $\frac{d}{dx}e^x = e^x$ (See Figure 4.2)
2. Same thing if there were a constant in front: $\frac{d}{dx}\alpha e^x = \alpha e^x$
3. Same thing no matter how many derivatives there are in front: $\frac{d^n}{dx^n}\alpha e^x = \alpha e^x$
4. Chain Rule: When the exponent is a function of x , remember to take derivative of that function and add to product. $\frac{d}{dx}e^{g(x)} = e^{g(x)}g'(x)$

Example 4.4.



$$f(x) = e^x$$



$$f'(x) = e^x$$

Figure 4.2: Derivative of the Exponential Function

Derivatives of exponents

Find the derivative for the following.

1. $f(x) = e^{-3x}$
2. $f(x) = e^{x^2}$
3. $f(x) = (x - 1)e^x$

Derivatives of logarithms

The natural log is the mirror image of the natural exponent and has mirroring properties, again, to repeat the theorem,

1. log prime x is one over x: $\frac{d}{dx} \log x = \frac{1}{x}$ (Figure 4.3)
2. Exponents become multiplicative constants: $\frac{d}{dx} \log x^k = \frac{d}{dx} k \log x = \frac{k}{x}$
3. Chain rule again: $\frac{d}{dx} \log u(x) = \frac{u'(x)}{u(x)}$
4. For any positive base b , $\frac{d}{dx} b^x = (\log b) (b^x)$.

Example 4.5.

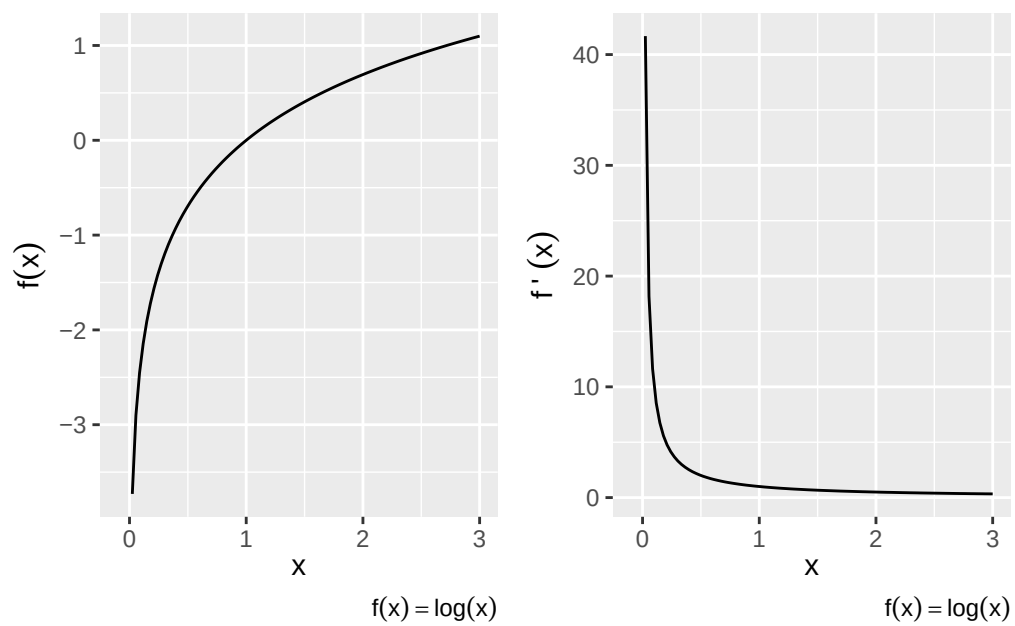


Figure 4.3: Derivative of the Natural Log

Derivatives of logs

Find dy/dx for the following.

1. $f(x) = \log(x^2 + 9)$
2. $f(x) = \log(\log x)$
3. $f(x) = (\log x)^2$
4. $f(x) = \log e^x$

Outline of Proof

We actually show the derivative of the log first, and then the derivative of the exponential naturally follows.

The general derivative of the log at any base a is solvable by the definition of derivatives.

$$(\log_a x)' = \lim_{h \rightarrow 0} \frac{1}{h} \log_a \left(1 + \frac{h}{x}\right)$$

Re-express $g = \frac{h}{x}$ and get

$$\begin{aligned} (\log_a x)' &= \frac{1}{x} \lim_{g \rightarrow 0} \log_a (1 + g)^{\frac{1}{g}} \\ &= \frac{1}{x} \log_a e \end{aligned}$$

By definition of e . As a special case, when $a = e$, then $(\log x)' = \frac{1}{x}$.

Now let's think about the inverse, taking the derivative of $y = a^x$.

$$\begin{aligned} y &= a^x \\ \Rightarrow \log y &= x \log a \\ \Rightarrow \frac{y'}{y} &= \log a \\ \Rightarrow y' &= y \log a \end{aligned}$$

Then in the special case where $a = e$,

$$(e^x)' = (e^x)$$

4.5 Partial Derivatives

What happens when there's more than variable that is changing?

If you can do ordinary derivatives, you can do partial derivatives: just hold all the other input variables constant except for the one you're differentiating with respect to. (Joe Blitzstein's Math Notes)

Suppose we have a function f now of two (or more) variables and we want to determine the rate of change relative to one of the variables. To do so, we would find its partial derivative, which is defined similar to the derivative of a function of one variable.

Partial Derivative: Let f be a function of the variables $(x_1, \dots, x_n)$. The partial derivative of f with respect to x_i is

$$\frac{\partial f}{\partial x_i}(x_1, \dots, x_n) = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_i + h, \dots, x_n) - f(x_1, \dots, x_i, \dots, x_n)}{h}$$

Only the i th variable changes — the others are treated as constants.

We can take higher-order partial derivatives, like we did with functions of a single variable, except now the higher-order partials can be with respect to multiple variables.

Example 4.6.

Partial derivatives

Notice that you can take partials with regard to different variables.

Suppose $f(x, y) = x^2 + y^2$. Then

$$\frac{\partial f}{\partial x}(x, y) =$$

$$\frac{\partial f}{\partial y}(x, y) =$$

$$\frac{\partial^2 f}{\partial x^2}(x, y) =$$

$$\frac{\partial^2 f}{\partial x \partial y}(x, y) =$$

Exercise 4.2.

Partial derivatives

Let $f(x, y) = x^3y^4 + e^x - \log y$. What are the following partial derivatives?

$$\begin{aligned}\frac{\partial f}{\partial x}(x, y) &= \\ \frac{\partial f}{\partial y}(x, y) &= \\ \frac{\partial^2 f}{\partial x^2}(x, y) &= \\ \frac{\partial^2 f}{\partial x \partial y}(x, y) &= \end{aligned}$$

4.6 Taylor Series Approximation

A common form of approximation used in statistics involves derivatives. A Taylor series is a way to represent common functions as infinite series (a sum of infinite elements) of the function's derivatives at some point a .

For example, Taylor series are very helpful in representing nonlinear (read: difficult) functions as linear (read: manageable) functions. One can thus **approximate** functions by using lower-order, finite series known as **Taylor polynomials**. If $a = 0$, the series is called a Maclaurin series.

Specifically, a Taylor series of a real or complex function $f(x)$ that is infinitely differentiable in the neighborhood of point a is:

$$\begin{aligned}f(x) &= f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \dots \\ &= \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!}(x-a)^n\end{aligned}$$

Taylor Approximation: We can often approximate the curvature of a function $f(x)$ at point a using a 2nd order Taylor polynomial around point a :

$$f(x) = f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + R_2$$

R_2 is the remainder (R for remainder, 2 for the fact that we took two derivatives) and often treated as negligible, giving us:

$$f(x) \approx f(a) + f'(a)(x-a) + \frac{f''(a)}{2}(x-a)^2$$

The more derivatives that are added, the smaller the remainder R and the more accurate the approximation. Proofs involving limits guarantee that the remainder converges to 0 as the order of derivation increases.

4.7 The Indefinite Integration

So far, we've been interested in finding the derivative $f = F'$ of a function F . However, sometimes we're interested in exactly the reverse: finding the function F for which f is its derivative. We refer to F as the antiderivative of f .

Definition 4.2.

Antiderivative

The antiderivative of a function $f(x)$ is a differentiable function F whose derivative is f .

$$F' = f.$$

Another way to describe is through the inverse formula. Let DF be the derivative of F . And let $DF(x)$ be the derivative of F evaluated at x . Then the antiderivative is denoted by D^{-1} (i.e., the inverse derivative). If $DF = f$, then $F = D^{-1}f$.

This definition bolsters the main takeaway about integrals and derivatives: They are inverses of each other.

Exercise 4.3.

Antiderivative

Find the antiderivative of the following:

1. $f(x) = \frac{1}{x^2}$
2. $f(x) = 3e^{3x}$

We know from derivatives how to manipulate F to get f . But how do you express the procedure to manipulate f to get F ? For that, we need a new symbol, which we will call indefinite integration.

:::{#def-indefint}

4.8 Indefinite Integral

The indefinite integral of $f(x)$ is written

$$\int f(x)dx$$

and is equal to the antiderivative of f .

Example 4.7.

Graphing

Draw the function $f(x)$ and its indefinite integral, $\int f(x)dx$

$$f(x) = (x^2 - 4)$$

Solution. The Indefinite Integral of the function $f(x) = (x^2 - 4)$ can, for example, be $F(x) = \frac{1}{3}x^3 - 4x$. But it can also be $F(x) = \frac{1}{3}x^3 - 4x + 1$, because the constant 1 disappears when taking the derivative.

Some of these functions are plotted in the bottom panel of Figure 4.4 as dotted lines.

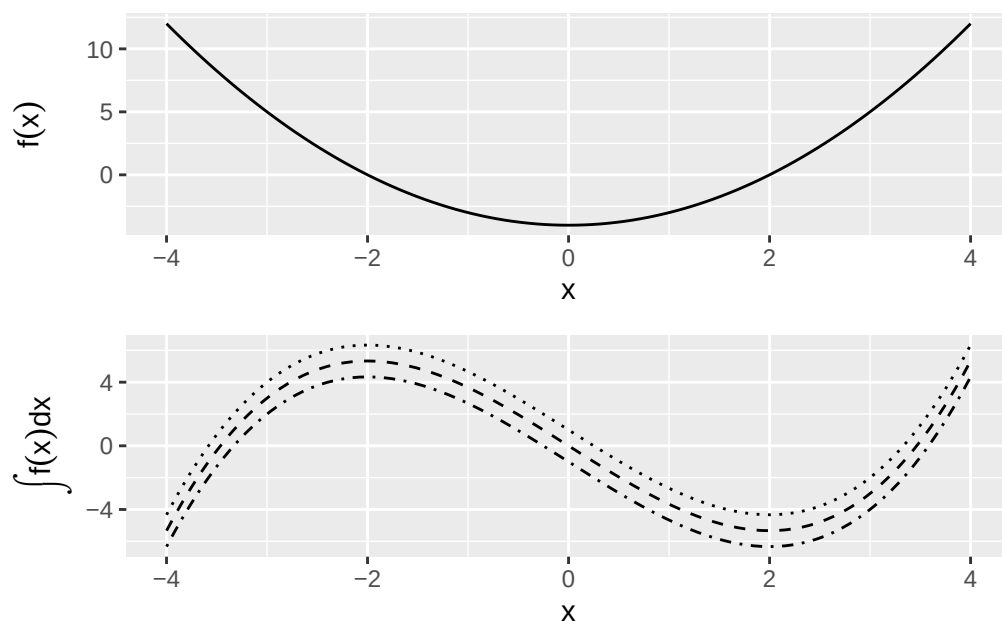


Figure 4.4: The Many Indefinite Integrals of a Function

Notice from these examples that while there is only a single derivative for any function, there are multiple antiderivatives: one for any arbitrary constant c . c just shifts the curve up or down on the y -axis. If more information is present about the antiderivative — e.g., that it passes through a particular point — then we can solve for a specific value of c .

Common Rules of Integration

Some common rules of integrals follow by virtue of being the inverse of a derivative.

1. Constants are allowed to slip out: $\int af(x)dx = a \int f(x)dx$
2. Integration of the sum is sum of integrations: $\int [f(x) + g(x)]dx = \int f(x)dx + \int g(x)dx$
3. Reverse Power-rule: $\int x^n dx = \frac{1}{n+1}x^{n+1} + c$
4. Exponents are still exponents: $\int e^x dx = e^x + c$
5. Recall the derivative of $\log(x)$ is one over x , and so: $\int \frac{1}{x} dx = \log x + c$
6. Reverse chain-rule: $\int e^{f(x)} f'(x) dx = e^{f(x)} + c$
7. More generally: $\int [f(x)]^n f'(x) dx = \frac{1}{n+1} [f(x)]^{n+1} + c$
8. Remember the derivative of a log of a function: $\int \frac{f'(x)}{f(x)} dx = \log f(x) + c$

Example 4.8.

Common Integration

Simplify the following indefinite integrals:

- $\int 3x^2 dx$
- $\int (2x+1)dx$
- $\int e^x e^{e^x} dx$

4.9 The Definite Integral: The Area under the Curve

If there is an indefinite integral, there *must* be a definite integral. Indeed there is, but the notion of definite integrals comes from a different objective: finding the area under a function. We will find, perhaps remarkably, that the formula we find to get the sum turns out to be expressible by the anti-derivative.

Suppose we want to determine the area $A(R)$ of a region R defined by a curve $f(x)$ and some interval $a \leq x \leq b$.

One way to calculate the area would be to divide the interval $a \leq x \leq b$ into n subintervals of length Δx and then approximate the region with a series of rectangles, where the base of each rectangle is Δx and the height is $f(x)$ at the midpoint of that interval. $A(R)$ would then be approximated by the area of the union of the rectangles, which is given by

$$S(f, \Delta x) = \sum_{i=1}^n f(x_i) \Delta x$$

and is called a **Riemann sum**.

As we decrease the size of the subintervals Δx , making the rectangles “thinner,” we would expect our approximation of the area of the region to become closer to the true area. This allows us to express the area as a limit of a series:

$$A(R) = \lim_{\Delta x \rightarrow 0} \sum_{i=1}^n f(x_i) \Delta x$$

Figure 4.5 shows that illustration. The curve depicted is $f(x) = -15(x-5) + (x-5)^3 + 50$. We want to approximate the area under the curve between the x values of 0 and 10. We can do

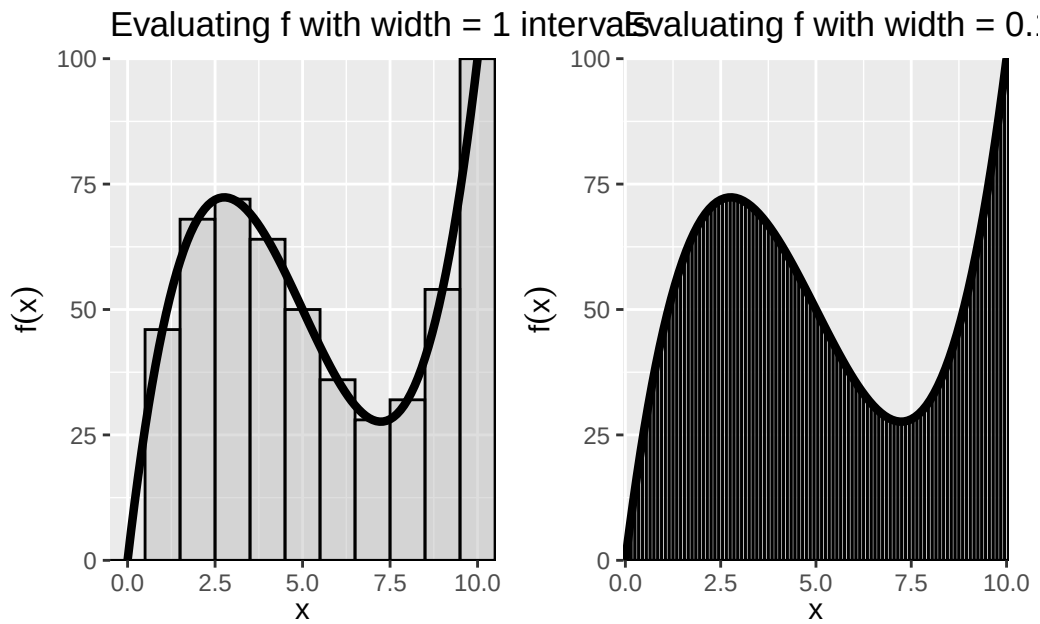


Figure 4.5: The Riemann Integral as a Sum of Evaluations

this in blocks of arbitrary width, where the sum of rectangles (the area of which is width times $f(x)$ evaluated at the midpoint of the bar) shows the Riemann Sum. As the width of the bars Δx becomes smaller, the better the estimate of $A(R)$.

This is how we define the “Definite” Integral:

Definition 4.3.

The Definite Integral (Riemann)

If for a given function f the Riemann sum approaches a limit as $\Delta x \rightarrow 0$, then that limit is called the Riemann integral of f from a to b . We express this with the $\int$, symbol, and write

$$\int_a^b f(x)dx = \lim_{\Delta x \rightarrow 0} \sum_{i=1}^n f(x_i)\Delta x$$

The most straightforward of a definite integral is the definite integral. That is, we read

$$\int_a^b f(x)dx$$

as the definite integral of f from a to b and we defined as the area under the “curve” $f(x)$ from point $x = a$ to $x = b$.

The fundamental theorem of calculus shows us that this sum is, in fact, the antiderivative.

Theorem 4.2.

First Fundamental Theorem of Calculus

Let the function f be bounded on $[a, b]$ and continuous on (a, b) . Then, suggestively, use the symbol $F(x)$ to denote the definite integral from a to x :

$$F(x) = \int_a^x f(t)dt, \quad a \leq x \leq b$$

Then $F(x)$ has a derivative at each point in (a, b) and

$$F'(x) = f(x), \quad a < x < b$$

That is, the definite integral function of f is the one of the antiderivatives of some f .

This is again a long way of saying that that differentiation is the inverse of integration. But now, we've covered definite integrals.

The second theorem gives us a simple way of computing a definite integral as a function of indefinite integrals.

Theorem 4.3.

Second Fundamental Theorem of Calculus

Let the function f be bounded on $[a, b]$ and continuous on (a, b) . Let F be any function that is continuous on $[a, b]$ such that $F'(x) = f(x)$ on (a, b) . Then

$$\int_a^b f(x)dx = F(b) - F(a)$$

So the procedure to calculate a simple definite integral $\int_a^b f(x)dx$ is then

1. Find the indefinite integral $F(x)$.
2. Evaluate $F(b) - F(a)$.

Example 4.9.

Definite Integral of a monomial

Solve $\int_1^3 3x^2 dx$.

Let $f(x) = 3x^2$.

Exercise 4.4.

Indefinite integrals

What is the value of $\int_{-2}^2 e^x e^{e^x} dx$?

Common Rules for Definite Integrals

The area-interpretation of the definite integral provides some rules for simplification.

1. There is no area below a point:

$$\int_a^a f(x) dx = 0$$

2. Reversing the limits changes the sign of the integral:

$$\int_a^b f(x) dx = - \int_b^a f(x) dx$$

3. Sums can be separated into their own integrals:

$$\int_a^b [\alpha f(x) + \beta g(x)] dx = \alpha \int_a^b f(x) dx + \beta \int_a^b g(x) dx$$

4. Areas can be combined as long as limits are linked:

$$\int_a^b f(x) dx + \int_b^c f(x) dx = \int_a^c f(x) dx$$

Exercise 4.5.

Definite integrals

Simplify the following definite integrals.

$$1. \int_1^1 3x^2 dx =$$

$$2. \int_0^4 (2x + 1) dx =$$

$$3. \int_{-2}^0 e^x e^{e^x} dx + \int_0^2 e^x e^{e^x} dx =$$

4.10 Integration by Substitution

From the second fundamental theorem of calculus, we now that a quick way to get a definite integral is to first find the indefinite integral, and then just plug in the bounds.

Sometimes the integrand (the thing that we are trying to take an integral of) doesn't appear integrable using common rules and antiderivatives. A method one might try is **integration by substitution**, which is related to the Chain Rule.

Suppose we want to find the indefinite integral

$$\int g(x) dx$$

but $g(x)$ is complex and none of the formulas we have seen so far seem to apply immediately. The trick is to come up with a *new* function $u(x)$ such that

$$g(x) = f[u(x)]u'(x).$$

Why does an introduction of yet another function end of simplifying things? Let's refer to the antiderivative of f as F . Then the chain rule tells us that

$$\frac{d}{dx} F[u(x)] = f[u(x)]u'(x)$$

. So, $F[u(x)]$ is the antiderivative of g . We can then write

$$\int g(x) dx = \int f[u(x)]u'(x) dx = \int \frac{d}{dx} F[u(x)] dx = F[u(x)] + c$$

To summarize, the procedure to determine the indefinite integral $\int g(x)dx$ by the method of substitution:

1. Identify some part of $g(x)$ that might be simplified by substituting in a single variable u (which will then be a function of x).
2. Determine if $g(x)dx$ can be reformulated in terms of u and du .
3. Solve the indefinite integral.
4. Substitute back in for x

Substitution can also be used to calculate a definite integral. Using the same procedure as above,

$$\int_a^b g(x)dx = \int_c^d f(u)du = F(d) - F(c)$$

where $c = u(a)$ and $d = u(b)$.

Example 4.10. Integration by Substitution I

Solve the indefinite integral

$$\int x^2\sqrt{x+1}dx.$$

For the above problem, we could have also used the substitution $u = \sqrt{x+1}$. Then $x = u^2 - 1$ and $dx = 2u du$. Substituting these in, we get

$$\int x^2\sqrt{x+1}dx = \int (u^2 - 1)^2 u 2u du$$

which when expanded is again a polynomial and gives the same result as above.

Another case in which integration by substitution is useful is with a fraction.

Example 4.11.

Integration by Substitution II

Simplify

$$\int_0^1 \frac{5e^{2x}}{(1+e^{2x})^{1/3}} dx.$$

4.11 Integration by Parts

Another useful integration technique is **integration by parts**, which is related to the Product Rule of differentiation. The product rule states that

$$\frac{d}{dx}(uv) = u \frac{dv}{dx} + v \frac{du}{dx}$$

Integrating this and rearranging, we get

$$\int u \frac{dv}{dx} dx = uv - \int v \frac{du}{dx} dx$$

or

$$\int u(x)v'(x)dx = u(x)v(x) - \int v(x)u'(x)dx$$

More easily remembered with the mnemonic “Ultraviolet Voodoo”:

$$\int u dv = uv - \int v du$$

where $du = u'(x)dx$ and $dv = v'(x)dx$.

For definite integrals, this is simply

$$\int_a^b u \frac{dv}{dx} dx = uv \Big|_a^b - \int_a^b v \frac{du}{dx} dx$$

Our goal here is to find expressions for u and dv that, when substituted into the above equation, yield an expression that’s more easily evaluated.

Example 4.12.

Integration by parts

Simplify the following integrals. These seemingly obscure forms of integrals come up often when integrating distributions.

$$\int x e^{ax} dx$$

Solution. Let $u = x$ and $\frac{dv}{dx} = e^{ax}$. Then $du = dx$ and $v = (1/a)e^{ax}$. Substituting this into the integration by parts formula, we obtain

$$\begin{aligned}\int x e^{ax} dx &= uv - \int v du \\ &= x \left(\frac{1}{a} e^{ax} \right) - \int \frac{1}{a} e^{ax} dx \\ &= \frac{1}{a} x e^{ax} - \frac{1}{a^2} e^{ax} + c\end{aligned}$$

Exercise 4.6.

Integration by parts

1. Integrate

$$\int x^n e^{ax} dx$$

2. Integrate

$$\int x^3 e^{-x^2} dx$$

5 Optimization

To optimize, we use derivatives and calculus. Optimization is to find the maximum or minimum of a function, and to find what value of an input gives that extremum. This has obvious uses in engineering. Many tools in the statistical toolkit use optimization. One of the most common ways of estimating a model is through “Maximum Likelihood Estimation”, done via optimizing a function (the likelihood).

Optimization also comes up in Economics, Formal Theory, and Political Economy all the time. A go-to model of human behavior is that they optimize a certain utility function. Humans are not pure utility maximizers, of course, but nuanced models of optimization – for example, adding constraints and adding uncertainty – will prove to be quite useful.

Example: Meltzer-Richard

A standard backdrop in comparative political economy, the Meltzer-Richard (1981) model states that redistribution of wealth should be higher in societies where the median income is much smaller than the average income. More to the point, typically income distributions where the median is very different from the average is one of high inequality. In other words, the Meltzer-Richard model says that highly unequal economies will have more re-distribution of wealth. Why is that the case? Here is a simplified example that is not the exact model by Meltzer and Richard¹, but adapted from Persson and Tabellini²

We will set the following things about our model human and model democracy.

- Individuals are indexed by i , and the total population is normalized to unity (“1”) without loss of generality.
- $U(\cdot)$, u for “utility”, is a function that is concave and increasing, and expresses the utility gained from public goods. This tells us that its first derivative is *positive*, and its second derivative is **negative**.
- y_i is the income of person i
- W_i , w for “welfare”, is the welfare of person i
- c_i , c for “consumption”, is the consumption utility of person i

¹Allan H. Meltzer and Scott F. Richard. “[A Rational Theory of the Size of Government](#)”. *Journal of Political Economy* 89:5 (1981), p. 914-927

²Adapted from Torsten Persson and Guido Tabellini, *Political Economics: Explaining Economic Policy*. MIT Press.

Also, the government is democratically elected and sets the following redistribution output:

- τ , t for “tax”, is a flat tax rate between 0 and 1 that is applied to everyone’s income.
- g , “g” for “goods”, is the amount of public goods that the government provides.

Suppose an individual’s welfare is given by:

$$W_i = c_i + U(g)$$

The consumption good is the person’s post-tax income.

$$c_i = (1 - \tau)y_i$$

Income varies by person (In the next section we will cover probability, by then we will know that we can express this by saying that y is a random variable with the cumulative distribution function F , i.e. $y \sim F$). Every distribution has a mean and an median.

- $E(y)$ is the average income of the society.
- $\text{med}(y)$ is the **median income** of the society.

What will happen in this economy? What will the tax rate be set too? How much public goods will be provided?

We’ve skipped ahead of some formal theory results of democracy, but hopefully these are conceptually intuitive. First, if a democracy is competitive, there is no slack in the government’s goods, and all tax revenue becomes a public good. So we can go ahead and set the constraint:

$$g = \sum_i \tau y_i P(y_i) = \tau E(y)$$

We can do this trick because of the “normalizes to unity” setting, but this is a general property of the average.

Now given this constraint we can re-write an individual’s welfare as

$$\begin{aligned} W_i &= \left(1 - \frac{g}{E(y)}\right) y_i + U(g) \\ &= (E(y) - g) \frac{1}{E(y)} y_i + U(g) \\ &= (E(y) - g) \frac{y_i}{E(y)} + U(g) \end{aligned}$$

When is the individual's welfare maximized, **as a function of the public good**?

$$\frac{d}{dg}W_i = -\frac{y_i}{E(y)} + \frac{d}{dg}U(g)$$

$\frac{d}{dg}W_i = 0$ when $\frac{d}{dg}U(g) = \frac{y_i}{E(y)}$, and so after expressing the derivative as $U_g = \frac{d}{dg}U(g)$ for simplicity,

$$g_i^* = U_g^{-1}\left(\frac{y_i}{E(y)}\right)$$

Now recall that because we assumed concavity, U_g is a negative sloping function whose value is positive. It can be shown that the inverse of such a function is also decreasing. Thus an individual's preferred level of government is determined by a single continuum, the person's income divided by the average income, and the function is **decreasing** in y_i . This is consistent with our intuition that richer people prefer less redistribution.

That was the amount for any given person. The government has to set one value of g , however. So what will that be? Now we will use another result, the median voter theorem. This says that under certain general electoral conditions (single-peaked preferences, two parties, majority rule), the policy winner will be that preferred by the median person in the population. Because the only thing that determines a person's preferred level of government is $y_i/E(y)$, we can presume that the median voter, whose income is $\text{med}(y)$ will prevail in their preferred choice of government. Therefore, we will see

$$g^* = U_g^{-1}\left(\frac{\text{med}(y)}{E(y)}\right)$$

What does this say about the level of redistribution we observe in an economy? The higher the average income is than the median income, which often (but not always) means *more* inequality, there should be *more* redistribution.

5.1 Maxima and Minima

The first derivative, $f'(x)$, quantifies the slope of a function. Therefore, it can be used to check whether the function $f(x)$ at the point x is increasing or decreasing at x .

1. **Increasing:** $f'(x) > 0$
2. **Decreasing:** $f'(x) < 0$
3. **Neither increasing nor decreasing:** $f'(x) = 0$ i.e. a maximum, minimum, or saddle point

So for example, $f(x) = x^2 + 2$ and $f'(x) = 2x$

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.

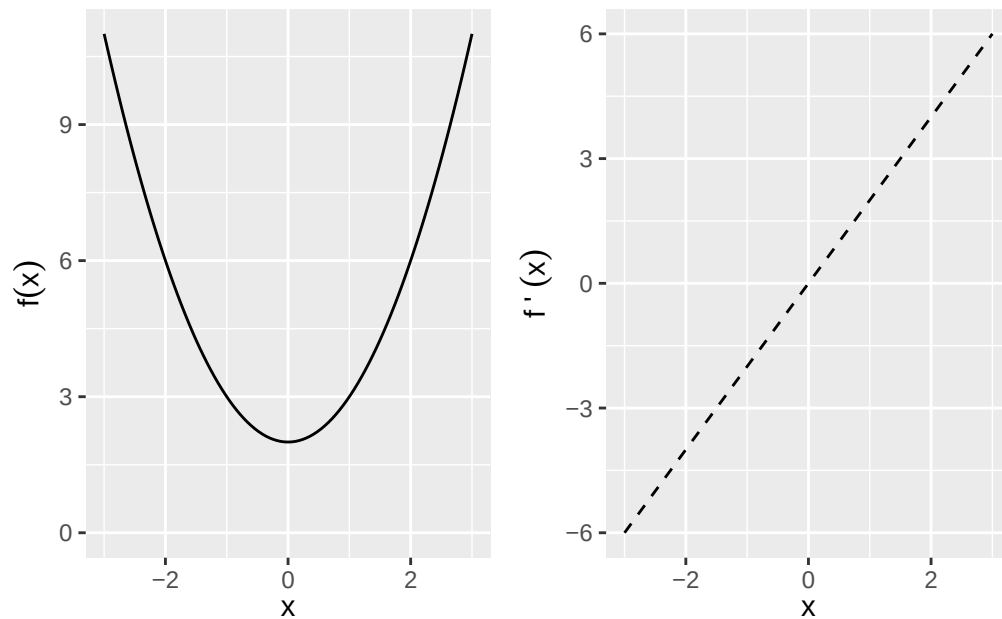


Figure 5.1: Maxima and Minima

Exercise 5.1.

Plotting a maximum and minimum

Plot $f(x) = x^3 + x^2 + 2$, plot its derivative, and identify where the derivative is zero. Is there a maximum or minimum?

The second derivative $f''(x)$ identifies whether the function $f(x)$ at the point x is

1. Concave down: $f''(x) < 0$
2. Concave up (convex): $f''(x) > 0$

Maximum (Minimum): x_0 is a **local maximum (minimum)** if $f(x_0) > f(x)$ ($f(x_0) < f(x)$) for all x within some open interval containing x_0 . x_0 is a **global maximum (minimum)** if $f(x_0) > f(x)$ ($f(x_0) < f(x)$) for all x in the domain of f .

Given the function f defined over domain D , all of the following are defined as **critical points**:

1. Any interior point of D where $f'(x) = 0$.
2. Any interior point of D where $f'(x)$ does not exist.
3. Any endpoint that is in D .

The maxima and minima will be a subset of the critical points.

Second Derivative Test of Maxima/Minima: We can use the second derivative to tell us whether a point is a maximum or minimum of $f(x)$.

1. Local Maximum: $f'(x) = 0$ and $f''(x) < 0$
2. Local Minimum: $f'(x) = 0$ and $f''(x) > 0$
3. Need more info: $f'(x) = 0$ and $f''(x) = 0$

Global Maxima and Minima Sometimes no global max or min exists — e.g., $f(x)$ not bounded above or below. However, there are three situations where we can fairly easily identify global max or min.

1. **Functions with only one critical point.** If x_0 is a local max or min of f and it is the only critical point, then it is the global max or min.
2. **Globally concave up or concave down functions.** If $f''(x)$ is never zero, then there is at most one critical point. That critical point is a global maximum if $f'' < 0$ and a global minimum if $f'' > 0$.

3. **Functions over closed and bounded intervals** must have both a global maximum and a global minimum.

Example 5.1.

Maxima and Minima by drawing

Find any critical points and identify whether they are a max, min, or saddle point:

1. $f(x) = x^2 + 2$
2. $f(x) = x^3 + 2$
3. $f(x) = |x^2 - 1|$, $x \in [-2, 2]$

5.2 Concavity of a Function

Concavity helps identify the curvature of a function, $f(x)$, in 2 dimensional space.

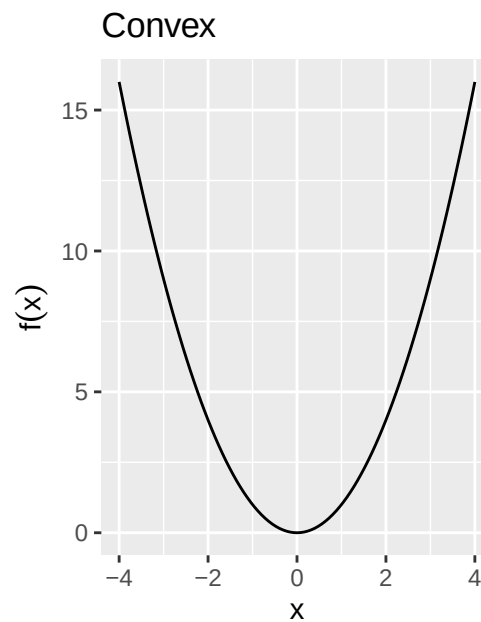
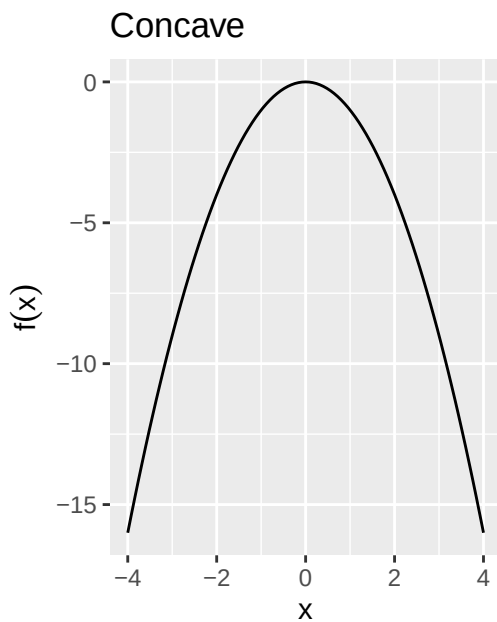
Definition 5.1.

Concave Function

A function f is strictly concave over the set S iff $\forall x_1, x_2 \in S$ and $\forall a \in (0, 1)$,

$$f(ax_1 + (1 - a)x_2) > af(x_1) + (1 - a)f(x_2)$$

Any line connecting two points on a concave function will lie *below* the function.



Definition 5.2.

Convex Function

Convex: A function f is strictly convex over the set S iff $\forall x_1, x_2 \in S$ and $\forall a \in (0, 1)$,

$$f(ax_1 + (1 - a)x_2) < af(x_1) + (1 - a)f(x_2)$$

Any line connecting two points on a convex function will lie above the function.

Sometimes, concavity and convexity are strict of a requirement. For most purposes of getting solutions, what we call quasi-concavity is enough.

Definition 5.3.

Quasiconcave Function

A function f is quasiconcave over the set S if $\forall x_1, x_2 \in S$ and $\forall a \in (0, 1)$,

$$f(ax_1 + (1 - a)x_2) \geq \min(f(x_1), f(x_2))$$

No matter what two points you select, the *lowest* valued point will always be an end point.

Definition 5.4.

Quasiconvex Function

A function f is quasiconvex over the set S if $\forall x_1, x_2 \in S$ and $\forall a \in (0, 1)$,

$$f(ax_1 + (1 - a)x_2) \leq \max(f(x_1), f(x_2))$$

No matter what two points you select, the *highest* valued point will always be an end point.

Second Derivative Test of Concavity: The second derivative can be used to understand concavity.

If

$$\begin{aligned} f''(x) < 0 &\Rightarrow \text{Concave} \\ f''(x) > 0 &\Rightarrow \text{Convex} \end{aligned}$$

Quadratic Forms

Quadratic forms is shorthand for a way to summarize a function. This is important for finding concavity because

1. Approximates local curvature around a point — e.g., used to identify max vs min vs saddle point.
2. They are simple to express even in n dimensions:
3. Have a matrix representation.

Quadratic Form: A polynomial where each term is a monomial of degree 2 in any number of variables:

$$\text{One variable: } Q(x_1) = a_{11}x_1^2$$

$$\text{Two variables: } Q(x_1, x_2) = a_{11}x_1^2 + a_{12}x_1x_2 + a_{22}x_2^2$$

$$\text{N variables: } Q(x_1, \dots, x_n) = \sum_{i \leq j} a_{ij}x_i x_j$$

which can be written in matrix terms:

One variable

$$Q(\mathbf{x}) = x_1^\top a_{11} x_1$$

N variables:

$$Q(\mathbf{x}) = \begin{pmatrix} x_1 & x_2 & \cdots & x_n \end{pmatrix} \begin{pmatrix} a_{11} & \frac{1}{2}a_{12} & \cdots & \frac{1}{2}a_{1n} \\ \frac{1}{2}a_{12} & a_{22} & \cdots & \frac{1}{2}a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{1}{2}a_{1n} & \frac{1}{2}a_{2n} & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \\ = \mathbf{x}^\top \mathbf{A} \mathbf{x}$$

For example, the Quadratic on $\mathbf{R}^2$:

$$Q(x_1, x_2) = \begin{pmatrix} x_1 & x_2 \end{pmatrix} \begin{pmatrix} a_{11} & \frac{1}{2}a_{12} \\ \frac{1}{2}a_{12} & a_{22} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \\ = a_{11}x_1^2 + a_{12}x_1x_2 + a_{22}x_2^2$$

Definiteness of Quadratic Forms

When the function $f(\mathbf{x})$ has more than two inputs, determining whether it has a maxima and minima (remember, functions may have many inputs but they have only one output) is a bit more tedious. Definiteness helps identify the curvature of a function, $Q(\mathbf{x})$, in n dimensional space.

Definiteness: By definition, a quadratic form always takes on the value of zero when $x = 0$, $Q(\mathbf{x}) = 0$ at $\mathbf{x} = 0$. The definiteness of the matrix $\mathbf{A}$ is determined by whether the quadratic form $Q(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x}$ is greater than zero, less than zero, or sometimes both over all $\mathbf{x} \neq 0$.

5.3 FOC and SOC

We can see from a graphical representation that if a point is a local maxima or minima, it must meet certain conditions regarding its derivative. These are so commonly used that we refer these to “First Order Conditions” (FOCs) and “Second Order Conditions” (SOCs) in the economic tradition.

First Order Conditions

When we examined functions of one variable x , we found critical points by taking the first derivative, setting it to zero, and solving for x . For functions of n variables, the critical points are found in much the same way, except now we set the partial derivatives equal to zero. Note: We will only consider critical points on the interior of a function's domain.

In a derivative, we only took the derivative with respect to one variable at a time. When we take the derivative separately with respect to all variables in the elements of $\mathbf{x}$ and then express the result as a vector, we use the term Gradient and Hessian.

Definition 5.5.

Gradient

Given a function $f(\mathbf{x})$ in n variables, the gradient $\nabla f(\mathbf{x})$ (the greek letter nabla) is a column vector, where the i th element is the partial derivative of $f(\mathbf{x})$ with respect to x_i :

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \frac{\partial f(\mathbf{x})}{\partial x_2} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_n} \end{pmatrix}$$

Before we know whether a point is a maxima or minima, if it meets the FOC it is a “Critical Point”.

Definition 5.6.

Critical Point

$\mathbf{x}^*$ is a critical point if and only if $\nabla f(\mathbf{x}^*) = 0$. If the partial derivative of $f(\mathbf{x})$ with respect to x^* is 0, then $\mathbf{x}^*$ is a critical point. To solve for $\mathbf{x}^*$, find the gradient, set each element equal to 0, and solve the system of equations.

$$\mathbf{x}^* = \begin{pmatrix} x_1^* \\ x_2^* \\ \vdots \\ x_n^* \end{pmatrix}$$

Example 5.2. Example: Given a function $f(\mathbf{x}) = (x_1 - 1)^2 + x_2^2 + 1$, find the (1) Gradient and (2) Critical point of $f(\mathbf{x})$.

Solution. Gradient

$$\begin{aligned} \nabla f(\mathbf{x}) &= \begin{pmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \frac{\partial f(\mathbf{x})}{\partial x_2} \end{pmatrix} \\ &= \begin{pmatrix} 2(x_1 - 1) \\ 2x_2 \end{pmatrix} \end{aligned}$$

Critical Point $\mathbf{x}^* =$

$$\begin{aligned} \frac{\partial f(\mathbf{x})}{\partial x_1} &= 2(x_1 - 1) = 0 \\ \Rightarrow x_1^* &= 1 \\ \frac{\partial f(\mathbf{x})}{\partial x_2} &= 2x_2 = 0 \\ \Rightarrow x_2^* &= 0 \end{aligned}$$

So

$$\mathbf{x}^* = (1, 0)$$

Second Order Conditions

When we found a critical point for a function of one variable, we used the second derivative as a indicator of the curvature at the point in order to determine whether the point was a min, max, or saddle (second derivative test of concavity). For functions of n variables, we use *second order partial derivatives* as an indicator of curvature.

Definition 5.7.

Hessian

Given a function $f(\mathbf{x})$ in n variables, the hessian $\mathbf{H}(\mathbf{x})$ is an $n \times n$ matrix, where the (i, j) th element is the second order partial derivative of $f(\mathbf{x})$ with respect to x_i and x_j :

$$\mathbf{H}(\mathbf{x}) = \begin{pmatrix} \frac{\partial^2 f(\mathbf{x})}{\partial x_1^2} & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_2 \partial x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial x_2^2} & \dots & \frac{\partial^2 f(\mathbf{x})}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_n \partial x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial x_n \partial x_2} & \dots & \frac{\partial^2 f(\mathbf{x})}{\partial x_n^2} \end{pmatrix}$$

Note that the hessian will be a symmetric matrix because $\frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_2} = \frac{\partial^2 f(\mathbf{x})}{\partial x_2 \partial x_1}$.

Also note that given that $f(\mathbf{x})$ is of quadratic form, each element of the hessian will be a constant.

These definitions will be employed when we determine the **Second Order Conditions** of a function:

Given a function $f(\mathbf{x})$ and a point $\mathbf{x}^*$ such that $\nabla f(\mathbf{x}^*) = 0$,

1. Hessian is Positive Definite $\implies$ Strict Local Min
2. Hessian is Positive Semidefinite $\forall \mathbf{x} \in B(\mathbf{x}^*, \epsilon)$ $\implies$ Local Min
3. Hessian is Negative Definite $\implies$ Strict Local Max
4. Hessian is Negative Semidefinite $\forall \mathbf{x} \in B(\mathbf{x}^*, \epsilon)$ $\implies$ Local Max
5. Hessian is Indefinite $\implies$ Saddle Point

Example 5.3.

Max and min with two dimensions

We found that the only critical point of $f(\mathbf{x}) = (x_1 - 1)^2 + x_2^2 + 1$ is at $\mathbf{x}^* = (1, 0)$. Is it a min, max, or saddle point?

Solution. The Hessian is

$$\mathbf{H}(\mathbf{x}) = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$$

The Leading principal minors of the Hessian are $M_1 = 2$; $M_2 = 4$. Now we consider Definiteness. Since both leading principal minors are positive, the Hessian is positive definite.

Maxima, Minima, or Saddle Point? Since the Hessian is positive definite and the gradient equals 0, $\mathbf{x}^* = (1, 0)$ is a strict local minimum.

Note: Alternate check of definiteness. Is $\mathbf{H}(\mathbf{x}^*) \geq 0 \quad \forall \quad \mathbf{x} \neq 0$

$$\begin{aligned} \mathbf{x}^\top \mathbf{H}(\mathbf{x}^*) \mathbf{x} &= \begin{pmatrix} x_1 & x_2 \end{pmatrix} \\ &= \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} \\ &\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = 2x_1^2 + 2x_2^2 \end{aligned}$$

For any $\mathbf{x} \neq 0$, $2(x_1^2 + x_2^2) > 0$, so the Hessian is positive definite and $\mathbf{x}^*$ is a strict local minimum.

Definiteness and Concavity

Although definiteness helps us to understand the curvature of an n-dimensional function, it does not necessarily tell us whether the function is globally concave or convex.

We need to know whether a function is globally concave or convex to determine whether a critical point is a global min or max. We can use the definiteness of the Hessian to determine whether a function is globally concave or convex:

1. Hessian is Positive Semidefinite $\forall \mathbf{x}$ $\implies$ Globally Convex
2. Hessian is Negative Semidefinite $\forall \mathbf{x}$ $\implies$ Globally Concave

Notice that the definiteness conditions must be satisfied over the entire domain.

5.4 Global Maxima and Minima

Global Max/Min Conditions: Given a function $f(\mathbf{x})$ and a point $\mathbf{x}^*$ such that $\nabla f(\mathbf{x}^*) = 0$,

1. $f(\mathbf{x})$ Globally Convex $\implies$ Global Min
2. $f(\mathbf{x})$ Globally Concave $\implies$ Global Max

Note that showing that $\mathbf{H}(\mathbf{x}^*)$ is negative semidefinite is not enough to guarantee $\mathbf{x}^*$ is a local max. However, showing that $\mathbf{H}(\mathbf{x})$ is negative semidefinite for all $\mathbf{x}$ guarantees that x^* is a global max. (The same goes for positive semidefinite and minima.)\

Example: Take $f_1(x) = x^4$ and $f_2(x) = -x^4$. Both have $x = 0$ as a critical point. Unfortunately, $f_1''(0) = 0$ and $f_2''(0) = 0$, so we can't tell whether $x = 0$ is a min or max for either. However, $f_1''(x) = 12x^2$ and $f_2''(x) = -12x^2$. For all x , $f_1''(x) \geq 0$ and $f_2''(x) \leq 0$ — i.e., $f_1(x)$ is globally convex and $f_2(x)$ is globally concave. So $x = 0$ is a global min of $f_1(x)$ and a global max of $f_2(x)$.

Exercise 5.2.

Optimization

Given $f(\mathbf{x}) = x_1^3 - x_2^3 + 9x_1x_2$, find any maxima or minima.

1. First order conditions.

a) Gradient $\nabla f(\mathbf{x}) =$

b) Critical Points $\mathbf{x}^* =$

2. Second order conditions.

a) Hessian $\mathbf{H}(\mathbf{x}) =$

b) Hessian $\mathbf{H}(\mathbf{x}_1^*) =$

c) Leading principal minors of $\mathbf{H}(\mathbf{x}_1^*) =$

- d) Definiteness of $\mathbf{H}(\mathbf{x}_1^*)$?
 - e) Maxima, Minima, or Saddle Point for $\mathbf{x}_1^*$?
 - f) Hessian $\mathbf{H}(\mathbf{x}_2^*) =$
 - g) Leading principal minors of $\mathbf{H}(\mathbf{x}_2^*) =$
 - h) Definiteness of $\mathbf{H}(\mathbf{x}_2^*)$?
 - i) Maxima, Minima, or Saddle Point for $\mathbf{x}_2^*$?
3. Global concavity/convexity.
- a) Is $f(\mathbf{x})$ globally concave/convex?
 - b) Are any $\mathbf{x}^*$ global minima or maxima?

5.5 Constrained Optimization

We have already looked at optimizing a function in one or more dimensions over the whole domain of the function. Often, however, we want to find the maximum or minimum of a function over some restricted part of its domain.

ex: Maximizing utility subject to a budget constraint

Types of Constraints: For a function $f(x_1, \dots, x_n)$, there are two types of constraints that can be imposed:

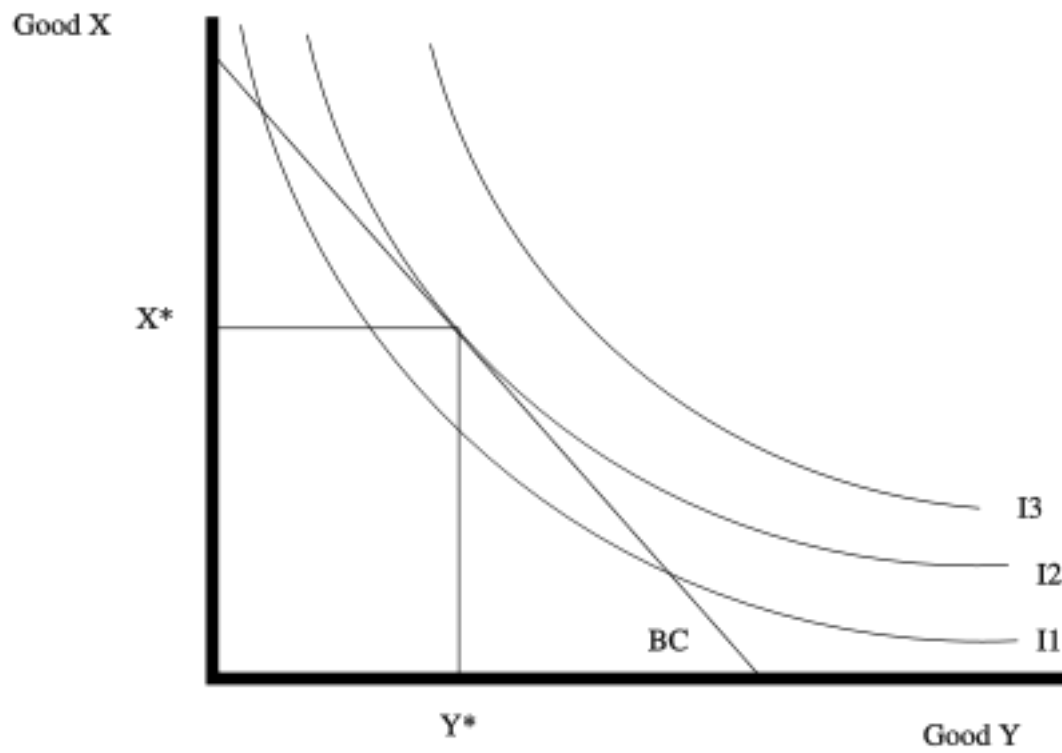


Figure 5.2: A typical Utility Function with a Budget Constraint

1. **Equality constraints:** constraints of the form $c(x_1, \dots, x_n) = r$. Budget constraints are the classic example of equality constraints in social science.
2. **Inequality constraints:** constraints of the form $c(x_1, \dots, x_n) \leq r$. These might arise from non-negativity constraints or other threshold effects.

In any constrained optimization problem, the constrained maximum will always be less than or equal to the unconstrained maximum. If the constrained maximum is less than the unconstrained maximum, then the constraint is binding. Essentially, this means that you can treat your constraint as an equality constraint rather than an inequality constraint.

For example, the budget constraint binds when you spend your entire budget. This generally happens because we believe that utility is strictly increasing in consumption, i.e. you always want more so you spend everything you have.

Any number of constraints can be placed on an optimization problem. When working with multiple constraints, always make sure that the set of constraints are not pathological; it must be possible for all of the constraints to be satisfied simultaneously.

Set-up for Constrained Optimization:

$$\max_{x_1, x_2} f(x_1, x_2) \text{ s.t. } c(x_1, x_2)$$

$$\min_{x_1, x_2} f(x_1, x_2) \text{ s.t. } c(x_1, x_2)$$

This tells us to maximize/minimize our function, $f(x_1, x_2)$, with respect to the choice variables, x_1, x_2 , subject to the constraint.

Example:

$$\max_{x_1, x_2} f(x_1, x_2) = -(x_1^2 + 2x_2^2) \text{ s.t. } x_1 + x_2 = 4$$

It is easy to see that the *unconstrained* maximum occurs at $(x_1, x_2) = (0, 0)$, but that does not satisfy the constraint. How should we proceed?

Equality Constraints

Equality constraints are the easiest to deal with because we know that the maximum or minimum has to lie on the (intersection of the) constraint(s).

The trick is to change the problem from a constrained optimization problem in n variables to an unconstrained optimization problem in $n + k$ variables, adding *one* variable for *each* equality constraint. We do this using a lagrangian multiplier.

Lagrangian function: The Lagrangian function allows us to combine the function we want to optimize and the constraint function into a single function. Once we have this single function, we can proceed as if this were an *unconstrained* optimization problem.

For each constraint, we must include a **Lagrange multiplier** (λ_i) as an additional variable in the analysis. These terms are the link between the constraint and the Lagrangian function.

Given a *two dimensional* set-up:

$$\max_{x_1, x_2} / \min_{x_1, x_2} f(x_1, x_2) \text{ s.t. } c(x_1, x_2) = a$$

We define the Lagrangian function $L(x_1, x_2, \lambda_1)$ as follows:

$$L(x_1, x_2, \lambda_1) = f(x_1, x_2) - \lambda_1(c(x_1, x_2) - a)$$

More generally, in *n dimensions*:

$$L(x_1, \dots, x_n, \lambda_1, \dots, \lambda_k) = f(x_1, \dots, x_n) - \sum_{i=1}^k \lambda_i(c_i(x_1, \dots, x_n) - r_i)$$

Getting the sign right: Note that above we subtract the lagrangian term *and* we subtract the constraint constant from the constraint function. Occasionally, you may see the following alternative form of the Lagrangian, which is *equivalent*:

$$L(x_1, \dots, x_n, \lambda_1, \dots, \lambda_k) = f(x_1, \dots, x_n) + \sum_{i=1}^k \lambda_i(r_i - c_i(x_1, \dots, x_n))$$

Here we add the lagrangian term *and* we subtract the constraining function from the constraint constant.

Using the Lagrangian to Find the Critical Points: To find the critical points, we take the partial derivatives of lagrangian function, $L(x_1, \dots, x_n, \lambda_1, \dots, \lambda_k)$, with respect to each of its variables (all choice variables $\mathbf{x}$ *and* all lagrangian multipliers). At a critical point, each of these partial derivatives must be equal to zero, so we obtain a system of $n + k$ equations in $n + k$ unknowns:

$$\begin{aligned} \frac{\partial L}{\partial x_1} &= \frac{\partial f}{\partial x_1} - \sum_{i=1}^k \lambda_i \frac{\partial c_i}{\partial x_1} = 0 \\ &\vdots \\ \frac{\partial L}{\partial x_n} &= \frac{\partial f}{\partial x_n} - \sum_{i=1}^k \lambda_i \frac{\partial c_i}{\partial x_n} = 0 \\ \frac{\partial L}{\partial \lambda_1} &= c_1(x_1, \dots, x_n) - r_1 = 0 \\ &\vdots \\ \frac{\partial L}{\partial \lambda_k} &= c_k(x_1, \dots, x_n) - r_k = 0 \end{aligned}$$

We can then solve this system of equations, because there are $n + k$ equations and $n + k$ unknowns, to calculate the critical point $(x_1^*, \dots, x_n^*, \lambda_1^*, \dots, \lambda_k^*)$.

Second-order Conditions and Unconstrained Optimization: There may be more than one critical point, i.e. we need to verify that the critical point we find is a maximum/minimum. Similar to unconstrained optimization, we can do this by checking the second-order conditions.

Example 5.4.

Constrained optimization with two goods and a budget constraint

Find the constrained optimization of

$$\max_{x_1, x_2} f(x) = -(x_1^2 + 2x_2^2) \text{ s.t. } x_1 + x_2 = 4$$

Solution. 1. Begin by writing the Lagrangian:

$$L(x_1, x_2, \lambda) = -(x_1^2 + 2x_2^2) - \lambda(x_1 + x_2 - 4)$$

2. Take the partial derivatives and set equal to zero:

$$\begin{aligned} \frac{\partial L}{\partial x_1} &= -2x_1 - \lambda &= 0 \\ \frac{\partial L}{\partial x_2} &= -4x_2 - \lambda &= 0 \\ \frac{\partial L}{\partial \lambda} &= -(x_1 + x_2 - 4) &= 0 \end{aligned}$$

3. Solve the system of equations: Using the first two partials, we see that $\lambda = -2x_1$ and $\lambda = -4x_2$. Set these equal to see that $x_1 = 2x_2$. Using the third partial and the above equality, $4 = 2x_2 + x_2$ from which we get

$$x_2^* = 4/3, x_1^* = 8/3, \lambda = -16/3$$

4. Therefore, the only critical point is $x_1^* = \frac{8}{3}$ and $x_2^* = \frac{4}{3}$

5. This gives $f(\frac{8}{3}, \frac{4}{3}) = -\frac{96}{9}$, which is less than the unconstrained optimum $f(0, 0) = 0$

Notice that when we take the partial derivative of L with respect to the Lagrangian multiplier and set it equal to 0, we return exactly our constraint! This is why signs matter.

5.6 Inequality Constraints

Inequality constraints define the boundary of a region over which we seek to optimize the function. This makes inequality constraints more challenging because we do not know if the maximum/minimum lies along one of the constraints (the constraint binds) or in the interior of the region.

We must introduce more variables in order to turn the problem into an unconstrained optimization.

Slack: For each inequality constraint $c_i(x_1, \dots, x_n) \leq a_i$, we define a slack variable s_i^2 for which the expression $c_i(x_1, \dots, x_n) \leq a_i - s_i^2$ would hold with equality. These slack variables capture how close the constraint comes to binding. We use s^2 rather than s to ensure that the slack is positive.

Slack is just a way to transform our constraints.

Given a two-dimensional set-up and these edited constraints:

$$\max_{x_1, x_2} / \min_{x_1, x_2} f(x_1, x_2) \text{ s.t. } c(x_1, x_2) \leq a_1$$

Adding in Slack:

$$\max_{x_1, x_2} / \min_{x_1, x_2} f(x_1, x_2) \text{ s.t. } c(x_1, x_2) \leq a_1 - s_1^2$$

We define the Lagrangian function $L(x_1, x_2, \lambda_1, s_1)$ as follows:

$$L(x_1, x_2, \lambda_1, s_1) = f(x_1, x_2) - \lambda_1(c(x_1, x_2) + s_1^2 - a_1)$$

More generally, in n dimensions:

$$L(x_1, \dots, x_n, \lambda_1, \dots, \lambda_k, s_1, \dots, s_k) = f(x_1, \dots, x_n) - \sum_{i=1}^k \lambda_i(c_i(x_1, \dots, x_n) + s_i^2 - a_i)$$

Finding the Critical Points: To find the critical points, we take the partial derivatives of the lagrangian function, $L(x_1, \dots, x_n, \lambda_1, \dots, \lambda_k, s_1, \dots, s_k)$, with respect to each of its variables (all choice variables x , all lagrangian multipliers λ , and all slack variables s). At a critical point, *each* of these partial derivatives must be equal to zero, so we obtain a system of $n + 2k$ equations in $n + 2k$ unknowns:

$$\begin{aligned}
\frac{\partial L}{\partial x_1} &= \frac{\partial f}{\partial x_1} - \sum_{i=1}^k \lambda_i \frac{\partial c_i}{\partial x_1} = 0 \\
&\vdots \\
\frac{\partial L}{\partial x_n} &= \frac{\partial f}{\partial x_n} - \sum_{i=1}^k \lambda_i \frac{\partial c_i}{\partial x_n} = 0 \\
\frac{\partial L}{\partial \lambda_1} &= c_1(x_1, \dots, x_n) + s_1^2 - b_1 = 0 \\
&\vdots \\
\frac{\partial L}{\partial \lambda_k} &= c_k(x_1, \dots, x_n) + s_k^2 - b_k = 0 \\
\frac{\partial L}{\partial s_1} &= 2s_1\lambda_1 = 0 \\
&\vdots \\
\frac{\partial L}{\partial s_k} &= 2s_k\lambda_k = 0
\end{aligned}$$

Complementary slackness conditions: The last set of first order conditions of the form $2s_i\lambda_i = 0$ (the partials taken with respect to the slack variables) are known as complementary slackness conditions. These conditions can be satisfied one of three ways:

1. $\lambda_i = 0$ and $s_i \neq 0$: This implies that the slack is positive and thus *the constraint does not bind*.
2. $\lambda_i \neq 0$ and $s_i = 0$: This implies that there is no slack in the constraint and *the constraint does bind*.
3. $\lambda_i = 0$ and $s_i = 0$: In this case, there is no slack but the *constraint binds trivially*, without changing the optimum.

Example: Find the critical points for the following constrained optimization:

$$\max_{x_1, x_2} f(x) = -(x_1^2 + 2x_2^2) \text{ s.t. } x_1 + x_2 \leq 4$$

1. Rewrite with the slack variables:

$$\max_{x_1, x_2} f(x) = -(x_1^2 + 2x_2^2) \text{ s.t. } x_1 + x_2 \leq 4 - s_1^2$$

2. Write the Lagrangian:

$$L(x_1, x_2, \lambda_1, s_1) = -(x_1^2 + 2x_2^2) - \lambda_1(x_1 + x_2 + s_1^2 - 4)$$

3. Take the partial derivatives and set equal to 0:

$$\begin{aligned}\frac{\partial L}{\partial x_1} &= -2x_1 - \lambda_1 = 0 \\ \frac{\partial L}{\partial x_2} &= -4x_2 - \lambda_1 = 0 \\ \frac{\partial L}{\partial \lambda_1} &= -(x_1 + x_2 + s_1^2 - 4) = 0 \\ \frac{\partial L}{\partial s_1} &= -2s_1\lambda_1 = 0\end{aligned}$$

4. Consider all ways that the complementary slackness conditions are solved:

Hypothesis	s_1	λ_1	x_1	x_2	$f(x_1, x_2)$
$s_1 = 0 \ \lambda_1 = 0$	No solution				
$s_1 \neq 0 \ \lambda_1 = 0$	2	0	0	0	0
$s_1 = 0 \ \lambda_1 \neq 0$	0	$-\frac{16}{3}$	$\frac{8}{3}$	$\frac{4}{3}$	$-\frac{32}{3}$
$s_1 \neq 0 \ \lambda_1 \neq 0$	No solution				

This shows that there are two critical points: $(0, 0)$ and $(\frac{8}{3}, \frac{4}{3})$.

5. Find maximum: Looking at the values of $f(x_1, x_2)$ at the critical points, we see that $f(x_1, x_2)$ is maximized at $x_1^* = 0$ and $x_2^* = 0$.

Exercise 5.3.

Constrained optimization

Example: Find the critical points for the following constrained optimization:

$$\max_{x_1, x_2} f(x) = -(x_1^2 + 2x_2^2) \text{ s.t. } \begin{array}{l} x_1 + x_2 \leq 4 \\ x_1 \geq 0 \\ x_2 \geq 0 \end{array}$$

1. Rewrite with the slack variables:

2. Write the Lagrangian:

3. Take the partial derivatives and set equal to zero:

4. Consider all ways that the complementary slackness conditions are solved:

Hypothesis	s_1	s_2	s_3	λ_1	λ_2	λ_3	x_1	x_2	$f(x_1, x_2)$
$s_1 = s_2 = s_3 = 0$									
$s_1 \neq 0, s_2 = s_3 = 0$									
$s_2 \neq 0, s_1 = s_3 = 0$									
$s_3 \neq 0, s_1 = s_2 = 0$									
$s_1 \neq 0, s_2 \neq 0, s_3 = 0$									
$s_1 \neq 0, s_3 \neq 0, s_2 = 0$									
$s_2 \neq 0, s_3 \neq 0, s_1 = 0$									
$s_1 \neq 0, s_2 \neq 0, s_3 \neq 0$									

5. Find maximum:

5.7 Kuhn-Tucker Conditions

As you can see, this can be a pain. When dealing explicitly with *non-negativity constraints*, this process is simplified by using the Kuhn-Tucker method.

Because the problem of maximizing a function subject to inequality and non-negativity constraints arises frequently in economics, the **Kuhn-Tucker conditions** provides a method that often makes it easier to both calculate the critical points and identify points that are (local) maxima.

Given a *two-dimensional set-up*:

$$\begin{array}{ll} \max_{x_1, x_2} / \min_{x_1, x_2} f(x_1, x_2) \text{ s.t.} & \begin{array}{l} c(x_1, x_2) \leq a_1 \\ x_1 \geq 0 \\ gx_2 \geq 0 \end{array} \end{array}$$

We define the Lagrangian function $L(x_1, x_2, \lambda_1)$ the same as if we did not have the non-negativity constraints:

$$L(x_1, x_2, \lambda_1) = f(x_1, x_2) - \lambda_1(c(x_1, x_2) - a_1)$$

More generally, in n dimensions:

$$L(x_1, \dots, x_n, \lambda_1, \dots, \lambda_k) = f(x_1, \dots, x_n) - \sum_{i=1}^k \lambda_i(c_i(x_1, \dots, x_n) - a_i)$$

Kuhn-Tucker and Complementary Slackness Conditions: To find the critical points, we first calculate the Kuhn-Tucker conditions by taking the partial derivatives of the lagrangian function, $L(x_1, \dots, x_n, \lambda_1, \dots, \lambda_k)$, with respect to each of its variables (all choice variables x and all lagrangian multipliers λ) and we calculate the *complementary slackness conditions* by multiplying each partial derivative by its respective variable *and* include non-negativity conditions for all variables (choice variables x and lagrangian multipliers λ).

Kuhn-Tucker Conditions

$$\begin{array}{l} \frac{\partial L}{\partial x_1} \leq 0, \dots, \frac{\partial L}{\partial x_n} \leq 0 \\ \frac{\partial L}{\partial \lambda_1} \geq 0, \dots, \frac{\partial L}{\partial \lambda_m} \geq 0 \end{array}$$

Complementary Slackness Conditions

$$x_1 \frac{\partial L}{\partial x_1} = 0, \dots, x_n \frac{\partial L}{\partial x_n} = 0$$

$$\lambda_1 \frac{\partial L}{\partial \lambda_1} = 0, \dots, \lambda_m \frac{\partial L}{\partial \lambda_m} = 0$$

Non-negativity Conditions

$$x_1 \geq 0 \quad \dots \quad x_n \geq 0$$

$$\lambda_1 \geq 0 \quad \dots \quad \lambda_m \geq 0$$

Note that some of these conditions are set equal to 0, while others are set as inequalities!

Note also that to minimize the function $f(x_1, \dots, x_n)$, the simplest thing to do is maximize the function $-f(x_1, \dots, x_n)$; all of the conditions remain the same after reformulating as a maximization problem.

There are additional assumptions (notably, $f(x)$ is quasi-concave and the constraints are convex) that are sufficient to ensure that a point satisfying the Kuhn-Tucker conditions is a global max; if these assumptions do not hold, you may have to check more than one point.

Finding the Critical Points with Kuhn-Tucker Conditions: Given the above conditions, to find the critical points we solve the above system of equations. To do so, we must check *all* border and interior solutions to see if they satisfy the above conditions.

In a two-dimensional set-up, this means we must check the following cases:

1. $x_1 = 0, x_2 = 0$ Border Solution
2. $x_1 = 0, x_2 \neq 0$ Border Solution
3. $x_1 \neq 0, x_2 = 0$ Border Solution
4. $x_1 \neq 0, x_2 \neq 0$ Interior Solution

Example 5.5.

Kuhn-Tucker with two variables

Solve the following optimization problem with inequality constraints

$$\max_{x_1, x_2} f(x) = -(x_1^2 + 2x_2^2)$$

$$\text{s.t.} \quad \begin{cases} x_1 + x_2 \leq 4 \\ x_1 \geq 0 \\ x_2 \geq 0 \end{cases}$$

1. Write the Lagrangian:

$$L(x_1, x_2, \lambda) = -(x_1^2 + 2x_2^2) - \lambda(x_1 + x_2 - 4)$$

2. Find the First Order Conditions:

Kuhn-Tucker Conditions

$$\frac{\partial L}{\partial x_1} = -2x_1 - \lambda \leq 0$$

$$\frac{\partial L}{\partial x_2} = -4x_2 - \lambda \leq 0$$

$$\frac{\partial L}{\partial \lambda} = -(x_1 + x_2 - 4) \geq 0$$

Complementary Slackness Conditions

$$x_1 \frac{\partial L}{\partial x_1} = x_1(-2x_1 - \lambda) = 0$$

$$x_2 \frac{\partial L}{\partial x_2} = x_2(-4x_2 - \lambda) = 0$$

$$\lambda \frac{\partial L}{\partial \lambda} = -\lambda(x_1 + x_2 - 4) = 0$$

Non-negativity Conditions

$$x_1 \geq 0$$

$$x_2 \geq 0$$

$$\lambda \geq 0$$

3. Consider all border and interior cases:

Hypothesis	λ	x_1	x_2	$f(x_1, x_2)$
$x_1 = 0, x_2 = 0$	0	0	0	0
$x_1 = 0, x_2 \neq 0$	-16	0	4	-32
$x_1 \neq 0, x_2 = 0$	-8	4	0	-16
$x_1 \neq 0, x_2 \neq 0$	$-\frac{16}{3}$	$\frac{8}{3}$	$\frac{4}{3}$	$-\frac{32}{3}$

4. Find Maximum: Three of the critical points violate the requirement that $\lambda \geq 0$, so the point $(0, 0, 0)$ is the maximum.

Exercise 5.4.

Kuhn-Tucker with logs

$$\max_{x_1, x_2} f(x) = \frac{1}{3} \log(x_1 + 1) + \frac{2}{3} \log(x_2 + 1) \text{ s.t. } \begin{array}{l} x_1 + 2x_2 \leq 4 \\ x_1 \geq 0 \\ x_2 \geq 0 \end{array}$$

1. Write the Lagrangian:
2. Find the First Order Conditions:
Kuhn-Tucker Conditions

Complementary Slackness Conditions

Non-negativity Conditions

3. Consider all border and interior cases:

Hypothesis	λ	x_1	x_2	$f(x_1, x_2)$
$x_1 = 0, x_2 = 0$				
$x_1 = 0, x_2 \neq 0$				
$x_1 \neq 0, x_2 = 0$				
$x_1 \neq 0, x_2 \neq 0$				

4. Find Maximum:

5.8 Applications of Quadratic Forms

Curvature and The Taylor Polynomial as a Quadratic Form: The Hessian is used in a Taylor polynomial approximation to $f(\mathbf{x})$ and provides information about the curvature of $f(\mathbf{x})$ at $\mathbf{x}$ — e.g., which tells us whether a critical point $\mathbf{x}^*$ is a min, max, or saddle point.

1. The second order Taylor polynomial about the critical point $\mathbf{x}^*$ is

$$f(\mathbf{x}^* + \mathbf{h}) = f(\mathbf{x}^*) + \nabla f(\mathbf{x}^*)\mathbf{h} + \frac{1}{2}\mathbf{h}^\top \mathbf{H}(\mathbf{x}^*)\mathbf{h} + R(\mathbf{h})$$

2. Since we're looking at a critical point, $\nabla f(\mathbf{x}^*) = 0$; and for small $\mathbf{h}$, $R(\mathbf{h})$ is negligible. Rearranging, we get

$$f(\mathbf{x}^* + \mathbf{h}) - f(\mathbf{x}^*) \approx \frac{1}{2}\mathbf{h}^\top \mathbf{H}(\mathbf{x}^*)\mathbf{h}$$

3. The Righthand side here is a quadratic form and we can determine the definiteness of $\mathbf{H}(x^*)$.

6 Probability Theory

Probability and Inferences are mirror images of each other, and both are integral to social science. Probability quantifies uncertainty, which is important because many things in the social world are at first uncertain. Inference is then the study of how to learn about facts you don't observe from facts you do observe.

6.1 Counting rules

Probability in high school is usually really about combinatorics: the probability of event A is the number of ways in which A can occur divided by the number of all other possibilities. This is a very simplified version of probability, which we can call the “counting definition of probability”, essentially because each possible event to count is often equally likely and discrete. But it is still good to review the underlying rules here.

Fundamental Theorem of Counting: If an object has j different characteristics that are independent of each other, and each characteristic i has n_i ways of being expressed, then there are $\prod_{i=1}^j n_i$ possible unique objects.

Example 6.1.

Counting Possibilities

Suppose we are given a stack of cards. Cards can be either red or black and can take on any of 13 values. There is only one of each color-number combination. In this case,

1. $j =$
2. $n_{\text{color}} =$
3. $n_{\text{number}} =$
4. Number of Outcomes =

We often need to count the number of ways to choose a subset from some set of possibilities. The number of outcomes depends on two characteristics of the process: does the order matter and is replacement allowed?

It is useful to think of any problem concretely, e.g. through a **sampling table**: If there are n objects which are numbered 1 to n and we select $k < n$ of them, how many different outcomes are possible?

If the order in which a given object is selected matters, selecting 4 numbered objects in the following order (1, 3, 7, 2) and selecting the same four objects but in a different order such as (7, 2, 1, 3) will be counted as different outcomes.

If replacement is allowed, there are always the same n objects to select from. However, if replacement is not allowed, there is always one less option than the previous round when making a selection. For example, if replacement is not allowed and I am selecting 3 elements from the following set $\{1, 2, 3, 4, 5, 6\}$, I will have 6 options at first, 5 options as I make my second selection, and 4 options as I make my third.

1. So if **order matters** AND we are sampling **with replacement**, the number of different outcomes is n^k .
2. If **order matters** AND we are sampling **without replacement**, the number of different outcomes is $n(n-1)(n-2)\dots(n-k+1) = \frac{n!}{(n-k)!}$.
3. If **order doesn't matter** AND we are sampling **without replacement**, the number of different outcomes is $\binom{n}{k} = \frac{n!}{(n-k)!k!}$.

Expression $\binom{n}{k}$ is read as “n choose k” and denotes $\frac{n!}{(n-k)!k!}$. Also, note that $0! = 1$.

Example 6.2.

Counting

There are five balls numbered from 1 through 5 in a jar. Three balls are chosen. How many possible choices are there?

1. Ordered, with replacement =
2. Ordered, without replacement =
3. Unordered, without replacement =

Exercise 6.1.

Counting

Four cards are selected from a deck of 52 cards. Once a card has been drawn, it is not reshuffled back into the deck. Moreover, we care only about the complete hand that we get (i.e. we care about the set of selected cards, not the sequence in which it was drawn). How many possible outcomes are there?

6.2 Probability

Probability Definitions: Formal and Informal

Many things in the world are uncertain. In everyday speech, we say that we are *uncertain* about the outcome of random events. Probability is a formal model of uncertainty which provides a measure of uncertainty governed by a particular set of rules. A different model of uncertainty would, of course, have a set of rules different from anything we discuss here. Our focus on probability is justified because it has proven to be a particularly useful model of uncertainty.

Sample Space (S): A set or collection of all possible outcomes from some process. Outcomes in the set can be discrete elements (countable) or points along a continuous interval (uncountable).

Probability Distribution Function: a mapping of each event in the sample space S to the real numbers that satisfy the following three axioms (also called Kolmogorov's Axioms).

Formally,

Definition 6.1.

Probability

Probability is a function that maps events from a sample space to a real number, obeying the axioms of probability.

The axioms of probability make sure that the separate events add up in terms of probability, and – for standardization purposes – that they add up to 1.

Definition 6.2.

Axioms of Probability

1. For any event A , $P(A) \geq 0$.
2. $P(S) = 1$
3. The Countable Additivity Axiom: For any sequence of *disjoint* (mutually exclusive) events $A_1, A_2, \dots$ (of which there may be infinitely many),

$$P\left(\bigcup_{i=1}^k A_i\right) = \sum_{i=1}^k P(A_i)$$

The last axiom is an extension of a union to infinite sets. When there are only two events in the space, it boils down to:

$$P(A_1 \cup A_2) = P(A_1) + P(A_2) \quad \text{for disjoint } A_1, A_2$$

Probability Operations

Using these three axioms, we can define all of the common rules of probability.

1. $P(\emptyset) = 0$
2. For any event A , $0 \leq P(A) \leq 1$.
3. $P(A^C) = 1 - P(A)$
4. If $A \subset B$ (A is a subset of B), then $P(A) \leq P(B)$.
5. For *any* two events A and B , $P(A \cup B) = P(A) + P(B) - P(A \cap B)$
6. Boole's Inequality: For any sequence of n events (which need not be disjoint) $A_1, A_2, \dots, A_n$, then $P\left(\bigcup_{i=1}^n A_i\right) \leq \sum_{i=1}^n P(A_i)$.

Example 6.3.

Probability

Assume we have an evenly-balanced, six-sided die.

Then,

1. Sample space $S =$
2. $P(1) = \dots = P(6) =$
3. $P(\emptyset) = P(7) =$
4. $P(\{1, 3, 5\}) =$
5. $P(\{1, 2\}^C) = P(\{3, 4, 5, 6\}) =$
6. Let $A = \{1, 2, 3, 4, 5\} \subset S$. Then $P(A) = 5/6 < P(S) =$
7. Let $A = \{1, 2, 3\}$ and $B = \{2, 4, 6\}$. Then $A \cup B$? $A \cap B$? $P(A \cup B)$?

Exercise 6.2.

Probability

Suppose you had a pair of four-sided dice. You sum the results from a single toss. Let us call this sum, or the outcome, X .

1. What is $P(X = 5)$, $P(X = 3)$, $P(X = 6)$?
2. What is $P(X = 5 \cup X = 3)^C$?

6.3 Conditional Probability and Bayes Rule

Conditional Probability: The conditional probability $P(A|B)$ of an event A is the probability of A , given that another event B has occurred. Conditional probability allows for the inclusion of other information into the calculation of the probability of an event. It is calculated as

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Note that conditional probabilities are probabilities and must also follow the Kolmogorov axioms of probability.

Example 6.4.

Conditional Probability 1

Assume A and B occur with the following frequencies:

	A	A^c
B	n_{ab}	$n_{a^c b}$
B^c	n_{ab^c}	$n_{(ab)^c}$

and let $n_{ab} + n_{a^c b} + n_{ab^c} + n_{(ab)^c} = N$. Then

1. $P(A) =$
2. $P(B) =$
3. $P(A \cap B) =$
4. $P(A|B) = \frac{P(A \cap B)}{P(B)} =$
5. $P(B|A) = \frac{P(A \cap B)}{P(A)} =$

Example 6.5.

Conditional Probability 2

A six-sided die is rolled. What is the probability of a 1, given the outcome is an odd number?

You could rearrange the fraction to highlight how a joint probability could be expressed as the product of a conditional probability.

Definition 6.3.

Multiplicative Law of Probability

The probability of the intersection of two events A and B is $P(A \cap B) = P(A)P(B|A) = P(B)P(A|B)$ which follows directly from the definition of conditional probability. More generally,

$$P(A_1 \cap \dots \cap A_k) = P(A_k | A_{k-1} \cap \dots \cap A_1) \times P(A_{k-1} | A_{k-2} \cap \dots \cap A_1) \times \dots \times P(A_2 | A_1) \times P(A_1)$$

Sometimes it is easier to calculate these conditional probabilities and sum them than it is to calculate $P(A)$ directly.

Definition 6.4.

Law of total probability

Let S be the sample space of some experiment and let the disjoint k events $B_1, \dots, B_k$ partition S , such that $P(B_1 \cup \dots \cup B_k) = P(S) = 1$. If A is some other event in S , then the events $A \cap B_1, A \cap B_2, \dots, A \cap B_k$ will form a partition of A and we can write A as

$$A = (A \cap B_1) \cup \dots \cup (A \cap B_k)$$

.

Since the k events are disjoint,

$$\begin{aligned} P(A) &= \sum_{i=1}^k P(A \cap B_i) \\ &= \sum_{i=1}^k P(B_i)P(A|B_i) \end{aligned}$$

Bayes Rule: Assume that events $B_1, \dots, B_k$ form a partition of the space S . Then by the Law of Total Probability

$$P(B_j|A) = \frac{P(A \cap B_j)}{P(A)} = \frac{P(B_j)P(A|B_j)}{\sum_{i=1}^k P(B_i)P(A|B_i)}$$

If there are only two states of B , then this is just

$$P(B_1|A) = \frac{P(B_1)P(A|B_1)}{P(B_1)P(A|B_1) + P(B_2)P(A|B_2)}$$

Bayes' rule determines the posterior probability of a state $P(B_j|A)$ by calculating the probability $P(A \cap B_j)$ that both the event A and the state B_j will occur and dividing it by the probability that the event will occur regardless of the state (by summing across all B_i). The states could be something like Normal/Defective, Healthy/Diseased, Republican/Democrat/Independent, etc. The event on which one conditions could be something like a sampling from a batch of components, a test for a disease, or a question about a policy position.

Prior and Posterior Probabilities: Above, $P(B_1)$ is often called the prior probability, since it's the probability of B_1 before anything else is known. $P(B_1|A)$ is called the posterior probability, since it's the probability after other information is taken into account.

Example 6.6.

Bayes' Rule

In a given town, 40% of the voters are Democrat and 60% are Republican. The president's budget is supported by 50% of the Democrats and 90% of the Republicans. If a randomly (equally likely) selected voter is found to support the president's budget, what is the probability that they are a Democrat?

Exercise 6.3.

Conditional Probability

Assume that 2% of the population of the U.S. are members of some extremist militia group. We develop a survey that positively classifies someone as being a member of a militia group given that they are a member 95% of the time and negatively classifies someone as not being a member of a militia group given that they are not a member 97% of the time. What is the probability that someone positively classified as being a member of a militia group is actually a militia member?

6.4 Independence

Definition 6.5.

Independence

If the occurrence or nonoccurrence of either events A and B provides no information about the occurrence or nonoccurrence of the other, then A and B are independent.

If A and B are independent, then

1. $P(A|B) = P(A)$
2. $P(B|A) = P(B)$
3. $P(A \cap B) = P(A)P(B)$
4. More generally than the above, $P(\bigcap_{i=1}^k A_i) = \prod_{i=1}^k P(A_i)$

Are mutually exclusive events independent of each other?

No. If A and B are mutually exclusive, then they cannot happen simultaneously. If we know that A occurred, then we know that B couldn't have occurred. Because of this, A and B aren't independent.

Pairwise Independence: A set of more than two events $A_1, A_2, \dots, A_k$ is pairwise independent if $P(A_i \cap A_j) = P(A_i)P(A_j)$, $\forall i \neq j$. Note that this does **not** necessarily imply joint independence.

Conditional Independence: If A and B are independent once you know the occurrence of a third event C , then A and B are conditionally independent (conditional on C):

1. $P(A|B \cap C) = P(A|C)$
2. $P(B|A \cap C) = P(B|C)$
3. $P(A \cap B|C) = P(A|C)P(B|C)$

Just because two events are conditionally independent does not mean that they are independent. Actually it is hard to think of real-world things that are “unconditionally” independent. That's why it's always important to ask about a finding: What was it conditioned on? For example, suppose that a graduate school admission decisions are done by only one professor, who picks a group of 50 bright students and flips a coin for each student to generate a class of about 25 students. Then the the probability that two students get accepted are conditionally independent, because they are determined by two separate coin tosses. However, this does not mean that their admittance is not completely independent. Knowing that student A got in gives us information about whether student B got in, if we think that the professor originally picked her pool of 50 students by merit.

Perhaps more counter-intuitively: If two events are already independent, then it might seem that no amount of “conditioning” will make them dependent. But this is not always so. For example, imagine that you own a house with a lawn (a very extreme hypothetical!) Let A be the event that it rained yesterday and B the event that your sprinkler system went off yesterday. Suppose that your sprinkler system is set to randomly go off and so A and B are independent of one another. $P(A | B) = P(A)$. But now let C be the event that the grass is wet. The grass can be wet *either* due to the rain or due to the sprinkler. For conditional independence to hold here, then $P(A | C)$ must be equal to $P(A | B \cap C)$. But this is not true.

Let $P(A) = .5$ and $P(B) = .5$.

The marginal probability $P(C)$ is

$$P(C) = P(A \cup B) = P(A) + P(B) - P(A \cap B) = .75$$

The conditional probability $P(A|C)$ is

$$P(A|C) = \frac{P(C|A)P(A)}{P(C)} = \frac{1 \times .5}{.75} = \frac{2}{3}$$

The conditional probability $P(A|B \cap C)$ is

$$P(A|B \cap C) = \frac{P(C \cap B|A)P(A)}{P(C \cap B)} = \frac{P(C \cap B|A)P(A)}{P(C|B)P(B)} \frac{.5 \times .5}{.5} = \frac{1}{2}$$

Intuitively, given that the grass is wet, knowing that it rained yesterday tells us that it is *less* likely that the sprinkler also went off!

6.5 Random Variables

Most questions in the social sciences involve events, rather than numbers per se. To analyze and reason about events quantitatively, we need a way of mapping events to numbers. A random variable does exactly that.

Definition 6.6.

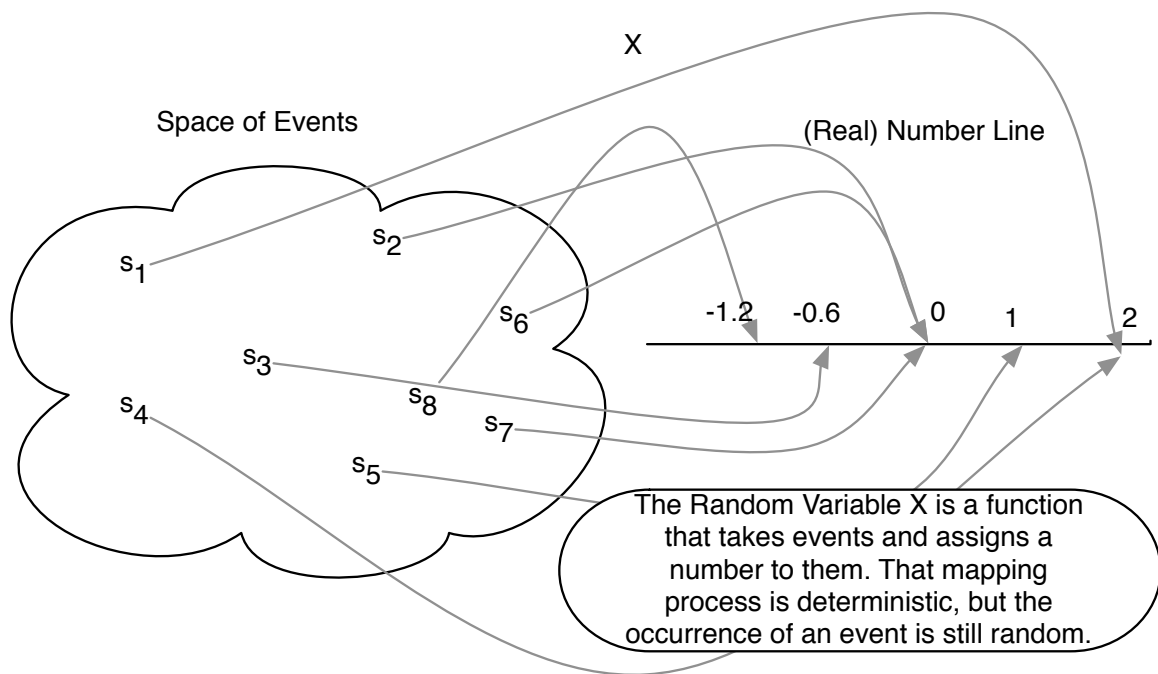


Figure 6.1: The Random Variable as a Real-Valued Function

Random Variable

A random variable is a measurable function X that maps from the sample space S to the set of real numbers R . It assigns a real number to every outcome $s \in S$.

Figure 6.1 shows a image of the function. It might seem strange to define a random variable as a function – which is neither random nor variable. The randomness comes from the realization of an event from the sample space s .

Randomness means that the outcome of some experiment is not deterministic, i.e. there is some probability ($0 < P(A) < 1$) that the event will occur.

The support of a random variable is all values for which there is a positive probability of occurrence.

Example: Flip a fair coin two times. What is the sample space?

A random variable must map events to the real line. For example, let a random variable X be the number of heads. The event (H, H) gets mapped to 2 ($X(s) = 2$), and the events $\{(H, T), (T, H)\}$ gets mapped to 1 ($X(s) = 1$), the event (T, T) gets mapped to 0 ($X(s) = 0$).

What are other possible random variables?

6.6 Distributions

We now have two main concepts in this section – probability and random variables. Given a sample space S and the same experiment, both probability and random variables take events as their inputs. But they output different things (probabilities measure the “size” of events, random variables give a number in a way that the analyst chose to define the random variable). How do the two concepts relate?

The concept of distributions is the natural bridge between these two concepts.

Definition 6.7.

Distribution of a random variable

A distribution of a random variable is a function that specifies the probabilities of all events associated with that random variable. There are several types of distributions: A probability mass function for a discrete random variable and probability density function for a continuous random variable.

Notice how the definition of distributions combines two ideas of random variables and probabilities of events. First, the distribution considers a random variable, call it X . X can take a number of possible numeric values.

Example 6.7.

Total Number of Occurrences

Consider three binary outcomes, one for each patient recovering from a disease: R_i denotes the event in which patient i ($i = 1, 2, 3$) recovers from a disease. R_1 , R_2 , and R_3 . How would we represent the total number of people who end up recovering from the disease?

Solution. Define the random variable X be the total number of people (out of three) who recover from the disease. Random variables are functions, that take as an input a set of events (in the sample space S) and deterministically assigns them to a number of the analyst's choice.

Recall that with each of these numerical values there is a class of *events*. In the previous example, for $X = 3$ there is one outcome (R_1, R_2, R_3) and for $X = 1$ there are multiple $(\{(R_1, R_2^c, R_3^c), (R_1^c, R_2, R_3^c), (R_1^c, R_2^c, R_3), \})$. Now, the thing to notice here is that each of these events naturally come with a probability associated with them. That is, $P(R_1, R_2, R_3)$ is a number from 0 to 1, as is $P(R_1, R_2^c, R_3^c)$. These all have probabilities because they are in the sample space S . The function that tells us these probabilities that are associated with a numerical value of a random variable is called a distribution.

In other words, a random variable X *induces a probability distribution* P (sometimes written P_X to emphasize that the probability density is about the r.v. X)

Discrete Random Variables

The formal definition of a random variable is easier to given by separating out two cases: discrete random variables when the numeric summaries of the events are discrete, and continuous random variables when they are continuous.

Definition 6.8.

Discrete Random Variable

X is a discrete random variable if it can assume only a finite or countably infinite number of distinct values. Examples: number of wars per year, heads or tails.

The distribution of a discrete r.v. is a PMF:

Definition 6.9.

Probability Mass Function

For a discrete random variable X , the probability mass function (Also referred to simply as the “probability distribution.”) (PMF), $p(x) = P(X = x)$, assigns probabilities to a countable number of distinct x values such that

1. $0 \leq p(x) \leq 1$
2. $\sum_y p(x) = 1$

Example: For a fair six-sided die, there is an equal probability of rolling any number. Since there are six sides, the probability mass function is then $p(y) = 1/6$ for $y = 1, \dots, 6$, 0 otherwise.}

In a discrete random variable, **cumulative distribution function** , $F(x)$ or $P(X \leq x)$, is the probability that X is less than or equal to some value x , or

$$P(X \leq x) = \sum_{i \leq x} p(i)$$

Properties a CDF must satisfy:

1. $F(x)$ is non-decreasing in x .
2. $\lim_{x \rightarrow -\infty} F(x) = 0$ and $\lim_{x \rightarrow \infty} F(x) = 1$
3. $F(x)$ is right-continuous.

Note that $P(X > x) = 1 - P(X \leq x)$.

Definition 6.10. For a fair six-sided die with its value as Y , What are the following?

1. $P(Y \leq 1)$
2. $P(Y \leq 3)$
3. $P(Y \leq 6)$

Continuous Random Variables

We also have a similar definition for *continuous* random variables.

Definition 6.11.

Continuous Random Variable

X is a continuous random variable if there exists a nonnegative function $f(x)$ defined for all real $x \in (-\infty, \infty)$, such that for any interval A , $P(X \in A) = \int_A f(x)dx$. Examples: age, income, GNP, temperature.

Definition 6.12.

Probability density function

The function f above is called the probability density function (pdf) of X and must satisfy

$$f(x) \geq 0$$
$$\int_{-\infty}^{\infty} f(x)dx = 1$$

Note also that $P(X = x) = 0$ — i.e., the probability of any point y is zero.

While continuous random variables do not have a PMF (since the PMF would be 0 at every point), the cumulative distribution function is defined in the exact same way. The cumulative distribution gives the probability that Y lies on the interval $(-\infty, y)$ and is defined as

$$F(x) = P(X \leq x) = \int_{-\infty}^x f(s)ds$$

We can also make statements about the probability of Y falling in an interval $a \leq y \leq b$.

$$P(a \leq x \leq b) = \int_a^b f(x)dx$$

The PDF and CDF are linked by the integral: The CDF of the integral of the PDF:

$$f(x) = F'(x) = \frac{dF(x)}{dx}$$

Example 6.8.

Continuous R.V.

For $f(y) = 1$, $0 < y < 1$, find: (1) The CDF $F(y)$ and (2) The probability $P(0.5 < y < 0.75)$.

6.7 Joint Distributions

Often, we are interested in two or more random variables defined on the same sample space. The distribution of these variables is called a joint distribution. Joint distributions can be made up of any combination of discrete and continuous random variables.

Joint Probability Distribution: If both X and Y are random variable, their joint probability mass/density function assigns probabilities to each pair of outcomes

Discrete:

$$p(x, y) = P(X = x, Y = y)$$

such that $p(x, y) \in [0, 1]$ and

$$\sum \sum p(x, y) = 1$$

Continuous:

$$f(x, y); P((X, Y) \in A) = \iint_A f(x, y) dx dy$$

s.t. $f(x, y) \geq 0$ and

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) dx dy = 1$$

If X and Y are independent, then $P(X = x, Y = y) = P(X = x)P(Y = y)$ and $f(x, y) = f(x)f(y)$

Marginal Probability Distribution: probability distribution of only one of the two variables (ignoring information about the other variable), we can obtain the marginal distribution by summing/integrating across the variable that we don't care about:

- Discrete: $p_X(x) = \sum_i p(x, y_i)$
- Continuous: $f_X(x) = \int_{-\infty}^{\infty} f(x, y) dy$

Conditional Probability Distribution: probability distribution for one variable, holding the other variable fixed. Recalling from the previous lecture that $P(A|B) = \frac{P(A \cap B)}{P(B)}$, we can write the conditional distribution as

- Discrete: $p_{Y|X}(y|x) = \frac{p(x, y)}{p_X(x)}, \quad p_X(x) > 0$
- Continuous: $f_{Y|X}(y|x) = \frac{f(x, y)}{f_X(x)}, \quad f_X(x) > 0$

Exercise 6.4.

Discrete, Joint Distributions

Suppose we are interested in the outcomes of flipping a coin and rolling a 6-sided die at the same time. The sample space for this process contains 12 elements:

$$\{(H, 1), (H, 2), (H, 3), (H, 4), (H, 5), (H, 6), (T, 1), (T, 2), (T, 3), (T, 4), (T, 5), (T, 6)\}$$

We can define two random variables X and Y such that $X = 1$ if heads and $X = 0$ if tails, while Y equals the number on the die.

We can then make statements about the joint distribution of X and Y . What are the following?

1. $P(X = x)$
2. $P(Y = y)$
3. $P(X = x, Y = y)$
4. $P(X = x|Y = y)$
5. Are X and Y independent?

6.8 Expectation

We often want to summarize some characteristics of the distribution of a random variable. The most important summary is the expectation (or expected value, or mean), in which the possible values of a random variable are weighted by their probabilities.

Definition 6.13.

Expectation of a discrete R.V.

The expected value of a discrete random variable Y is

$$E(Y) = \sum_y yP(Y = y) = \sum_y yp(y)$$

In words, it is the weighted average of all possible values of Y , weighted by the probability that y occurs. It is not necessarily the number we would expect Y to take on, but the average value of Y after a large number of repetitions of an experiment.

Example 6.9.

Expectation of a discrete R.V.

What is the expectation of a fair, six-sided die?

Expectation of a Continuous Random Variable: The expected value of a continuous random variable is similar in concept to that of the discrete random variable, except that instead of summing using probabilities as weights, we integrate using the density to weight. Hence, the expected value of the continuous variable Y is defined by

$$E(Y) = \int_y y f(y) dy$$

Example 6.10.

Expectation of a continuous R.V.

Find $E(Y)$ for $f(y) = \frac{1}{1.5}$, $0 < y < 1.5$.

Expected Value of a Function

Remember: An Expected Value is a type of weighted average. We can extend this to composite functions. For random variable Y ,

If Y is Discrete with PMF $p(y)$,

$$E[g(Y)] = \sum_y g(y)p(y)$$

If Y is Continuous with PDF $f(y)$,

$$E[g(Y)] = \int_{-\infty}^{\infty} g(y)f(y)dy$$

Properties of Expected Values

Dealing with Expectations is easier when the thing inside is a sum. The intuition behind this that Expectation is an integral, which is a type of sum.

1. Expectation of a constant is a constant

$$E(c) = c$$

2. Constants come out

$$E(cg(Y)) = cE(g(Y))$$

3. Expectation is Linear

$$E(g(Y_1) + \dots + g(Y_n)) = E(g(Y_1)) + \dots + E(g(Y_n)),$$

regardless of independence

4. Expected Value of Expected Values:

$$E(E(Y)) = E(Y)$$

(because the expected value of a random variable is a constant)

Finally, if X and Y are independent, even products are easy:

$$E(XY) = E(X)E(Y)$$

Conditional Expectation: With joint distributions, we are often interested in the expected value of a variable Y if we could hold the other variable X fixed. This is the conditional expectation of Y given $X = x$:

1. Y discrete: $E(Y|X = x) = \sum_y y p_{Y|X}(y|x)$
2. Y continuous: $E(Y|X = x) = \int_y y f_{Y|X}(y|x) dy$

The conditional expectation is often used for prediction when one knows the value of X but not Y

We can obtain a marginal expectation from the conditional expectation by way of the **Law of Total Expectation**

$$E[Y] = E[E[Y|X]]$$

For discrete r.v.:

$$E[Y] = \sum_{x \in \mathcal{X}} E[Y|X = x] P(X = x)$$

where $\mathcal{X}$ is the support of X

For continuous r.v.:

$$E[Y] = \int_{\mathcal{X}} E[Y|X = x] f_X(x) dx$$

where $f_X()$ denotes the PDF of X

6.9 Variance and Covariance

We can also look at other summaries of the distribution, which build on the idea of taking expectations. Variance tells us about the “spread” of the distribution; it is the expected value of the squared deviations from the mean of the distribution. The standard deviation is simply the square root of the variance.

Definition 6.14. The Variance of a Random Variable Y is

$$\text{Var}(Y) = E[(Y - E(Y))^2] = E(Y^2) - [E(Y)]^2$$

The Standard Deviation is the square root of the variance :

$$SD(Y) = \sigma_Y = \sqrt{\text{Var}(Y)}$$

Example 6.11. Given the following PMF:

$$f(x) = \begin{cases} \frac{3!}{x!(3-x)!} \left(\frac{1}{2}\right)^3 & x = 0, 1, 2, 3 \\ 0 & \text{otherwise} \end{cases}$$

What is $\text{Var}(x)$?

Hint: First calculate $E(X)$ and $E(X^2)$

Definition 6.15.

Covariance

The covariance measures the degree to which two random variables vary together; if the covariance between X and Y is positive, X tends to be larger than its mean when Y is larger than its mean.

$$\text{Cov}(X, Y) = E[(X - E(X))(Y - E(Y))]$$

We can also write this as

$$\begin{aligned}\text{Cov}(X, Y) &= E(XY - XE(Y) - E(X)Y + E(X)E(Y)) \\ &= E(XY) - E(X)E(Y) - E(X)E(Y) + E(X)E(Y) \\ &= E(XY) - E(X)E(Y)\end{aligned}$$

The covariance of a variable with itself is the variance of that variable.

The Covariance is unfortunately hard to interpret in magnitude. The correlation is a standardized version of the covariance, and always ranges from -1 to 1.

Definition 6.16.

Correlation

The correlation coefficient is the covariance divided by the standard deviations of X and Y . It is a unitless measure and always takes on values in the interval $[-1, 1]$.

$$\text{Corr}(X, Y) = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X)\text{Var}(Y)}} = \frac{\text{Cov}(X, Y)}{SD(X)SD(Y)}$$

Properties of Variance and Covariance:

1. $\text{Var}(c) = 0$
2. $\text{Var}(cY) = c^2\text{Var}(Y)$
3. $\text{Cov}(Y, Y) = \text{Var}(Y)$
4. $\text{Cov}(X, Y) = \text{Cov}(Y, X)$
5. $\text{Cov}(aX, bY) = ab\text{Cov}(X, Y)$
6. $\text{Cov}(X + a, Y) = \text{Cov}(X, Y)$
7. $\text{Cov}(X + Z, Y + W) = \text{Cov}(X, Y) + \text{Cov}(X, W) + \text{Cov}(Z, Y) + \text{Cov}(Z, W)$
8. $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X, Y)$

Exercise 6.5.

Expectation and Variance 1

Suppose we have a PMF with the following characteristics:

$$P(X = -2) = \frac{1}{5}$$

$$P(X = -1) = \frac{1}{6}$$

$$P(X = 0) = \frac{1}{5}$$

$$P(X = 1) = \frac{1}{15}$$

$$P(X = 2) = \frac{11}{30}$$

1. Calculate the expected value of X

Define the random variable $Y = X^2$.

2. Calculate the expected value of Y. (Hint: It would help to derive the PMF of Y first in order to calculate the expected value of Y in a straightforward way)
3. Calculate the variance of X.

Exercise 6.6.

Expectation and Variance 2

1. Find the expectation and variance

Given the following PDF:

$$f(x) = \begin{cases} \frac{3}{10}(3x - x^2) & 0 \leq x \leq 2 \\ 0 & \text{otherwise} \end{cases}$$

Exercise 6.7.

Expectation and Variance 3

1. Find the mean and standard deviation of random variable X . The PDF of this X is as follows:

$$f(x) = \begin{cases} \frac{1}{4}x & 0 \leq x \leq 2 \\ \frac{1}{4}(4-x) & 2 \leq x \leq 4 \\ 0 & \text{otherwise} \end{cases}$$

2. Next, calculate $P(X < \mu - \sigma)$. Remember, μ is the mean and σ is the standard deviation

6.10 Distributions

A distribution is defined by its cumulative distribution function. There are many common distributions that have useful properties that appear in probability and statistics.

Two *discrete* distributions used often are:

Definition 6.17.

Binomial Distribution

Y is distributed binomial if it represents the number of “successes” observed in n independent, identical “trials,” where the probability of success in any trial is p and the probability of failure is $q = 1 - p$.

For any particular sequence of y successes and $n - y$ failures, the probability of obtaining that sequence is $p^y q^{n-y}$ (by the multiplicative law and independence). However, there are $\binom{n}{y} = \frac{n!}{(n-y)!y!}$ ways of obtaining a sequence with y successes and $n - y$ failures. So the binomial distribution is given by

$$p(y) = \binom{n}{y} p^y q^{n-y}, \quad y = 0, 1, 2, \dots, n$$

with mean $\mu = E(Y) = np$ and variance $\sigma^2 = \text{Var}(Y) = npq$.

Example 6.12.

Binomial distribution

Republicans vote for Democrat-sponsored bills 2% of the time. What is the probability that out of 10 Republicans questioned, half voted for a particular Democrat-sponsored bill? What is the mean number of Republicans voting for Democrat-sponsored bills? The variance? 1. $P(Y = 5) = 1$. $E(Y) = 1$. $\text{Var}(Y) = 6$

Definition 6.18.

Poisson Distribution

A random variable Y has a Poisson distribution if

$$P(Y = y) = \frac{\lambda^y}{y!} e^{-\lambda}, \quad y = 0, 1, 2, \dots, \quad \lambda > 0$$

The Poisson has the unusual feature that its expectation equals its variance: $E(Y) = \text{Var}(Y) = \lambda$. The Poisson distribution is often used to model rare event counts: counts of the number of events that occur during some unit of time. λ is often called the “arrival rate.”

Example 6.13.

Poisson Distribution

Border disputes occur between two countries through a Poisson Distribution, at a rate of 2 per month. What is the probability of 0, 2, and less than 5 disputes occurring in a month?

Two *continuous* distributions used often are:

Definition 6.19.

Uniform Distribution

A random variable Y has a continuous uniform distribution on the interval (α, β) if its density is given by

$$f(y) = \frac{1}{\beta - \alpha}, \quad \alpha \leq y \leq \beta$$

The mean and variance of Y are $E(Y) = \frac{\alpha + \beta}{2}$ and $\text{Var}(Y) = \frac{(\beta - \alpha)^2}{12}$.

Example 6.14.

Uniform

For Y uniformly distributed over $(1, 3)$, what are the following probabilities?

1. $P(Y = 2)$
2. Its density evaluated at 2, or $f(2)$
3. $P(Y \leq 2)$
4. $P(Y > 2)$

Definition 6.20.

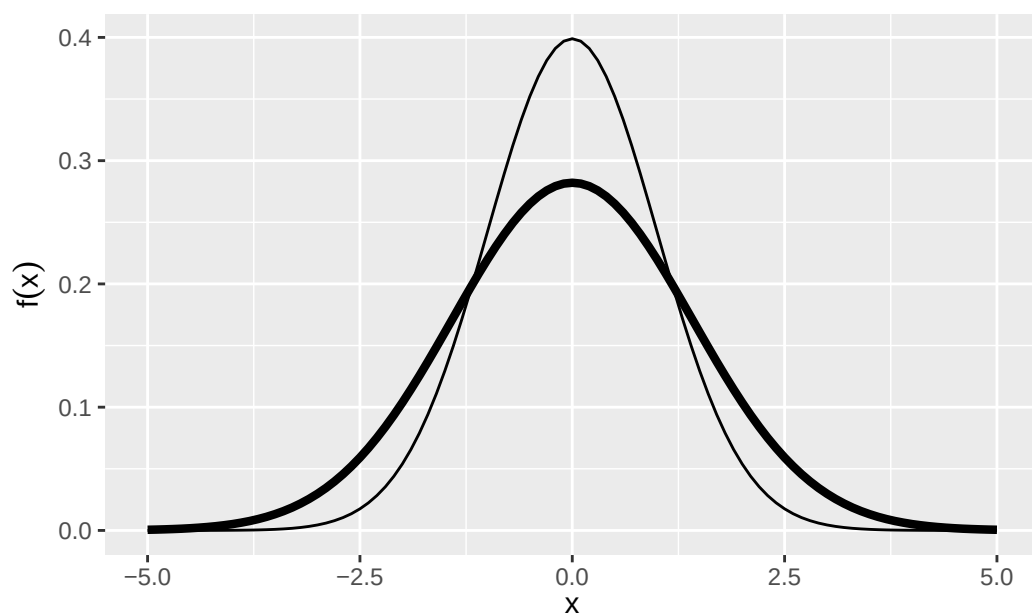
Normal Distribution

A random variable Y is normally distributed with mean $E(Y) = \mu$ and variance $\text{Var}(Y) = \sigma^2$ if its density is

$$f(y) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(y-\mu)^2}{2\sigma^2}}$$

See Figure 6.2 are various Normal Distributions with the same $\mu = 1$ and two versions of the variance.

Warning: Using ``size`` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use ``linewidth`` instead.



Thick line: variance = 2, Normal line: variance = 1

Figure 6.2: Normal Distribution Density

6.11 Summarizing Observed Events (Data)

So far, we've talked about distributions in a theoretical sense, looking at different properties of random variables. We don't observe random variables; we observe realizations of the random variable. These realizations of events are roughly equivalent to what we mean by "data". We'll spend more time in the intro class talking about this from the standpoint of *estimands*, *estimators* and *estimates*.

Sample mean: This is the most common measure of central tendency, calculated by summing across the observations and dividing by the number of observations.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Example:

X	6	3	7	5	5	5	6	4	7	2
Y	1	2	1	2	2	1	2	0	2	0

1. $\bar{x} =$ $\bar{y} =$
2. $\text{median}(x) =$ $\text{median}(y) =$
3. $m_x =$ $m_y =$

Dispersion: We also typically want to know how spread out the data are relative to the center of the observed distribution. There are several ways to measure dispersion.

Sample variance: The sample variance is the sum of the squared deviations from the sample mean, divided by the number of observations minus 1.

$$\widehat{\text{Var}}(X) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Again, this is an *estimator* of the variance of a random variable; we divide by $n-1$ instead of n in order to get an unbiased estimator.

Standard deviation: The sample standard deviation is the square root of the sample variance.

$$\widehat{SD}(X) = \sqrt{\widehat{\text{Var}}(X)} = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Covariance and Correlation: Both of these quantities measure the degree to which two variables vary together, and are estimates of the covariance and correlation of two random variables as defined above.

1. **Sample covariance:** $\hat{\text{Cov}}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$

2. **Sample correlation:** $\hat{\text{Corr}} = \frac{\hat{\text{Cov}}(X, Y)}{\sqrt{\hat{\text{Var}}(X)\hat{\text{Var}}(Y)}}$

Example 6.15.

Sample Covariance and Correlation

Example: Using the above table, calculate the sample versions of:

1. $\text{Cov}(X, Y)$
2. $\text{Corr}(X, Y)$

6.12 Asymptotic Theory

In theoretical and applied research, asymptotic arguments are often made. In this section we briefly introduce some of this material.

What are asymptotics? In probability theory, asymptotic analysis is the study of limiting behavior. By limiting behavior, we mean the behavior of some random process as the number of observations gets larger and larger.

Why is this important? We rarely know the true process governing the events we see in the social world. It is helpful to understand how such unknown processes theoretically must behave and asymptotic theory helps us do this.

6.12.1 CLT and LLN

We are now finally ready to revisit, with a bit more precise terms, the two pillars of statistical theory we motivated Section @ref(limitsfun) with.

Theorem 6.1.

Central Limit Theorem

Let $\{X_n\} = \{X_1, X_2, \dots\}$ be a sequence of i.i.d. random variables with finite mean (μ) and variance (σ^2). Then, the sample mean $\bar{X}_n = \frac{X_1 + X_2 + \dots + X_n}{n}$ increasingly converges into a Normal distribution.

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \text{Normal}(0, 1),$$

Another way to write this as a probability statement is that for all real numbers a ,

$$P\left(\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \leq a\right) \rightarrow \Phi(a)$$

as $n \rightarrow \infty$, where

$$\Phi(x) = \int_{-\infty}^x \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}} dx$$

is the CDF of a Normal distribution with mean 0 and variance 1.

This result means that, as n grows, the distribution of the sample mean $\bar{X}_n = \frac{1}{n}(X_1 + X_2 + \dots + X_n)$ is approximately normal with mean μ and standard deviation $\frac{\sigma}{\sqrt{n}}$, i.e.,

$$\bar{X}_n \approx \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right).$$

The standard deviation of $\bar{X}_n$ (which is roughly a measure of the precision of $\bar{X}_n$ as an estimator of μ) decreases at the rate $1/\sqrt{n}$, so, for example, to increase its precision by 10 (i.e., to get one more digit right), one needs to collect $10^2 = 100$ times more units of data.

Intuitively, this result also justifies that whenever a lot of small, independent processes somehow combine together to form the realized observations, practitioners often feel comfortable assuming Normality.

Theorem 6.2.

Weak Law of Large Numbers (LLN)

For any draw of independent random variables with the same mean μ , the sample average after n draws, $\bar{X}_n = \frac{1}{n}(X_1 + X_2 + \dots + X_n)$, converges in probability to the expected value of X , μ as $n \rightarrow \infty$:

$$\lim_{n \rightarrow \infty} P(|\bar{X}_n - \mu| > \varepsilon) = 0$$

A shorthand of which is $\bar{X}_n \xrightarrow{p} \mu$, where the arrow is read as “converges in probability to” as $n \rightarrow \infty$. In other words, $P(\lim_{n \rightarrow \infty} \bar{X}_n = \mu) = 1$. This is an important motivation for the widespread use of the sample mean, as well as the intuition link between averages and expected values.

More precisely this version of the LLN is called the *weak* law of large numbers because it leaves open the possibility that $|\bar{X}_n - \mu| > \varepsilon$ occurs many times. The *strong* law of large numbers states that, under a few more conditions, the probability that the limit of the sample average is the true mean is 1 (and other possibilities occur with probability 0), but the difference is rarely consequential in practice.

The Strong Law of Large Numbers holds so long as the expected value exists; no other assumptions are needed. However, the rate of convergence will differ greatly depending on the distribution underlying the observed data. When extreme observations occur often (i.e. kurtosis is large), the rate of convergence is much slower. Cf. The distribution of financial returns.

6.12.2 Big $\mathcal{O}$ Notation

Some of you may encounter “big-OH”-notation. If f, g are two functions, we say that $f = \mathcal{O}(g)$ if there exists some constant, c , such that $f(n) \leq c \times g(n)$ for large enough n . This notation is useful for simplifying complex problems in game theory, computer science, and statistics.

Example 6.16. What is $\mathcal{O}(5 \exp(0.5n) + n^2 + n/2)$? Answer: $\exp(n)$. Why? Because, for large n ,

$$\frac{5 \exp(0.5n) + n^2 + n/2}{\exp(n)} \leq \frac{c \exp(n)}{\exp(n)} = c.$$

whenever $n > 4$ and where $c = 1$.

7 Linear Algebra

Topics: • Working with Vectors • Linear Independence • Basics of Matrix Algebra • Square Matrices • Linear Equations • Systems of Linear Equations • Systems of Equations as Matrices • Solving Augmented Matrices and Systems of Equations • Rank • The Inverse of a Matrix • Inverse of Larger Matrices

7.1 Working with Vectors

Vector: A vector in n -space is an ordered list of n numbers. These numbers can be represented as either a row vector or a column vector:

$$\mathbf{v} (v_1 \ v_2 \ \dots \ v_n), \mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix}$$

We can also think of a vector as defining a point in n -dimensional space, usually $\mathbf{R}^n$; each element of the vector defines the coordinate of the point in a particular direction.

Vector Addition and Subtraction: If two vectors, $\mathbf{u}$ and $\mathbf{v}$, have the same length (i.e. have the same number of elements), they can be added (subtracted) together:

$$\mathbf{u} + \mathbf{v} = (u_1 + v_1 \ u_2 + v_2 \ \dots \ u_k + v_n)$$

$$\mathbf{u} - \mathbf{v} = (u_1 - v_1 \ u_2 - v_2 \ \dots \ u_k - v_n)$$

Scalar Multiplication: The product of a scalar c (i.e. a constant) and vector $\mathbf{v}$ is:

$$c\mathbf{v} = (cv_1 \ cv_2 \ \dots \ cv_n)$$

Vector Inner Product: The inner product (also called the dot product or scalar product) of two vectors $\mathbf{u}$ and $\mathbf{v}$ is again defined if and only if they have the same number of elements

$$\mathbf{u} \cdot \mathbf{v} = u_1v_1 + u_2v_2 + \dots + u_nv_n = \sum_{i=1}^n u_iv_i$$

If $\mathbf{u} \cdot \mathbf{v} = 0$, the two vectors are orthogonal (or perpendicular).

Vector Norm: The norm of a vector is a measure of its length. There are many different ways to calculate the norm, but the most common is the Euclidean norm (which corresponds to our usual conception of distance in three-dimensional space):

$$\|\mathbf{v}\| = \sqrt{\mathbf{v} \cdot \mathbf{v}} = \sqrt{v_1v_1 + v_2v_2 + \cdots + v_nv_n}$$

Example 7.1.

Vector Algebra

Let $a = (2 \ 1 \ 2)$, $b = (3 \ 4 \ 5)$. Calculate the following:

1. $a - b$
2. $a \cdot b$

Exercise 7.1.

Vector Algebra

Let $u = \begin{pmatrix} 7 & 1 & -5 & 3 \end{pmatrix}$, $v = \begin{pmatrix} 9 & -3 & 2 & 8 \end{pmatrix}$, $w = \begin{pmatrix} 1 & 13 & -7 & 2 & 15 \end{pmatrix}$, and $c = 2$. Calculate the following:

1. $u - v$
2. cw
3. $u \cdot v$
4. $w \cdot v$

7.2 Linear Independence

Linear combinations: The vector $\mathbf{u}$ is a linear combination of the vectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$ if

$$\mathbf{u} = c_1 \mathbf{v}_1 + c_2 \mathbf{v}_2 + \dots + c_k \mathbf{v}_k$$

For example, $\begin{pmatrix} 9 & 13 & 17 \end{pmatrix}$ is a linear combination of the following three vectors: $\begin{pmatrix} 1 & 2 & 3 \end{pmatrix}$, $\begin{pmatrix} 2 & 3 & 4 \end{pmatrix}$, and $\begin{pmatrix} 3 & 4 & 5 \end{pmatrix}$. This is because $\begin{pmatrix} 9 & 13 & 17 \end{pmatrix} = (2) \begin{pmatrix} 1 & 2 & 3 \end{pmatrix} + (-1) \begin{pmatrix} 2 & 3 & 4 \end{pmatrix} + 3 \begin{pmatrix} 3 & 4 & 5 \end{pmatrix}$

Linear independence: A set of vectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$ is linearly independent if the only solution to the equation

$$c_1 \mathbf{v}_1 + c_2 \mathbf{v}_2 + \dots + c_k \mathbf{v}_k = \mathbf{0}$$

is $c_1 = c_2 = \dots = c_k = 0$. If another solution exists, the set of vectors is linearly dependent.

A set S of vectors is linearly dependent if and only if at least one of the vectors in S can be written as a linear combination of the other vectors in S .

Linear independence is only defined for sets of vectors with the same number of elements; any linearly independent set of vectors in n -space contains at most n vectors.

Since $\begin{pmatrix} 9 & 13 & 17 \end{pmatrix}$ is a linear combination of $\begin{pmatrix} 1 & 2 & 3 \end{pmatrix}$, $\begin{pmatrix} 2 & 3 & 4 \end{pmatrix}$, and $\begin{pmatrix} 3 & 4 & 5 \end{pmatrix}$, these 4 vectors constitute a linearly dependent set.

Example 7.2.

Linear independence

Are the following sets of vectors linearly independent?

1. $\begin{pmatrix} 2 & 3 & 1 \end{pmatrix}$ and $\begin{pmatrix} 4 & 6 & 1 \end{pmatrix}$
2. $\begin{pmatrix} 1 & 0 & 0 \end{pmatrix}$, $\begin{pmatrix} 0 & 5 & 0 \end{pmatrix}$, and $\begin{pmatrix} 10 & 10 & 0 \end{pmatrix}$

Exercise 7.2.

Linear independence

Are the following sets of vectors linearly independent?

1. $\mathbf{v}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \mathbf{v}_2 = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}, \mathbf{v}_3 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$

2. $\mathbf{v}_1 = \begin{pmatrix} 2 \\ 2 \\ 1 \end{pmatrix}, \mathbf{v}_2 = \begin{pmatrix} -4 \\ 6 \\ 5 \end{pmatrix}, \mathbf{v}_3 = \begin{pmatrix} -2 \\ 8 \\ 6 \end{pmatrix}$

7.3 Basics of Matrix Algebra

Matrix: A matrix is an array of real numbers arranged in m rows by n columns. The dimensionality of the matrix is defined as the number of rows by the number of columns, $m \times n$.

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

Note that you can think of vectors as special cases of matrices; a column vector of length k is a $k \times 1$ matrix, while a row vector of the same length is a $1 \times k$ matrix.

It's also useful to think of matrices as being made up of a collection of row or column vectors. For example,

$$\mathbf{A} = (\mathbf{a}_1 \quad \mathbf{a}_2 \quad \cdots \quad \mathbf{a}_m)$$

Matrix Addition: Let $\mathbf{A}$ and $\mathbf{B}$ be two $m \times n$ matrices.

$$\mathbf{A} + \mathbf{B} = \begin{pmatrix} a_{11} + b_{11} & a_{12} + b_{12} & \cdots & a_{1n} + b_{1n} \\ a_{21} + b_{21} & a_{22} + b_{22} & \cdots & a_{2n} + b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} + b_{m1} & a_{m2} + b_{m2} & \cdots & a_{mn} + b_{mn} \end{pmatrix}$$

Note that matrices **A** and **B** must have the same dimensionality, in which case they are **conformable for addition**.

Example 7.3.

Matrix addition

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 1 & 2 \end{pmatrix}$$

$$\mathbf{A} + \mathbf{B} =$$

Scalar Multiplication: Given the scalar s , the scalar multiplication of $s\mathbf{A}$ is

$$s\mathbf{A} = s \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} = \begin{pmatrix} sa_{11} & sa_{12} & \cdots & sa_{1n} \\ sa_{21} & sa_{22} & \cdots & sa_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ sa_{m1} & sa_{m2} & \cdots & sa_{mn} \end{pmatrix}$$

Example 7.4.

Scalar Multiplication

$$s = 2, \quad \mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$

$$s\mathbf{A} =$$

Matrix Multiplication: If $\mathbf{A}$ is an $m \times k$ matrix and $\mathbf{B}$ is a $k \times n$ matrix, then their product $\mathbf{C} = \mathbf{AB}$ is the $m \times n$ matrix where

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{ik}b_{kj}$$

Example 7.5.

Matrix multiplication

$$1. \begin{pmatrix} a & b \\ c & d \\ e & f \end{pmatrix} \begin{pmatrix} A & B \\ C & D \end{pmatrix} =$$

$$2. \begin{pmatrix} 1 & 2 & -1 \\ 3 & 1 & 4 \end{pmatrix} \begin{pmatrix} -2 & 5 \\ 4 & -3 \\ 2 & 1 \end{pmatrix} =$$

Note that the number of columns of the first matrix must equal the number of rows of the second matrix, in which case they are **conformable for multiplication**. The sizes of the matrices (including the resulting product) must be

$$(m \times k)(k \times n) = (m \times n)$$

Also note that if $\mathbf{AB}$ exists, $\mathbf{BA}$ exists only if $\dim(\mathbf{A}) = m \times n$ and $\dim(\mathbf{B}) = n \times m$.

This does not mean that $\mathbf{AB} = \mathbf{BA}$. $\mathbf{AB} = \mathbf{BA}$ is true only in special circumstances, like when $\mathbf{A}$ or $\mathbf{B}$ is an identity matrix or $\mathbf{A} = \mathbf{B}^{-1}$.

Laws of Matrix Algebra:

1. Associative: $(A + B) + C = A + (B + C)$
 $(AB)C = A(BC)$
2. Commutative: $\mathbf{A} + B = B + A$
3. Distributive: $\mathbf{A}(B + C) = AB + AC$
 $(A + B)C = AC + BC$

Commutative law for multiplication does not hold – the order of multiplication matters:

$$\mathbf{AB} \neq \mathbf{BA}$$

For example,

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ -1 & 3 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}$$
$$\mathbf{AB} = \begin{pmatrix} 2 & 3 \\ -2 & 2 \end{pmatrix}, \quad \mathbf{BA} = \begin{pmatrix} 1 & 7 \\ -1 & 3 \end{pmatrix}$$

Transpose: The transpose of the $m \times n$ matrix $\mathbf{A}$ is the $n \times m$ matrix $\mathbf{A}^T$ (also written $\mathbf{A}'$) obtained by interchanging the rows and columns of $\mathbf{A}$.

For example,

$$\mathbf{A} = \begin{pmatrix} 4 & -2 & 3 \\ 0 & 5 & -1 \end{pmatrix}, \quad \mathbf{A}^T = \begin{pmatrix} 4 & 0 \\ -2 & 5 \\ 3 & -1 \end{pmatrix}$$

$$\mathbf{B} = \begin{pmatrix} 2 \\ -1 \\ 3 \end{pmatrix}, \quad \mathbf{B}^T = (2 \quad -1 \quad 3)$$

The following rules apply for transposed matrices:

1. $(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T$
2. $(\mathbf{A}^T)^T = \mathbf{A}$
3. $(s\mathbf{A})^T = s\mathbf{A}^T$
4. $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$; and by induction $(\mathbf{ABC})^T = \mathbf{C}^T \mathbf{B}^T \mathbf{A}^T$

Example of $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$:

$$\mathbf{A} = \begin{pmatrix} 1 & 3 & 2 \\ 2 & -1 & 3 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 0 & 1 \\ 2 & 2 \\ 3 & -1 \end{pmatrix}$$

$$(\mathbf{AB})^T = \left[\begin{pmatrix} 1 & 3 & 2 \\ 2 & -1 & 3 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 2 & 2 \\ 3 & -1 \end{pmatrix} \right]^T = \begin{pmatrix} 12 & 7 \\ 5 & -3 \end{pmatrix}$$

$$\mathbf{B}^T \mathbf{A}^T = \begin{pmatrix} 0 & 2 & 3 \\ 1 & 2 & -1 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 3 & -1 \\ 2 & 3 \end{pmatrix} = \begin{pmatrix} 12 & 7 \\ 5 & -3 \end{pmatrix}$$

Exercise 7.3.

Matrix Multiplication

Let

$$A = \begin{pmatrix} 2 & 0 & -1 & 1 \\ 1 & 2 & 0 & 1 \end{pmatrix}$$

$$B = \begin{pmatrix} 1 & 5 & -7 \\ 1 & 1 & 0 \\ 0 & -1 & 1 \\ 2 & 0 & 0 \end{pmatrix}$$

$$C = \begin{pmatrix} 3 & 2 & -1 \\ 0 & 4 & 6 \end{pmatrix}$$

Calculate the following:

1.

$$AB$$

2.

$$BA$$

3.

$$(BC)^T$$

4.

$$BC^T$$

7.4 Systems of Linear Equations

Linear Equation: $a_1x_1 + a_2x_2 + \cdots + a_nx_n = b$

a_i are parameters or coefficients. x_i are variables or unknowns.

Linear because only one variable per term and degree is at most 1.

We are often interested in solving linear systems like

$$\begin{array}{rclcrcl} x & - & 3y & = & -3 \\ 2x & + & y & = & 8 \end{array}$$

More generally, we might have a system of m equations in n unknowns

$$\begin{array}{cccccccl} a_{11}x_1 & + & a_{12}x_2 & + & \cdots & + & a_{1n}x_n & = & b_1 \\ a_{21}x_1 & + & a_{22}x_2 & + & \cdots & + & a_{2n}x_n & = & b_2 \\ \vdots & & & & \vdots & & & & \vdots \\ a_{m1}x_1 & + & a_{m2}x_2 & + & \cdots & + & a_{mn}x_n & = & b_m \end{array}$$

A **solution** to a linear system of m equations in n unknowns is a set of n numbers $x_1, x_2, \dots, x_n$ that satisfy each of the m equations.

Example: $x = 3$ and $y = 2$ is the solution to the above 2×2 linear system. If you graph the two lines, you will find that they intersect at $(3, 2)$.

Does a linear system have one, no, or multiple solutions? For a system of 2 equations with 2 unknowns (i.e., two lines): __

One solution: The lines intersect at exactly one point.

No solution: The lines are parallel.

Infinite solutions: The lines coincide.

Methods to solve linear systems:

1. Substitution
2. Elimination of variables
3. Matrix methods

Exercise 7.4.

Linear Equations

Provide a system of 2 equations with 2 unknowns that has

1. one solution
2. no solution
3. infinite solutions

7.5 Systems of Equations as Matrices

Matrices provide an easy and efficient way to represent linear systems such as

$$\begin{array}{ccccccccc} a_{11}x_1 & + & a_{12}x_2 & + & \cdots & + & a_{1n}x_n & = & b_1 \\ a_{21}x_1 & + & a_{22}x_2 & + & \cdots & + & a_{2n}x_n & = & b_2 \\ \vdots & & & & \vdots & & \vdots & & \\ a_{m1}x_1 & + & a_{m2}x_2 & + & \cdots & + & a_{mn}x_n & = & b_m \end{array}$$

as

$$\mathbf{A}x = b$$

where

The $m \times n$ **coefficient matrix** $\mathbf{A}$ is an array of mn real numbers arranged in m rows by n columns:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

The unknown quantities are represented by the vector $\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$.

The right hand side of the linear system is represented by the vector $\mathbf{b} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$.

Augmented Matrix: When we append $\mathbf{b}$ to the coefficient matrix $\mathbf{A}$, we get the augmented matrix $\widehat{\mathbf{A}} = [\mathbf{A}|\mathbf{b}]$

$$\left(\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & & \ddots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{array} \right)$$

Exercise 7.5. Create an augmented matrix that represent the following system of equations:

$$2x_1 - 7x_2 + 9x_3 - 4x_4 = 8$$

$$41x_2 + 9x_3 - 5x_6 = 11$$

$$x_1 - 15x_2 - 11x_5 = 9$$

7.6 Finding Solutions to Augmented Matrices and Systems of Equations

Row Echelon Form: Our goal is to translate our augmented matrix or system of equations into row echelon form. This will provide us with the values of the vector $\mathbf{x}$ which solve the system. We use the row operations to change coefficients in the lower triangle of the augmented matrix to 0. An augmented matrix of the form

$$\left(\begin{array}{cccc|c} a'_{11} & a'_{12} & a'_{13} & \cdots & a'_{1n} & b'_1 \\ 0 & a'_{22} & a'_{23} & \cdots & a'_{2n} & b'_2 \\ 0 & 0 & a'_{33} & \cdots & a'_{3n} & b'_3 \\ 0 & 0 & 0 & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & a'_{mn} & b'_m \end{array} \right)$$

is said to be in row echelon form — each row has more leading zeros than the row preceding it.

Reduced Row Echelon Form: We can go one step further and put the matrix into reduced row echelon form. Reduced row echelon form makes the value of $\mathbf{x}$ which solves the system very obvious. For a system of m equations in m unknowns, with no all-zero rows, the reduced row echelon form would be

$$\left(\begin{array}{ccccc|c} \boxed{1} & 0 & 0 & 0 & 0 & b_1^* \\ 0 & \boxed{1} & 0 & 0 & 0 & b_2^* \\ 0 & 0 & \boxed{1} & 0 & 0 & b_3^* \\ 0 & 0 & 0 & \ddots & 0 & \vdots \\ 0 & 0 & 0 & 0 & \boxed{1} & b_m^* \end{array} \right)$$

Gaussian and Gauss-Jordan elimination: We can conduct elementary row operations to get our augmented matrix into row echelon or reduced row echelon form. The methods of transforming a matrix or system into row echelon and reduced row echelon form are referred to as Gaussian elimination and Gauss-Jordan elimination, respectively.

Elementary Row Operations: To do Gaussian and Gauss-Jordan elimination, we use three basic operations to transform the augmented matrix into another augmented matrix that represents an equivalent linear system – equivalent in the sense that the same values of x_j solve both the original and transformed matrix/system:

Interchanging Rows: Suppose we have the augmented matrix

$$\widehat{\mathbf{A}} = \left(\begin{array}{cc|c} a_{11} & a_{12} & b_1 \\ a_{21} & a_{22} & b_2 \end{array} \right)$$

If we interchange the two rows, we get the augmented matrix

$$\left(\begin{array}{cc|c} a_{21} & a_{22} & b_2 \\ a_{11} & a_{12} & b_1 \end{array} \right)$$

which represents a linear system equivalent to that represented by matrix $\widehat{\mathbf{A}}$.

Multiplying by a Constant: If we multiply the second row of matrix $\widehat{\mathbf{A}}$ by a constant c , we get the augmented matrix

$$\left(\begin{array}{cc|c} a_{11} & a_{12} & b_1 \\ ca_{21} & ca_{22} & cb_2 \end{array} \right)$$

which represents a linear system equivalent to that represented by matrix $\widehat{\mathbf{A}}$.

Adding (subtracting) Rows: If we add (subtract) the first row of matrix $\widehat{\mathbf{A}}$ to the second, we obtain the augmented matrix

$$\left(\begin{array}{cc|c} a_{11} & a_{12} & b_1 \\ a_{11} + a_{21} & a_{12} + a_{22} & b_1 + b_2 \end{array} \right)$$

which represents a linear system equivalent to that represented by matrix $\widehat{\mathbf{A}}$.

Example 7.6.

Solving systems of equations

Solve the following system of equations by using elementary row operations:

$$\begin{array}{rclcrcl} x & - & 3y & = & -3 \\ 2x & + & y & = & 8 \end{array}$$

Exercise 7.6.

Solving Systems of Equations

Put the following system of equations into augmented matrix form. Then, using Gaussian or Gauss-Jordan elimination, solve the system of equations by putting the matrix into row echelon or reduced row echelon form.

$$1. \begin{cases} x + y + 2z = 2 \\ 3x - 2y + z = 1 \\ y - z = 3 \end{cases}$$

$$2. \begin{cases} 2x + 3y - z = -8 \\ x + 2y - z = 12 \\ -x - 4y + z = -6 \end{cases}$$

7.7 Rank — and Whether a System Has One, Infinite, or No Solutions

To determine how many solutions exist, we can use information about (1) the number of equations m , (2) the number of unknowns n , and (3) the **rank** of the matrix representing the linear system.

Rank: The maximum number of linearly independent row or column vectors in the matrix. This is equivalent to the number of nonzero rows of a matrix in row echelon form. For any matrix $\mathbf{A}$, the row rank always equals column rank, and we refer to this number as the rank of $\mathbf{A}$.

For example

$$\begin{pmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 6 \end{pmatrix}$$

Rank = 3

$$\begin{pmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 0 \end{pmatrix}$$

Rank = 2

Exercise 7.7.

Rank of Matrices

Find the rank of each matrix below:

(Hint: transform the matrices into row echelon form. Remember that the number of nonzero rows of a matrix in row echelon form is the rank of that matrix)

1.

$$\begin{pmatrix} 1 & 1 & 2 \\ 2 & 1 & 3 \\ 1 & 2 & 3 \end{pmatrix}$$

2.

$$\begin{pmatrix} 1 & 3 & 3 & -3 & 3 \\ 1 & 3 & 1 & 1 & 3 \\ 1 & 3 & 2 & -1 & -2 \\ 1 & 3 & 0 & 3 & -2 \end{pmatrix}$$

7.8 The Inverse of a Matrix

Identity Matrix: The $n \times n$ identity matrix $\mathbf{I}_n$ is the matrix whose diagonal elements are 1 and all off-diagonal elements are 0. Examples:

$$\mathbf{I}_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{I}_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Inverse Matrix: An $n \times n$ matrix $\mathbf{A}$ is **nonsingular** or **invertible** if there exists an $n \times n$ matrix $\mathbf{A}^{-1}$ such that

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}_n$$

where $\mathbf{A}^{-1}$ is the inverse of $\mathbf{A}$. If there is no such $\mathbf{A}^{-1}$, then $\mathbf{A}$ is singular or not invertible.

Example: Let

$$\mathbf{A} = \begin{pmatrix} 2 & 3 \\ 2 & 2 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} -1 & \frac{3}{2} \\ 1 & -1 \end{pmatrix}$$

Since

$$\mathbf{AB} = \mathbf{BA} = \mathbf{I}_n$$

we conclude that $\mathbf{B}$ is the inverse, $\mathbf{A}^{-1}$, of $\mathbf{A}$ and that $\mathbf{A}$ is nonsingular.

Properties of the Inverse:

- If the inverse exists, it is unique.
- If $\mathbf{A}$ is nonsingular, then $\mathbf{A}^{-1}$ is nonsingular.
- $(\mathbf{A}^{-1})^{-1} = \mathbf{A}$
- If $\mathbf{A}$ and $\mathbf{B}$ are nonsingular, then $\mathbf{AB}$ is nonsingular
- $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$
- If $\mathbf{A}$ is nonsingular, then $(\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T$

Procedure to Find $\mathbf{A}^{-1}$: We know that if $\mathbf{B}$ is the inverse of $\mathbf{A}$, then

$$\mathbf{AB} = \mathbf{BA} = \mathbf{I}_n$$

Looking only at the first and last parts of this

$$\mathbf{AB} = \mathbf{I}_n$$

Solving for $\mathbf{B}$ is equivalent to solving for n linear systems, where each column of $\mathbf{B}$ is solved for the corresponding column in $\mathbf{I}_n$. We can solve the systems simultaneously by augmenting $\mathbf{A}$ with $\mathbf{I}_n$ and performing Gauss-Jordan elimination on $\mathbf{A}$. If Gauss-Jordan elimination on $[\mathbf{A}|\mathbf{I}_n]$ results in $[\mathbf{I}_n|\mathbf{B}]$, then $\mathbf{B}$ is the inverse of $\mathbf{A}$. Otherwise, $\mathbf{A}$ is singular.

To summarize: To calculate the inverse of $\mathbf{A}$

1. Form the augmented matrix $[\mathbf{A}|\mathbf{I}_n]$
2. Using elementary row operations, transform the augmented matrix to reduced row echelon form.
3. The result of step 2 is an augmented matrix $[\mathbf{C}|\mathbf{B}]$.
 - a. If $\mathbf{C} = \mathbf{I}_n$, then $\mathbf{B} = \mathbf{A}^{-1}$.
 - b. If $\mathbf{C} \neq \mathbf{I}_n$, then $\mathbf{C}$ has a row of zeros. This means $\mathbf{A}$ is singular and $\mathbf{A}^{-1}$ does not exist.

Example 7.7.

Matrix Inverse

Find the inverse of the following matrices:

1. $\mathbf{A} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 2 & 3 \\ 5 & 5 & 1 \end{pmatrix}$

Exercise 7.8.

Matrix Inverse

Find the inverse of the following matrix:

1. $\mathbf{A} = \begin{pmatrix} 1 & 0 & 4 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

7.9 Linear Systems and Inverses

Let's return to the matrix representation of a linear system

$$\mathbf{Ax} = \mathbf{b}$$

If $\mathbf{A}$ is an $n \times n$ matrix, then $\mathbf{Ax} = \mathbf{b}$ is a system of n equations in n unknowns. Suppose $\mathbf{A}$ is nonsingular. Then $\mathbf{A}^{-1}$ exists. To solve this system, we can multiply each side by $\mathbf{A}^{-1}$ and reduce it as follows:

$$\begin{aligned}\mathbf{A}^{-1}(\mathbf{Ax}) &= \mathbf{A}^{-1}\mathbf{b} \\ (\mathbf{A}^{-1}\mathbf{A})\mathbf{x} &= \mathbf{A}^{-1}\mathbf{b} \\ \mathbf{I}_n\mathbf{x} &= \mathbf{A}^{-1}\mathbf{b} \\ \mathbf{x} &= \mathbf{A}^{-1}\mathbf{b}\end{aligned}$$

Hence, given $\mathbf{A}$ and $\mathbf{b}$ and given that $\mathbf{A}$ is nonsingular, then $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ is a unique solution to this system.

Exercise 7.9.

Solve linear system using inverses

Use the inverse matrix to solve the following linear system:

$$\begin{aligned}-3x + 4y &= 5 \\ 2x - y &= -10\end{aligned}$$

Hint: the linear system above can be written in the matrix form

$$\mathbf{A}\mathbf{z} = \mathbf{b}$$

$$\text{given } \mathbf{A} = \begin{pmatrix} -3 & 4 \\ 2 & -1 \end{pmatrix}$$

$$\mathbf{z} = \begin{pmatrix} x \\ y \end{pmatrix} \text{ and } \mathbf{b} = \begin{pmatrix} 5 \\ -10 \end{pmatrix}$$

7.10 Determinants

Singularity: Determinants can be used to determine whether a square matrix is nonsingular.

A square matrix is nonsingular if and only if its determinant is not zero.

Determinant of a 1×1 matrix, $\mathbf{A}$, equals a_{11}

Determinant of a 2×2 matrix, $\mathbf{A}$, $\begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix}$:

$$\begin{aligned}\det(\mathbf{A}) &= |\mathbf{A}| \\ &= a_{11}a_{22} - a_{12}a_{21} \\ &= a_{11}a_{22} - a_{12}a_{21}\end{aligned}$$

We can extend the second to last equation above to get the definition of the determinant of a 3×3 matrix:

$$\begin{aligned}
\begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix} &= a_{11} \begin{vmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{vmatrix} - a_{12} \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix} + a_{13} \begin{vmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{vmatrix} \\
&= a_{11}(a_{22}a_{33} - a_{23}a_{32}) - a_{12}(a_{21}a_{33} - a_{23}a_{31}) + a_{13}(a_{21}a_{32} - a_{22}a_{31})
\end{aligned}$$

Let's extend this now to any $n \times n$ matrix. Let's define $\mathbf{A}_{ij}$ as the $(n-1) \times (n-1)$ submatrix of $\mathbf{A}$ obtained by deleting row i and column j . Let the (i, j) th **minor** of $\mathbf{A}$ be the determinant of $\mathbf{A}_{ij}$:

$$M_{ij} = |\mathbf{A}_{ij}|$$

Then for any $n \times n$ matrix $\mathbf{A}$

$$|\mathbf{A}| = a_{11}M_{11} - a_{12}M_{12} + \cdots + (-1)^{n+1}a_{1n}M_{1n}$$

For example, in figuring out whether the following matrix has an inverse?

$$\mathbf{A} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 2 & 3 \\ 5 & 5 & 1 \end{pmatrix}$$

1. Calculate its determinant.

$$\begin{aligned}
&= 1(2 - 15) - 1(0 - 15) + 1(0 - 10) \\
&= -13 + 15 - 10 \\
&= -8
\end{aligned}$$

2. Since $|\mathbf{A}| \neq 0$, we conclude that $\mathbf{A}$ has an inverse.

8 Determinants

Determine whether the following matrices are nonsingular:

$$1. \begin{pmatrix} 1 & 0 & 1 \\ 2 & 1 & 2 \\ 1 & 0 & -1 \end{pmatrix}$$

$$2. \begin{pmatrix} 2 & 1 & 2 \\ 1 & 0 & 1 \\ 4 & 1 & 4 \end{pmatrix}$$

8.1 Getting Inverse of a Matrix using its Determinant

Thus far, we have a number of algorithms to

1. Find the solution of a linear system,
2. Find the inverse of a matrix

but these remain just that — algorithms. At this point, we have no way of telling how the solutions x_j change as the parameters a_{ij} and b_i change, except by changing the values and “rerunning” the algorithms.

With determinants, we can provide an explicit formula for the inverse and therefore provide an explicit formula for the solution of an $n \times n$ linear system.

Hence, we can examine how changes in the parameters and b_i affect the solutions x_j .

Determinant Formula for the Inverse of a 2×2 :

The determinant of a 2×2 matrix $\mathbf{A} \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ is defined as:

$$\frac{1}{\det(\mathbf{A})} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

For example, Let’s calculate the inverse of matrix A from Exercise @ref(exr:invlinsys) using the determinant formula.

Recall,

$$A = \begin{pmatrix} -3 & 4 \\ 2 & -1 \end{pmatrix}$$

$$\det(\mathbf{A}) = (-3)(-1) - (4)(2) = 3 - 8 = -5$$

$$\frac{1}{\det(\mathbf{A})} \begin{pmatrix} -1 & -4 \\ -2 & -3 \end{pmatrix}$$

$$\frac{1}{-5} \begin{pmatrix} -1 & -4 \\ -2 & -3 \end{pmatrix}$$

$$\begin{pmatrix} \frac{1}{5} & \frac{4}{5} \\ \frac{2}{5} & \frac{3}{5} \end{pmatrix}$$

9 Calculate Inverse using Determinant Formula

Calculate the inverse of A

$$A = \begin{pmatrix} 3 & 5 \\ -7 & 2 \end{pmatrix}$$

10 Programming: Orientation and Reading in Data¹

Motivation: Data and You

The modal social science project starts by importing existing datasets. Datasets come in all shapes and sizes. As you search for new data you may encounter dozens of file extensions – csv, xlsx, dta, sav, por, Rdata, Rds, txt, xml, json, shp ... the list continues. Although these files can often be cumbersome, its a good to be able to find a way to encounter any file that your research may call for.

Reviewing data import will allow us to get on the same page on how computer systems work.

Where are we? Where are we headed?

Today we'll cover:

- What's what in RStudio
- What R is, at a high level
- How to read in data
- Comment on coding style on the way

Check your understanding

- What is the difference between a file and a folder?
- In the RStudio windows, what is the difference between the “Source” Pane and the “Console”? What is a “code chunk”?
- How do you read a R help page? What is the **Usage** section, the **Values** section, and the **Examples** section?
- What use is the “Environment” Pane?
- How would you read in a spreadsheet in R?
- How would you figure out what variables are in the data? size of the data?
- How would you read in a **csv** file, a **dta** file, a **sav** file?

¹Special thanks to Shiro Kuriwaki for developing the original version of this tutorial

10.1 General Orientation

1. RStudio is a **GUI** and an IDE for the programming language R. A Graphical User Interface allows users to interface with the software (in this case R) using graphical aids like buttons and tabs. Often we don't think of GUIs because to most computer users, everything is a GUI (like Microsoft Word or your "Control Panel"), but it's always there! A Integrated Development Environment just says that the software to interface with R comes with useful useful bells and whistles to give you shortcuts.

The **Console** is kind of a the core window through which you see your GUI actually operating through R. It's not graphical so might not be as intuitive. But all your results, commands, errors, warnings.. you see them in here. A console tells you what's going on now.

2. Via the GUI, you the analyst needs to send instructions, or **commands**, to the R application. The verb for this is "run" or "execute" the command. Computer programs ask users to provide instructions in very specific formats. While a English-speaking human can understand a sentence with a few typos in it by filling in the blanks, the same typo or misplaced character would halt a computer program. Each program has its own requirements for how commands should be typed; after all, each of these is its own language. We refer to the way a program needs its commands to be formatted as its **syntax**.
3. Theoretically, one could do all their work by typing in commands into the Console. But that would be a lot of work, because you'd have to give instructions each time you start your data analysis. Moreover, you'll have no record of what you did. That's why you need a **script**. This is a type of **code**. It can be referred to as a **source** because that is the source of your commands. Source is also used as a verb; "source the script" just means execute it. RStudio doesn't start out with a script, so you can make one from "File > New" or the New file icon.
4. You can also open scripts that are in folders in your computer. A script is a type of File. Find your Files in the bottom-right "Files" pane.

To load a dataset, you need to specify where that file is. Computer files (data, documents, programs) are organized hierarchically, like a branching tree. Folders can contain files, and also other folders. The GUI toolbar makes this lineaar and hiearchical relationship apparent. When we turn to locate the file in our commands, we need another set of syntax. Importantly, denote the hierarchy of a folder by the / (slash) symbol. `data/input/2018-08` indicates the 2018-08 folder, which is included in the `input` folder, which is in turn included in the `data` folder.

Files (but not folders) have "file extensions" which you are probably familiar with already: `.docx`, `.pdf`, and `.pdf`. The file extensions you will see in a stats or quantitative social science class are:

- **.pdf**: PDF, a convenient format to view documents and slides in, regardless of Mac/Windows.
 - **.csv**: A comma separated values file
 - **.xlsx**: Microsoft Excel file
 - **.dta**: Stata data
 - **.sav**: SPSS data
 - **.R**: R code (script)
 - **.Rmd**: Rmarkdown code (text + code)
 - **.do**: Stata code (script)
5. In R, there are two main types of scripts. A classic **.R** file and a **.Rmd** file (for Rmarkdown). A **.R** file is just lines and lines of R code that is meant to be inserted right into the Console. A **.Rmd** tries to weave code and English together, to make it easier for users to create reports that interact with data and intersperse R code with explanation.

Rmarkdown facilitates is the use of **code chunks**, which are used here. These start and end with three back-ticks. In the beginning, we can add options in curly braces (`{}`). Specifying `r` in the beginning tells to render it as R code. Options like `echo = TRUE` switch between showing the code that was executed or not; `eval = TRUE` switch between evaluating the code. More about Rmarkdown in later sections. For example, this code chunk would evaluate `1 + 1` and show its output when compiled, but not display the code that was executed.

10.2 But what is R

R is a programming language primarily used for statistical computing. It's free, open source and has an extensive community that is constantly developing new tools and packages that extend its functionality in a lot of different ways (for example, the **tidyverse** project).

One feature of many programming languages is that they allow for “object-oriented” programming. In an object-oriented programming paradigm, we work with “objects” – some sort of structure that exists in the computer memory – that contains attributes and on which we can execute code (methods). R's object-oriented support has some interesting quirks compared to other languages like C or Python, but

Everything in R is an object, including its most basic *data types*. R has six basic data types:

- character
- numeric (real or decimal)
- integer
- logical

- complex

These *data types* make up the basic data structures of R

- atomic vectors
- lists
- matrix
- data frame
- factors

Beyond that, many R routines

10.3 The Computer and You: Giving Instructions

We'll do the Peanut Butter and Jelly Exercise in class as an introduction to programming for those who are new.

Assignment: Take 5 minutes to write down on a piece of paper, how to make a peanut butter and jelly sandwich. Be as concise and unambiguous as possible so that a robot (who doesn't know what a PBJ is) would understand. You can assume that there will be loaf of sliced bread, a jar of jelly, a jar of peanut butter, and a knife.

Simpler assignment: Say we just want a robot to be able to tell us if we have enough ingredients to make a peanut butter and jelly sandwich. Write down instructions so that if told how many slices of bread, servings of peanut butter, and servings of jelly you have, the robot can tell you if you can make a PBJ.

Now, translate the simpler assignment into R code using the code below as a starting point:

```
n_bread <- 8
n_pb <- 3
n_jelly <- 9

# write instructions in R here
```

10.4 Base-R vs. tidyverse

One last thing before we jump into data. Many things in R and other open source packages have competing standards. A lecture on a technique inevitably biases one standard over

another. Right now among R users in this area, there are two families of functions: base-R and tidyverse. R instructors thus face a dilemma about which to teach primarily.²

In this prefresher, we try our best to choose the one that is most useful to the modal task of social science researchers, and make use of the tidyverse functions in most applications. but feel free to suggest changes to us or to the booklet.

Although you do not need to choose one over the other, for beginners it is confusing what is a tidyverse function and what is not. Many of the tidyverse *packages* are covered in this 2017 graphic below, and the cheat-sheets that other programmers have written: <https://www.rstudio.com/resources/cheatsheets/>

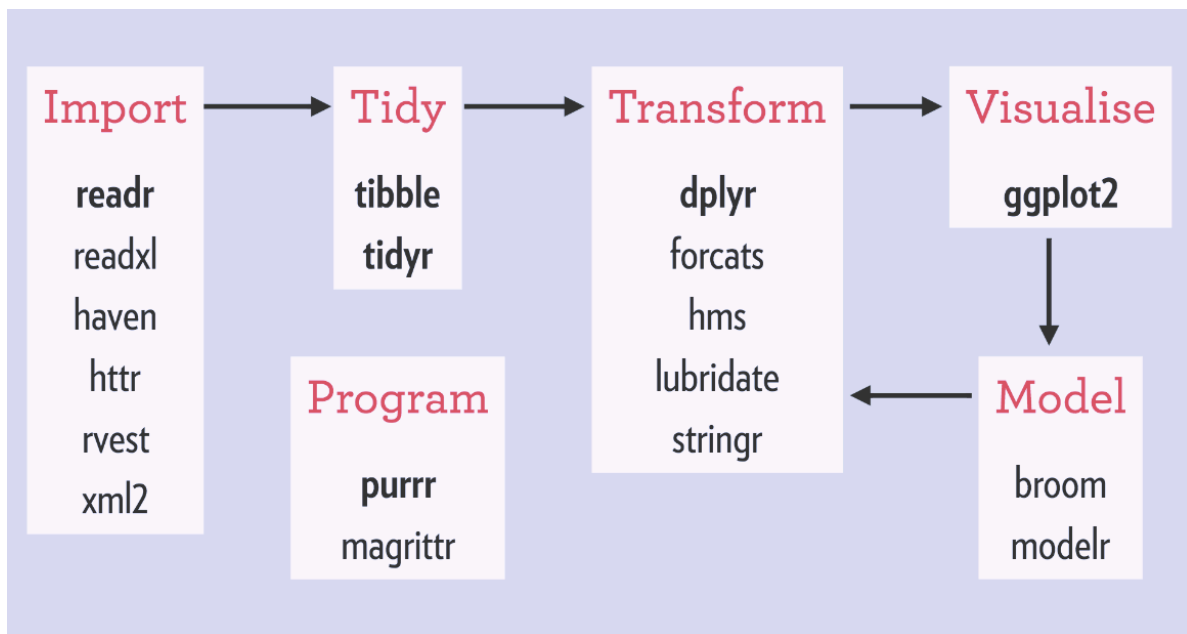


Figure 10.1: Names of Packages in the tidyverse Family

The following side-by-side comparison of commands for a particular function compares some tidyverse and non-tidyverse functions (which we refer to loosely as base-R). This list is not meant to be comprehensive and more to give you a quick rule of thumb.

Dataframe subsetting

²See for example this community discussion: <https://community.rstudio.com/t/base-r-and-the-tidyverse/2965/17>

In order to ...	in tidyverse:	in base-R:
Count each category	<code>count(df, var)</code>	<code>table(df\$var)</code>
Filter rows by condition	<code>filter(df, var == "Female")</code>	<code>df[df\$var == "Female",]</code> or <code>subset(df, var == "Female")</code>
Extract columns	<code>select(df, var1, var2)</code>	<code>df[, c("var1", "var2")]</code>
Extract a single column as a vector	<code>pull(df, var)</code>	<code>df[["var"]]</code> or <code>df[, "var"]</code>
Combine rows	<code>bind_rows()</code>	<code>rbind()</code>
Combine columns	<code>bind_cols()</code>	<code>cbind()</code>
Create a dataframe	<code>tibble(x = vec1, y = vec2)</code>	<code>data.frame(x = vec1, y = vec2)</code>
Turn a dataframe into a tidyverse dataframe	<code>tbl_df(df)</code>	

Remember that tidyverse applies to *dataframes* only, not vectors. For subsetting vectors, use the base-R functions with the square brackets.

Read data

Some non-tidyverse functions are not quite “base-R” but have similar relationships to tidyverse. For these, we recommend using the *tidyverse* functions as a general rule due to their common format, simplicity, and scalability.

In order to ...	in tidyverse:	in base-R:
Read a Excel file	<code>read_excel()</code>	<code>read.xlsx()</code>
Read a csv	<code>read_csv()</code>	<code>read.csv()</code>
Read a Stata file	<code>read_dta()</code>	<code>read.dta()</code>
Substitute strings	<code>str_replace()</code>	<code>gsub()</code>
Return matching strings	<code>str_subset()</code>	<code>grep(., value = TRUE)</code>
Merge <code>data1</code> and <code>data2</code> on variables <code>x1</code> and <code>x2</code>	<code>left_join(data1, data2, by = c("x1", "x2"))</code>	<code>merge(data1, data2, by.x = "x1", by.y = "x2", all.x = TRUE)</code>

Visualization

Plotting by `ggplot2` (from your tutorials) is also a tidyverse family.

In order to ...	in tidyverse:	in base-R:
Make a scatter plot	<code>ggplot(data, aes(x, y)) + geom_point()</code>	<code>plot(data\$x, data\$y)</code>
Make a line plot	<code>ggplot(data, aes(x, y)) + geom_line()</code>	<code>plot(data\$x, data\$y, type = "l")</code>
Make a histogram	<code>ggplot(data, aes(x, y)) + geom_histogram()</code>	<code>hist(data\$x, data\$y)</code>

10.5 A is for Athens

For our first dataset, let's try reading in a dataset on the Ancient Greek world. Political Theorists and Political Historians study the domestic systems, international wars, cultures and writing of this era to understand the first instance of democracy, the rise and overturning of tyranny, and the legacies of political institutions.

This POLIS dataset was generously provided by Professor Josiah Ober of Stanford University. This dataset includes information on city states in the Ancient Greek world, parts of it collected by careful work by historians and archaeologists. It is part of his recent books on Greece (Ober 2015), “The Rise and Fall of Classical Greece”³ and *Institutions in Ancient Athens* (Ober 2010), “Democracy and Knowledge: Innovation and Learning in Classical Athens.”⁴

10.5.1 Locating the Data

What files do we have in the `data/input` folder?

<code>data/input/CES Guide 2022.pdf</code>	<code>data/input/Nunn_Wantchekon_AER_2011.dta</code>
<code>data/input/Nunn_Wantchekon_sample.dta</code>	<code>data/input/acs2015_1percent.csv</code>
<code>data/input/ces22_subset.dta</code>	<code>data/input/gapminder_wide.Rds</code>
<code>data/input/gapminder_wide.tab</code>	<code>data/input/german_credit.sav</code>
<code>data/input/justices_court-median.csv</code>	<code>data/input/ober_2018.xlsx</code>
<code>data/input/sample_mid.csv</code>	<code>data/input/sample_polity.csv</code>
<code>data/input/upshot-siena-polls.csv</code>	<code>data/input/usc2010_001percent.Rds</code>
<code>data/input/usc2010_001percent.csv</code>	

A typical file format is Microsoft Excel. Although this is not usually the best format for R because of its highly formatted structure as opposed to plain text, recent packages have made this fairly easy.

³Ober, Josiah (2015). *The Rise and Fall of Classical Greece*. Princeton University Press.

⁴Ober, Josiah (2010). *Democracy and Knowledge: Innovation and Learning in Classical Athens*. Princeton University Press.

10.5.2 Reading in Data

In Rstudio, a good way to start is to use the GUI and the Import tool. Once you click a file, an option to “Import Dataset” comes up. RStudio picks the right function for you, and you can copy that code, but it’s important to eventually be able to write that code yourself.

For the first time using an outside package, you first need to install it.

```
install.packages("readxl")
```

After that, you don’t need to install it again. But you **do** need to load it each time.

```
library(readxl)
```

The package `readxl` has a website: <https://readxl.tidyverse.org/>. Other packages are not as user-friendly, but they have a help page with a table of contents of all their functions.

```
help(package = readxl)
```

From the help page, we see that `read_excel()` is the function that we want to use.

Let’s try it.

```
library(readxl)
ober <- read_excel("data/input/ober_2018.xlsx")
```

Review: what does the `/` mean? Why do we need the `data` term first? Does the argument need to be in quotes?

10.5.3 Inspecting

For almost any dataset, you usually want to do a couple of standard checks first to understand what you loaded.

```
ober
```

```
# A tibble: 1,035 x 10
```

	polis_number	Name	Latitude	Longitude	Hellenicity	Fame	Size	Colonies	Regime
	<dbl>	<chr>	<dbl>	<dbl>	<chr>	<dbl>	<chr>	<dbl>	<chr>
1	1	Alal~	42.1	9.51	most Greek	1.12	100~	0	<NA>
2	2	Empo~	42.1	3.11	most barba~	2.12	25-1~	0	<NA>

3	3 Mass~	43.3	5.38	most	Greek	4	25-1~	2	no ev~
4	4 Rhode	42.3	3.17	most	Greek	0.87	<NA>	0	<NA>
5	5 Abak~	38.1	15.1	most	barba~	1	<NA>	0	<NA>
6	6 Adra~	37.7	14.8	most	Greek	1	<NA>	0	<NA>
7	7 Agyr~	37.7	14.5	most	Greek	1.25	<NA>	0	no ev~
8	8 Aitna	38.2	15.6	most	Greek	3.25	200~	1	no ev~
9	9 Akra~	37.3	13.6	most	Greek	6.37	500 ~	0	evide~
10	10 Akrai	37.1	14.9	most	Greek	1.25	<NA>	0	<NA>

i 1,025 more rows
i 1 more variable: Delian <chr>

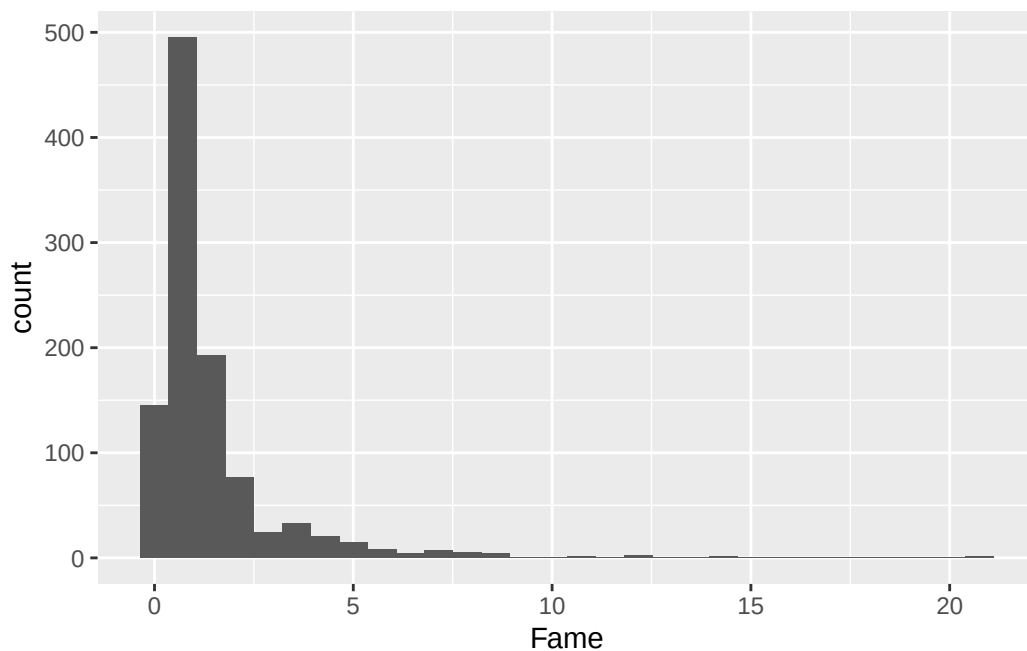
```
dim(ober)
```

```
[1] 1035  10
```

Graphics are useful for grasping your data - we will cover them more deeply later on.

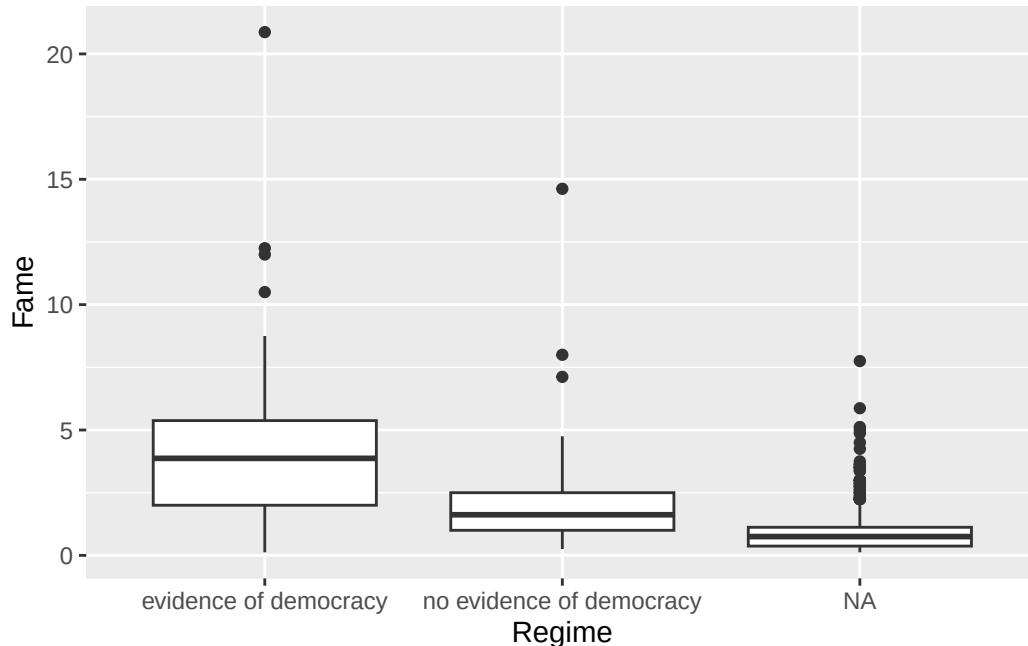
```
ggplot(ober, aes(x = Fame)) + geom_histogram()
```

`stat\_bin()` using `bins = 30`. Pick better value with `binwidth`.



What about the distribution of fame by regime?

```
ggplot(ober, aes(y = Fame, x = Regime, group = Regime)) +  
  geom_boxplot()
```



What do the 1's, 2's, and 3's stand for?

10.5.4 Finding observations

These **tidyverse** commands from the **dplyr** package are newer and not built-in, but they are one of the increasingly more popular ways to wrangle data.

- 80 percent of your data wrangling needs might be doable with these basic **dplyr** functions: **select**, **mutate**, **group_by**, **summarize**, and **arrange**.
- These verbs roughly correspond to the same commands in SQL, another important language in data science.
- The **%>%** symbol is a pipe. It takes the thing on the left side and pipes it down to the function on the right side. We could have done **count(cen10, race)** as **cen10 %>% count(race)**. That means take **cen10** and pass it on to the function **count**, which will count observations by race and return a collapsed dataset with the categories in its own variable and their respective counts in **n**.

Exercises

1

What is the Fame value of Delphoi?

```
# Enter here
```

2

Find the polis with the top 10 Fame values.

```
# Enter here
```

3

Make a scatterplot with the number of colonies on the x-axis and Fame on the y-axis.

```
# Enter here
```

4

Find the correct function to read the following datasets into your R instance.

- `data/input/acs2015_1percent.csv`: A one percent sample of the American Community Survey
- `data/input/gapminder_wide.tab`: Country-level wealth and health from Gapminder⁵
- `data/input/gapminder_wide.Rds`: A Rds version of the Gapminder (What is a Rds file? What's the difference?)
- `data/input/Nunn_Wantchekon_sample.dta`: A sample from the Afrobarometer survey (which we'll explore tomorrow). `.dta` is a Stata format.
- `data/input/german_credit.sav`: A hypothetical dataset on consumer credit. `.sav` is a SPSS format.

Our Recommendations: Look at the packages `haven` and `readr`

```
# Enter here, perhaps making a chunk for each file.
```

⁵Formatted and taken from <https://doi.org/10.7910/DVN/GJQNEQ>

5

Read Ober's codebook and find a variable that you think is interesting. Check the distribution of that variable in your data, get a couple of statistics, and summarize it in English.

```
# Enter here
```

11 Programming: Manipulating Vectors and Matrices¹

Motivation

[Nunn and Wantchekon \(2011\)](#) – “The Slave Trade and the Origins of Mistrust in Africa”² – argues that across African countries, the distrust of co-ethnics fueled by the slave trade has had long-lasting effects on modern day trust in these territories. They argued that the slave trade created distrust in these societies in part because as some African groups were employed by European traders to capture their neighbors and bring them to the slave ships.

Nunn and Wantchekon use a variety of statistical tools to make their case (adding controls, ordered logit, instrumental variables, falsification tests, causal mechanisms), many of which will be covered in future courses. In this module we will only touch on their first set of analysis that use Ordinary Least Squares (OLS). OLS is likely the most common application of linear algebra in the social sciences. We will cover some linear algebra, matrix manipulation, and vector manipulation from this data.

Where are we? Where are we headed?

Up till now, you should have covered:

- R basic programming
- Data Import
- Statistical Summaries.

Today we’ll cover

- Matrices & Dataframes in R
- Manipulating variables
- And other R tips

¹Special thanks to Shiro Kuriwaki and Yon Soo Park for developing the original module

²[Nunn, Nathan, and Leonard Wantchekon. 2011. “The Slave Trade and the Origins of Mistrust in Africa.” American Economic Review 101\(7\): 3221–52.](#)

11.1 Read Data

```
library(haven)
nunn_full <- read_dta("data/input/Nunn_Wantchekon_AER_2011.dta")
```

Nunn and Wantchekon's main dataset has more than 20,000 observations. Each observation is a respondent from the Afrobarometer survey.

```
head(nunn_full)
```

```
# A tibble: 6 x 59
  respno ethnicity murdock_name isocode region district townvill location_id
  <chr>   <chr>      <chr>      <chr> <chr>   <chr>   <chr>      <dbl>
1 BEN0001 fon      FON      BEN    atlnatiq~ KPOMASSE TOKPA-D~      30
2 BEN0002 fon      FON      BEN    atlnatiq~ KPOMASSE TOKPA-D~      30
3 BEN0003 fon      FON      BEN    atlnatiq~ OUIDAH   3ARROND      31
4 BEN0004 fon      FON      BEN    atlnatiq~ OUIDAH   3ARROND      31
5 BEN0005 fon      FON      BEN    atlnatiq~ OUIDAH   PAHOU       32
6 BEN0006 fon      FON      BEN    atlnatiq~ OUIDAH   PAHOU       32
# i 51 more variables: trust_relatives <dbl>, trust_neighbors <dbl>,
#   intra_group_trust <dbl>, inter_group_trust <dbl>,
#   trust_local_council <dbl>, ln_export_area <dbl>, export_area <dbl>,
#   export_pop <dbl>, ln_export_pop <dbl>, age <dbl>, age2 <dbl>, male <dbl>,
#   urban_dum <dbl>, occupation <dbl>, religion <dbl>, living_conditions <dbl>,
#   education <dbl>, near_dist <dbl>, distsea <dbl>, loc_murdock_name <chr>,
#   loc_ln_export_area <dbl>, local_council_performance <dbl>, ...
```

```
colnames(nunn_full)
```

```
[1] "respno"           "ethnicity"
[3] "murdock_name"     "isocode"
[5] "region"           "district"
[7] "townvill"         "location_id"
[9] "trust_relatives"  "trust_neighbors"
[11] "intra_group_trust" "inter_group_trust"
[13] "trust_local_council" "ln_export_area"
[15] "export_area"      "export_pop"
[17] "ln_export_pop"    "age"
[19] "age2"             "male"
```

[21]	"urban_dum"	"occupation"
[23]	"religion"	"living_conditions"
[25]	"education"	"near_dist"
[27]	"distsea"	"loc_murdock_name"
[29]	"loc_ln_export_area"	"local_council_performance"
[31]	"council_listen"	"corrupt_local_council"
[33]	"school_present"	"electricity_present"
[35]	"piped_water_present"	"sewage_present"
[37]	"health_clinic_present"	"district_ethnic_frac"
[39]	"frac_ethnicity_in_district"	"townvill_nonethnic_mean_exports"
[41]	"district_nonethnic_mean_exports"	"region_nonethnic_mean_exports"
[43]	"country_nonethnic_mean_exports"	"murdock_centra_dist_coast"
[45]	"centroid_lat"	"centroid_long"
[47]	"explorer_contact"	"railway_contact"
[49]	"dist_Saharan_node"	"dist_Saharan_line"
[51]	"malaria_ecology"	"v30"
[53]	"v33"	"fishing"
[55]	"exports"	"ln_exports"
[57]	"total_missions_area"	"ln_init_pop_density"
[59]	"cities_1400_dum"	

First, let's consider a small subset of this dataset.

```
nunn <- read_dta("data/input/Nunn_Wantchekon_sample.dta")
```

```
nunn
```

```
# A tibble: 10 x 5
```

	trust_neighbors	exports	ln_exports	export_area	ln_export_area
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	2	666.	6.50	14.0	2.71
2	0	0	0	0	0
3	3	0.631	0.489	0.0424	0.0416
4	3	0	0	0	0
5	3	389.	5.97	6.31	1.99
6	2	0	0	0	0
7	1	0.388	0.328	0.00407	0.00406
8	3	855.	6.75	37.7	3.66
9	0	16.4	2.85	1.22	0.796
10	3	0.435	0.361	0.0383	0.0376

11.2 data.frame vs. matrices

This is a `data.frame` object.

```
class(nunn)
```

```
[1] "tbl_df"      "tbl"        "data.frame"
```

But it can be also consider a matrix in the linear algebra sense. What are the dimensions of this matrix?

```
nrow(nunn)
```

```
[1] 10
```

`data.frames` and matrices have much overlap in R, but to explicitly treat an object as a matrix, you'd need to coerce its class. Let's call this matrix `X`.

```
X <- as.matrix(nunn)
```

What is the difference between a `data.frame` and a matrix? A `data.frame` can have columns that are of different types, whereas — in a matrix — all columns must be of the same type (usually either “numeric” or “character”).

You can think of data frames maybe as matrices-plus, because a column can take on characters as well as numbers. As we just saw, this is often useful for real data analyses.

Another way to think about data frames is that it is a type of list. Try the `str()` code below and notice how it is organized in slots. Each slot is a vector. They can be vectors of numbers or characters.

```
# enter this on your console
str(cen10)
```

11.3 Handling matrices in R

You can easily transpose a matrix

```
X
```

	trust_neighbors	exports	ln_exports	export_area	ln_export_area
[1,]	2	665.9652100	6.5027380	13.975566864	2.706419945
[2,]	0	0.0000000	0.0000000	0.000000000	0.000000000
[3,]	3	0.6306777	0.4889957	0.042445682	0.041569568
[4,]	3	0.0000000	0.0000000	0.000000000	0.000000000
[5,]	3	388.6097107	5.9651456	6.306191921	1.988722205
[6,]	2	0.0000000	0.0000000	0.000000000	0.000000000
[7,]	1	0.3883497	0.3281158	0.004067405	0.004059155
[8,]	3	854.9580688	6.7522216	37.707386017	3.656030416
[9,]	0	16.3584309	2.8540783	1.216732502	0.796034217
[10,]	3	0.4349871	0.3611559	0.038332380	0.037615947

```
t(X)
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]
trust_neighbors	2.000000	0	3.00000000	3	3.000000	2	1.000000000
exports	665.965210	0	0.63067770	0	388.609711	0	0.388349682
ln_exports	6.502738	0	0.48899570	0	5.965146	0	0.328115761
export_area	13.975567	0	0.04244568	0	6.306192	0	0.004067405
ln_export_area	2.706420	0	0.04156957	0	1.988722	0	0.004059155
	[,8]	[,9]	[,10]				
trust_neighbors	3.000000	0.0000000	3.00000000				
exports	854.958069	16.3584309	0.43498713				
ln_exports	6.752222	2.8540783	0.36115587				
export_area	37.707386	1.2167325	0.03833238				
ln_export_area	3.656030	0.7960342	0.03761595				

What are the values of all rows in the first column?

```
X[, 1]
```

```
[1] 2 0 3 3 3 2 1 3 0 3
```

What are all the values of “exports”? (i.e. return the whole “exports” column)

```
X[, "exports"]
```

```
[1] 665.9652100  0.0000000  0.6306777  0.0000000 388.6097107  0.0000000  
[7]  0.3883497 854.9580688 16.3584309  0.4349871
```

What is the first observation (i.e. first row)?

```
X[1, ]
```

trust_neighbors	exports	ln_exports	export_area	ln_export_area
2.000000	665.965210	6.502738	13.975567	2.706420

What is the value of the first variable of the first observation?

```
X[1, 1]
```

```
trust_neighbors  
2
```

Pause and consider the following problem on your own. What is the following code doing?

```
X[X[, "trust_neighbors"] == 0, "export_area"]
```

```
[1] 0.000000 1.216733
```

Why does it give the same output as the following?

```
X[which(X[, "trust_neighbors"] == 0), "export_area"]
```

```
[1] 0.000000 1.216733
```

Some more manipulation

```
X + X
```

	trust_neighbors	exports	ln_exports	export_area	ln_export_area
[1,]	4	1331.9304199	13.0054760	27.951133728	5.41283989
[2,]	0	0.0000000	0.0000000	0.000000000	0.00000000
[3,]	6	1.2613554	0.9779914	0.084891364	0.08313914
[4,]	6	0.0000000	0.0000000	0.000000000	0.00000000
[5,]	6	777.2194214	11.9302912	12.612383842	3.97744441
[6,]	4	0.0000000	0.0000000	0.000000000	0.00000000
[7,]	2	0.7766994	0.6562315	0.008134809	0.00811831
[8,]	6	1709.9161377	13.5044432	75.414772034	7.31206083
[9,]	0	32.7168617	5.7081566	2.433465004	1.59206843
[10,]	6	0.8699743	0.7223117	0.076664761	0.07523189

X - X

	trust_neighbors	exports	ln_exports	export_area	ln_export_area
[1,]	0	0	0	0	0
[2,]	0	0	0	0	0
[3,]	0	0	0	0	0
[4,]	0	0	0	0	0
[5,]	0	0	0	0	0
[6,]	0	0	0	0	0
[7,]	0	0	0	0	0
[8,]	0	0	0	0	0
[9,]	0	0	0	0	0
[10,]	0	0	0	0	0

t(X) %*% X

	trust_neighbors	exports	ln_exports	export_area	ln_export_area
trust_neighbors	54.00000	5066.219	54.03615	160.2383	
exports	5066.21910	1325748.804	12468.85827	44016.0715	
ln_exports	54.03615	12468.858	132.08403	386.6140	
export_area	160.23827	44016.072	386.61401	1658.4152	
ln_export_area	22.58871	5714.037	56.45567	189.1962	
ln_export_area					
trust_neighbors	22.58871				
exports	5714.03702				
ln_exports	56.45567				
export_area	189.19615				
ln_export_area	25.28311				

```
cbind(X, 1:10)
```

	trust_neighbors	exports	ln_exports	export_area	ln_export_area	
[1,]	2	665.9652100	6.5027380	13.975566864	2.706419945	1
[2,]	0	0.0000000	0.0000000	0.000000000	0.000000000	2
[3,]	3	0.6306777	0.4889957	0.042445682	0.041569568	3
[4,]	3	0.0000000	0.0000000	0.000000000	0.000000000	4
[5,]	3	388.6097107	5.9651456	6.306191921	1.988722205	5
[6,]	2	0.0000000	0.0000000	0.000000000	0.000000000	6
[7,]	1	0.3883497	0.3281158	0.004067405	0.004059155	7
[8,]	3	854.9580688	6.7522216	37.707386017	3.656030416	8
[9,]	0	16.3584309	2.8540783	1.216732502	0.796034217	9
[10,]	3	0.4349871	0.3611559	0.038332380	0.037615947	10

```
cbind(X, 1)
```

	trust_neighbors	exports	ln_exports	export_area	ln_export_area	
[1,]	2	665.9652100	6.5027380	13.975566864	2.706419945	1
[2,]	0	0.0000000	0.0000000	0.000000000	0.000000000	1
[3,]	3	0.6306777	0.4889957	0.042445682	0.041569568	1
[4,]	3	0.0000000	0.0000000	0.000000000	0.000000000	1
[5,]	3	388.6097107	5.9651456	6.306191921	1.988722205	1
[6,]	2	0.0000000	0.0000000	0.000000000	0.000000000	1
[7,]	1	0.3883497	0.3281158	0.004067405	0.004059155	1
[8,]	3	854.9580688	6.7522216	37.707386017	3.656030416	1
[9,]	0	16.3584309	2.8540783	1.216732502	0.796034217	1
[10,]	3	0.4349871	0.3611559	0.038332380	0.037615947	1

```
colnames(X)
```

```
[1] "trust_neighbors" "exports"          "ln_exports"      "export_area"
[5] "ln_export_area"
```

11.4 Variable Transformations

`exports` is the total number of slaves that were taken from the individual's ethnic group between Africa's four slave trades between 1400-1900.

What is `ln_exports`? The article describes this as the natural log of one plus the `exports`. This is a transformation of one column by a particular function

```
log(1 + X[, "exports"])
```

```
[1] 6.5027379 0.0000000 0.4889957 0.0000000 5.9651455 0.0000000 0.3281158  
[8] 6.7522214 2.8540783 0.3611559
```

Question for you: why add the 1?

Verify that this is the same as `X[, "ln_exports"]`

11.5 Linear Combinations

In Table 1 we see “OLS Estimates”. These are estimates of OLS coefficients and standard errors. You do not need to know what these are for now, but it doesn’t hurt to getting used to seeing them.

TABLE 1—OLS ESTIMATES OF THE DETERMINANTS OF TRUST IN NEIGHBORS

Dependent variable: Trust of neighbors	Slave exports (thousands) (1)	Exports/ area (2)	Exports/ historical pop (3)	ln (1 + exports) (4)	ln (1 + exports/ area) (5)	ln (1 + exports/ historical pop) (6)
Estimated coefficient	−0.00068 [0.00014] (0.00015) {0.00013}	−0.019 [0.005] (0.005) {0.005}	−0.531 [0.147] (0.147) {0.165}	−0.037 [0.014] (0.014) {0.015}	−0.159 [0.034] (0.034) {0.034}	−0.743 [0.187] (0.187) {0.212}
Individual controls	Yes	Yes	Yes	Yes	Yes	Yes
District controls	Yes	Yes	Yes	Yes	Yes	Yes
Country fixed effects	Yes	Yes	Yes	Yes	Yes	Yes
Number of observations	20,027	20,027	17,644	20,027	20,027	17,644
Number of ethnicities	185	185	157	185	185	157
Number of districts	1,257	1,257	1,214	1,257	1,257	1,214
R ²	0.16	0.16	0.15	0.15	0.16	0.15

Notes: The table reports OLS estimates. The unit of observation is an individual. Below each coefficient three standard errors are reported. The first, reported in square brackets, is standard errors adjusted for clustering within ethnic groups. The second, reported in parentheses, is standard errors adjusted for two-way clustering within ethnic groups and within districts. The third, reported in curly brackets, is T. G. Conley (1999) standard errors adjusted for two-dimensional spatial autocorrelation. The standard errors are constructed assuming a window with weights equal to one for observations less than five degrees apart and zero for observations further apart. The individual controls are for age, age squared, a gender indicator variable, five living conditions fixed effects, ten education fixed effects, 18 religion fixed effects, 25 occupation fixed effects, and an indicator for whether the respondent lives in an urban location. The district controls include ethnic fractionalization of each district and the share of the district's population that is the same ethnicity as the respondent.

A very crude way to describe regression is through linear combinations. The simplest linear combination is a one-to-one transformation.

Take the first number in Table 1, which is -0.00068. Now, multiply this by `exports`

```
-0.00068 * X[, "exports"]
```

```
[1] -0.4528563428  0.0000000000 -0.0004288608  0.0000000000 -0.2642546033
[6]  0.0000000000 -0.0002640778 -0.5813714868 -0.0111237330 -0.0002957912
```

Now, just one more step. Make a new matrix with just `exports` and the value 1

```
x2 <- cbind(1, X[, "exports"])
```

name this new column “intercept”

```
colnames(X2)
```

NULL

```
colnames(X2) <- c("intercept", "exports")
```

What are the dimensions of the matrix X2?

```
dim(X2)
```

```
[1] 10  2
```

Now consider a new matrix, called B.

```
B <- matrix(c(1.62, -0.00068))
```

What are the dimensions of B?

```
dim(B)
```

```
[1] 2 1
```

What is the product of X2 and B? From the dimensions, can you tell if it will be conformable?

```
X2 %*% B
```

```
      [,1]  
[1,] 1.167144  
[2,] 1.620000  
[3,] 1.619571  
[4,] 1.620000  
[5,] 1.355745  
[6,] 1.620000  
[7,] 1.619736  
[8,] 1.038629  
[9,] 1.608876  
[10,] 1.619704
```

What is this multiplication doing in terms of equations?

11.6 Matrix Basics

Let's take a look at Matrices in the context of R

```
cen10 <- read_csv("data/input/usc2010_001percent.csv")
head(cen10)
```

```
# A tibble: 6 x 4
  state      sex    age race
  <chr>    <chr> <dbl> <chr>
1 New York Female     8 White
2 Ohio      Male    24 White
3 Nevada     Male    37 White
4 Michigan  Female    12 White
5 Maryland  Female    18 Black/Negro
6 New Hampshire Male    50 White
```

What is the dimension of this dataframe? What does the number of rows represent? What does the number of columns represent?

```
dim(cen10)
```

```
[1] 30871      4
```

```
nrow(cen10)
```

```
[1] 30871
```

```
ncol(cen10)
```

```
[1] 4
```

What variables does this dataset hold? What kind of information does it have?

```
colnames(cen10)
```

```
[1] "state" "sex"   "age"   "race"
```

We can access column vectors, or vectors that contain values of variables by using the \$ sign

```
head(cen10$state)
```

```
[1] "New York"      "Ohio"          "Nevada"        "Michigan"
[5] "Maryland"      "New Hampshire"
```

```
head(cen10$race)
```

```
[1] "White"      "White"      "White"      "White"      "Black/Negro"
[6] "White"
```

We can look at a unique set of variable values by calling the unique function

```
unique(cen10$state)
```

```
[1] "New York"      "Ohio"          "Nevada"
[4] "Michigan"      "Maryland"      "New Hampshire"
[7] "Iowa"          "Missouri"      "New Jersey"
[10] "California"    "Texas"         "Pennsylvania"
[13] "Washington"    "West Virginia" "Idaho"
[16] "North Carolina" "Massachusetts" "Connecticut"
[19] "Arkansas"      "Indiana"       "Wisconsin"
[22] "Maine"         "Tennessee"    "Minnesota"
[25] "Florida"       "Oklahoma"     "Montana"
[28] "Georgia"       "Arizona"      "Colorado"
[31] "Virginia"      "Illinois"     "Oregon"
[34] "Kentucky"      "South Carolina" "Kansas"
[37] "Louisiana"     "Alabama"      "District of Columbia"
[40] "Mississippi"   "Utah"         "Delaware"
[43] "Nebraska"      "Alaska"       "New Mexico"
[46] "South Dakota"  "Hawaii"       "Vermont"
[49] "Rhode Island"  "Wyoming"      "North Dakota"
```

How many different states are represented (this dataset includes DC as a state)?

```
length(unique(cen10$state))
```

```
[1] 51
```

Matrices are rectangular structures of numbers (they have to be numbers, and they can't be characters).

A cross-tab can be considered a matrix:

```
table(cen10$race, cen10$sex)
```

	Female	Male
American Indian or Alaska Native	142	153
Black/Negro	2070	1943
Chinese	192	162
Japanese	51	26
Other Asian or Pacific Islander	587	542
Other race, nec	877	962
Three or more major races	37	51
Two major races	443	426
White	11252	10955

```
cross_tab <- table(cen10$race, cen10$sex)  
dim(cross_tab)
```

```
[1] 9 2
```

```
cross_tab[6, 2]
```

```
[1] 962
```

But a subset of your data – individual values– can be considered a matrix too.

```
# First 20 rows of the entire data  
# Below two lines of code do the same thing  
cen10[1:20, ]
```

```
# A tibble: 20 x 4
```

	state <chr>	sex <chr>	age <dbl>	race <chr>
1	New York	Female	8	White
2	Ohio	Male	24	White
3	Nevada	Male	37	White
4	Michigan	Female	12	White
5	Maryland	Female	18	Black/Negro
6	New Hampshire	Male	50	White
7	Iowa	Female	51	White
8	Missouri	Female	41	White
9	New Jersey	Male	62	White
10	California	Male	25	White
11	Texas	Female	23	White
12	Pennsylvania	Female	66	White
13	California	Female	57	White
14	Texas	Female	73	Other race, nec
15	California	Male	43	White
16	Washington	Male	29	White
17	Texas	Male	8	White
18	Missouri	Male	78	White
19	West Virginia	Male	10	White
20	Idaho	Female	9	White

```
cen10 %>% slice(1:20)
```

```
# A tibble: 20 x 4
```

	state <chr>	sex <chr>	age <dbl>	race <chr>
1	New York	Female	8	White
2	Ohio	Male	24	White
3	Nevada	Male	37	White
4	Michigan	Female	12	White
5	Maryland	Female	18	Black/Negro
6	New Hampshire	Male	50	White
7	Iowa	Female	51	White
8	Missouri	Female	41	White
9	New Jersey	Male	62	White
10	California	Male	25	White
11	Texas	Female	23	White
12	Pennsylvania	Female	66	White

13	California	Female	57	White
14	Texas	Female	73	Other race, nec
15	California	Male	43	White
16	Washington	Male	29	White
17	Texas	Male	8	White
18	Missouri	Male	78	White
19	West Virginia	Male	10	White
20	Idaho	Female	9	White

```
# Of the first 20 rows of the entire data, look at values of just race and age
# Below two lines of code do the same thing
cen10[1:20, c("race", "age")]
```

```
# A tibble: 20 x 2
  race      age
  <chr>    <dbl>
1 White      8
2 White     24
3 White     37
4 White     12
5 Black/Negro 18
6 White     50
7 White     51
8 White     41
9 White     62
10 White     25
11 White     23
12 White     66
13 White     57
14 Other race, nec 73
15 White     43
16 White     29
17 White      8
18 White     78
19 White     10
20 White      9
```

```
cen10 %>% slice(1:20) %>% select(race, age)
```

```
# A tibble: 20 x 2
  race      age
  <chr>    <dbl>
1 White      8
2 White     24
3 White     37
4 White     12
5 Black/Negro 18
6 White     50
7 White     51
8 White     41
9 White     62
10 White     25
11 White     23
12 White     66
13 White     57
14 Other race, nec 73
15 White     43
16 White     29
17 White      8
18 White     78
19 White     10
20 White      9
```

A vector is a special type of matrix with only one column or only one row

```
# One column
cen10[1:10, c("age")]
```

```
# A tibble: 10 x 1
  age
  <dbl>
1     8
2    24
3    37
4    12
5    18
6    50
7    51
8    41
9    62
10   25
```

```
cen10 %>% slice(1:10) %>% select(c("age"))
```

```
# A tibble: 10 x 1
```

```
  age
<dbl>
1     8
2    24
3    37
4    12
5    18
6    50
7    51
8    41
9    62
10   25
```

```
# One row
cen10[2, ]
```

```
# A tibble: 1 x 4
```

```
  state sex    age race
<chr> <chr> <dbl> <chr>
1 Ohio  Male    24  White
```

```
cen10 %>% slice(2)
```

```
# A tibble: 1 x 4
```

```
  state sex    age race
<chr> <chr> <dbl> <chr>
1 Ohio  Male    24  White
```

What if we want a special subset of the data? For example, what if I only want the records of individuals in California? What if I just want the age and race of individuals in California?

```
# subset for CA rows
ca_subset <- cen10[cen10$state == "California", ]
```

```
ca_subset_tidy <- cen10 %>% filter(state == "California")

all_equal(ca_subset, ca_subset_tidy)
```

Warning: `all\_equal()` was deprecated in dplyr 1.1.0.
i Please use `all.equal()` instead.
i And manually order the rows/cols as needed

```
[1] TRUE
```

```
# subset for CA rows and select age and race
ca_subset_age_race <- cen10[cen10$state == "California", c("age", "race")]

ca_subset_age_race_tidy <- cen10 %>% filter(state == "California") %>% select(age, race)

all_equal(ca_subset_age_race, ca_subset_age_race_tidy)
```

```
[1] TRUE
```

Some common operators that can be used to filter or to use as a condition. Remember, you can use the unique function to look at the set of all values a variable holds in the dataset.

```
# all individuals older than 30 and younger than 70
s1 <- cen10[cen10$age > 30 & cen10$age < 70, ]
s2 <- cen10 %>% filter(age > 30 & age < 70)
all_equal(s1, s2)
```

```
[1] TRUE
```

```
# all individuals in either New York or California
s3 <- cen10[cen10$state == "New York" | cen10$state == "California", ]
s4 <- cen10 %>% filter(state == "New York" | state == "California")
all_equal(s3, s4)
```

```
[1] TRUE
```

```
# all individuals in any of the following states: California, Ohio, Nevada, Michigan
s5 <- cen10[cen10$state %in% c("California", "Ohio", "Nevada", "Michigan"), ]
s6 <- cen10 %>% filter(state %in% c("California", "Ohio", "Nevada", "Michigan"))
all_equal(s5, s6)
```

[1] TRUE

```
# all individuals NOT in any of the following states: California, Ohio, Nevada, Michigan
s7 <- cen10[!(cen10$state %in% c("California", "Ohio", "Nevada", "Michigan")), ]
s8 <- cen10 %>% filter(!state %in% c("California", "Ohio", "Nevada", "Michigan"))
all_equal(s7, s8)
```

[1] TRUE

Checkpoint

1

Get the subset of cen10 for non-white individuals (Hint: look at the set of values for the race variable by using the unique function)

```
# Enter here
```

2

Get the subset of cen10 for females over the age of 40

```
# Enter here
```

3

Get all the serial numbers for black, male individuals who don't live in Ohio or Nevada.

```
# Enter here
```

Exercises

1

Let

$$\mathbf{A} = \begin{bmatrix} 0.6 & 0.2 \\ 0.4 & 0.8 \end{bmatrix}$$

Use R to write code that will create the matrix A , and then consecutively multiply A to itself 4 times. What is the value of A^4 ?

```
## Enter yourself
```

Note that R notation of matrices is different from the math notation. Simply trying $\mathbf{X}^n$ where $\mathbf{X}$ is a matrix will only take the power of each element to n . Instead, this problem asks you to perform matrix multiplication.

2

Let's apply what we learned about subsetting or filtering/selecting. Use the `nunn_full` dataset you have already loaded

- a) First, show all observations (rows) that have a "male" variable higher than 0.5

```
## Enter yourself
```

- b) Next, create a matrix / dataframe with only two columns: "trust_neighbors" and "age"

```
## Enter yourself
```

- c) Lastly, show all values of "trust_neighbors" and "age" for observations (rows) that have the "male" variable value that is higher than 0.5

```
## Enter yourself
```

3

Find a way to generate a vector of “column averages” of the matrix **X** from the Nunn and Wantchekon data in one line of code. Each entry in the vector should contain the sample average of the values in the column. So a 100 by 4 matrix should generate a length-4 matrix.

4

Similarly, generate a vector of “column medians”.

5

Consider the regression that was run to generate Table 1:

```
form <- "trust_neighbors ~ exports + age + age2 + male + urban_dum + factor(education) +  
lm_1_1 <- lm(as.formula(form), nunn_full)  
  
# The below coef function returns a vector of OLS coefficients  
coef(lm_1_1)
```

(Intercept)	exports
1.619913e+00	-6.791360e-04
age	age2
8.395936e-03	-5.473436e-05
male	urban_dum
4.550246e-02	-1.404551e-01
factor(education)1	factor(education)2
1.709816e-02	-5.224591e-02
factor(education)3	factor(education)4
-1.373770e-01	-1.889619e-01
factor(education)5	factor(education)6
-1.893494e-01	-2.400767e-01
factor(education)7	factor(education)8
-2.850748e-01	-1.232085e-01
factor(education)9	factor(occupation)1
-2.406437e-01	6.185655e-02
factor(occupation)2	factor(occupation)3
7.392168e-02	3.356158e-02
factor(occupation)4	factor(occupation)5
7.942048e-03	6.661126e-02

factor(occupation)6	factor(occupation)7
-7.563297e-02	1.699699e-02
factor(occupation)8	factor(occupation)9
-9.428177e-02	-9.981440e-02
factor(occupation)10	factor(occupation)11
-3.307068e-02	-2.300045e-02
factor(occupation)12	factor(occupation)13
-1.564540e-01	-1.441370e-02
factor(occupation)14	factor(occupation)15
-5.566414e-02	-2.343762e-01
factor(occupation)16	factor(occupation)18
-1.306947e-02	-1.729589e-01
factor(occupation)19	factor(occupation)20
-1.770261e-01	-2.457800e-02
factor(occupation)21	factor(occupation)22
-4.936813e-02	-1.068511e-01
factor(occupation)23	factor(occupation)24
-9.712205e-02	1.292371e-02
factor(occupation)25	factor(occupation)995
2.623186e-02	-1.195063e-03
factor(religion)2	factor(religion)3
5.395953e-02	7.887878e-02
factor(religion)4	factor(religion)5
4.749150e-02	4.318455e-02
factor(religion)6	factor(religion)7
-1.787694e-02	-3.616542e-02
factor(religion)10	factor(religion)11
6.015041e-02	2.237845e-01
factor(religion)12	factor(religion)13
2.627086e-01	-6.812813e-02
factor(religion)14	factor(religion)15
4.673681e-02	3.844555e-01
factor(religion)360	factor(religion)361
3.656843e-01	3.416413e-01
factor(religion)362	factor(religion)363
8.230393e-01	3.856565e-01
factor(religion)995	factor(living_conditions)2
4.161301e-02	4.395862e-02
factor(living_conditions)3	factor(living_conditions)4
8.627372e-02	1.197428e-01
factor(living_conditions)5	district_ethnic_frac
1.203606e-01	-1.553648e-02
frac_ethnicity_in_district	isocodeBWA

1.011222e-01	-4.258953e-01
isocodeGHA	isocodeKEN
1.135307e-02	-1.819556e-01
isocodeLSO	isocodeMDG
-5.511200e-01	-3.315727e-01
isocodeMLI	isocodeMOZ
7.528101e-02	8.223730e-02
isocodeMWI	isocodeNAM
3.062497e-01	-1.397541e-01
isocodeNGA	isocodeSEN
-2.381525e-01	3.867371e-01
isocodeTZA	isocodeUGA
2.079366e-01	-6.443732e-02
isocodeZAF	isocodeZMB
-2.179153e-01	-2.172868e-01

First, get a small subset of the `nunn_full` dataset. This time, sample 20 rows and select for variables `exports`, `age`, `age2`, `male`, and `urban_dum`. To this small subset, add (`bind_cols()` in tidyverse or `cbind()` in base R) a column of 1's; this represents the intercept. If you need some guidance, look at how we sampled 10 rows selected for a different set of variables above in the lecture portion.

```
# Enter here
```

Next let's try calculating predicted values of levels of trust in neighbors by multiplying coefficients for the intercept, `exports`, `age`, `age2`, `male`, and `urban_dum` to the actual observed values for those variables in the small subset you've just created.

```
# Hint: You can get just selected elements from the vector returned by coef(lm_1_1)

# For example, the below code gives you the first 3 elements of the original vector
coef(lm_1_1)[1:3]
```

```
(Intercept)      exports      age
1.619913146 -0.000679136  0.008395936
```

```
# Also, the below code gives you the coefficient elements for intercept and male
coef(lm_1_1)[c("(Intercept)", "male")]
```

```
(Intercept)      male
1.61991315  0.04550246
```

12 Objects, Functions, Loops

Where are we? Where are we headed?

Up till now, you should have covered:

- R basic programming
- Data Import
- Statistical Summaries
- Visualization

Today we'll cover

- Objects
- Functions
- Loops

12.1 What is an object?

Now that we have covered some hands-on ways to use graphics, let's go into some fundamentals of the R language.

Let's first set up

```
library(dplyr)
library(readr)
library(haven)
library(ggplot2)
```

```
cen10 <- read_csv("data/input/usc2010_001percent.csv", col_types = cols())
```

Objects are abstract symbols in which you store data. Here we will create an object from `copy`, and assign `cen10` to it.

```
copy <- cen10
```

This looks the same as the original dataset:

```
copy
```

```
# A tibble: 30,871 x 4
  state      sex    age race
  <chr>    <chr> <dbl> <chr>
1 New York Female     8 White
2 Ohio     Male    24 White
3 Nevada   Male    37 White
4 Michigan Female   12 White
5 Maryland Female   18 Black/Negro
6 New Hampshire Male    50 White
7 Iowa     Female   51 White
8 Missouri Female   41 White
9 New Jersey Male    62 White
10 California Male    25 White
# i 30,861 more rows
```

What happens if you do this next?

```
copy <- ""
```

It got reassigned:

```
copy
```

```
[1] ""
```

12.1.1 Lists

Lists are one of the most generic and flexible type of object. You can make an empty list by the function `list()`

```
my_list <- list()
my_list
```

```
list()
```

And start filling it in. Slots on the list are invoked by double square brackets `[[ ]]`

```
my_list[[1]] <- "contents of the first slot -- this is a string"
my_list[["slot 2"]] <- "contents of slot named slot 2"
my_list
```

```
[[1]]
[1] "contents of the first slot -- this is a string"

$`slot 2`
[1] "contents of slot named slot 2"
```

each slot can be anything. What are we doing here? We are defining the 1st slot of the list `my_list` to be a vector `c(1, 2, 3, 4, 5)`

```
my_list[[1]] <- c(1, 2, 3, 4, 5)
my_list
```

```
[[1]]
[1] 1 2 3 4 5

$`slot 2`
[1] "contents of slot named slot 2"
```

You can even make nested lists. Let's say we want the 1st slot of the list to be another list of three elements.

```
my_list[[1]][[1]] <- "subitem 1 in slot 1 of my_list"
my_list[[1]][[2]] <- "subitem 1 in slot 2 of my_list"
my_list[[1]][[3]] <- "subitem 1 in slot 3 of my_list"

my_list
```

```
[[1]]
[1] "subitem 1 in slot 1 of my_list" "subitem 1 in slot 2 of my_list"
[3] "subitem 1 in slot 3 of my_list" "4"
[5] "5"

$`slot 2`
[1] "contents of slot named slot 2"
```

12.2 Making your own objects

We've covered one type of object, which is a list. You saw it was quite flexible. How many types of objects are there?

There are an infinite number of objects, because people make their own class of object. You can detect the type of the object (the class) by the function `class`

Object can be said to be an instance of a class.

Analogies:

class - Pokemon, **object** - Pikachu

class - Book, **object** - To Kill a Mockingbird

class - DataFrame, **object** - 2010 census data

class - Character, **object** - "Programming is Fun"

What is type (class) of object is `cen10`?

```
class(cen10)
```

```
[1] "spec_tbl_df" "tbl_df"      "tbl"        "data.frame"
```

What about this text?

```
class("some random text")
```

```
[1] "character"
```

To change or create the class of any object, you can *assign* it. To do this, assign the name of your class to character to an object's `class()`.

We can start from a simple list. For example, say we wanted to store data about pokemon. Because there is no pre-made package for this, we decide to make our own class.

```
pikachu <- list(name = "Pikachu",
               number = 25,
               type = "Electric",
               color = "Yellow")
```

and we can give it any class name we want.

```
class(pikachu) <- "Pokemon"
str(pikachu)
```

List of 4

```
$ name : chr "Pikachu"
$ number: num 25
$ type : chr "Electric"
$ color : chr "Yellow"
- attr(*, "class")= chr "Pokemon"
```

```
pikachu$type
```

```
[1] "Electric"
```

We can even define **class-specific** methods. For example, the `summary()` function is commonly used to summarize the output of a particular object (such as an `lm()` regression object). However, `summary()` will behave differently depending on what object you give it. Why? Because there is a version of `summary` defined specifically for `lm()` objects. Let's define a `summary()` method for the `Pokemon` class

```
# Input: an object of class "Pokemon"
# Output: A text summary of the Pokemon
summary.Pokemon <- function(x){
  out_text <- paste(x$name, " is a(n) ", x$type,
    " type Pokemon. Its PokeDex number is ",
    x$number, ". It is ", x$color, ".\n", sep="")
  cat(out_text)
}
```

Now let's call the generic `summary()` function on `pikachu`

```
summary(pikachu)
```

Pikachu is a(n) Electric type Pokemon. Its PokeDex number is 25. It is Yellow.

12.2.1 Seeing R through objects

Most of the R objects that you will see as you advance are their own objects. For example, here's a linear regression object

```
ols <- lm(mpg ~ wt + vs + gear + carb, mtcars)
class(ols)
```

```
[1] "lm"
```

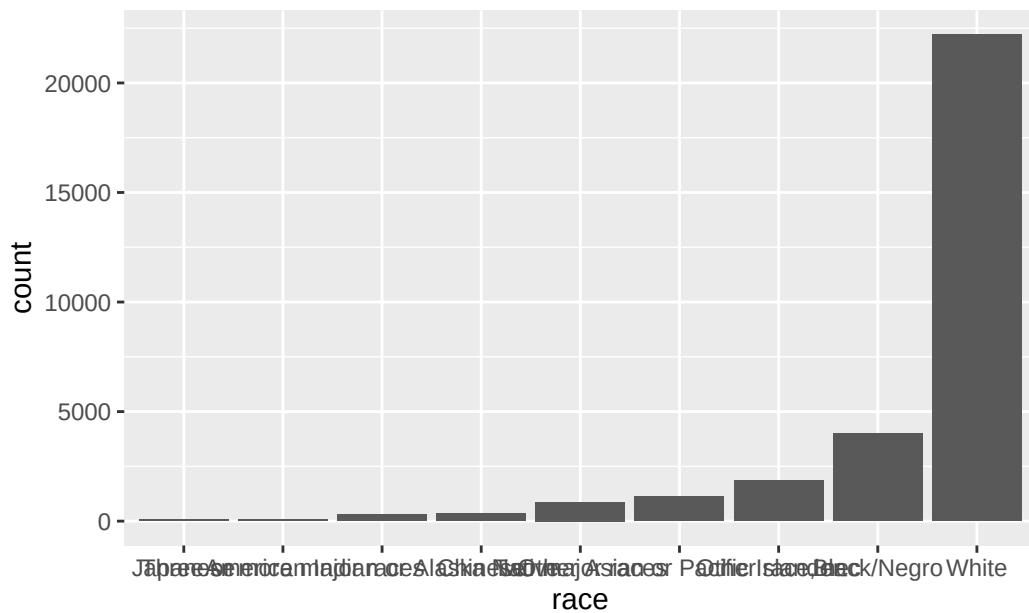
Anything can be an object! Even graphs (in `ggplot`) can be assigned, re-assigned, and edited.

```
grp_race <- group_by(cen10, race)%>%
  summarize(count = n())

grp_race_ordered <- arrange(grp_race, count) %>%
  mutate(race = forcats::as_factor(race))

gg_tab <- ggplot(data = grp_race_ordered) +
  aes(x = race, y = count) +
  geom_col() +
  labs(caption = "Source: U.S. Census 2010")

gg_tab
```



Source: U.S. Census 2010

You can change the orientation

```
gg_tab<- gg_tab + coord_flip()
```

12.2.2 Parsing an object by `str()`s

It can be hard to understand an R object because it's contents are unknown. The function `str`, short for structure, is a quick way to look into the innards of an object

```
str(my_list)
```

List of 2

```
$      : chr [1:5] "subitem 1 in slot 1 of my_list" "subitem 1 in slot 2 of my_list" "subi
$ slot 2: chr "contents of slot named slot 2"
```

```
class(my_list)
```

```
[1] "list"
```

Same for the object we just made

```
str(pikachu)
```

List of 4

```
$ name : chr "Pikachu"  
$ number: num 25  
$ type : chr "Electric"  
$ color : chr "Yellow"  
- attr(*, "class")= chr "Pokemon"
```

What does a `ggplot` object look like? Very complicated, but at least you can see it:

```
# enter this on your console  
str(gg_tab)
```

12.3 Types of variables

In the social science we often analyze variables. As you saw in the tutorial, different types of variables require different care.

A key link with what we just learned is that variables are also types of R objects.

12.3.1 scalars

One number. How many people did we count in our Census sample?

```
nrow(cen10)
```

```
[1] 30871
```

Question: What proportion of our census sample is Native American? This number is also a scalar

```
# Enter yourself  
unique(cen10$race)
```

```
[1] "White" "Black/Negro"
[3] "Other race, nec" "American Indian or Alaska Native"
[5] "Chinese" "Other Asian or Pacific Islander"
[7] "Two major races" "Three or more major races"
[9] "Japanese"
```

```
mean(cen10$race == "American Indian or Alaska Native")
```

```
[1] 0.009555894
```

Hint: you can use the function `mean()` to calculate the sample mean. The sample proportion is the mean of a sequence of number, where your event of interest is a 1 (or TRUE) and others are 0 (or FALSE).

12.3.2 numeric vectors

A sequence of numbers.

```
grp_race_ordered$count
```

```
[1] 77 88 295 354 869 1129 1839 4013 22207
```

```
class(grp_race_ordered$count)
```

```
[1] "integer"
```

Or even, all the ages of the millions of people in our Census. Here are just the first few numbers of the list.

```
head(cen10$age)
```

```
[1] 8 24 37 12 18 50
```

12.3.3 characters (aka strings)

This can be just one stretch of characters

```
my_name <- "Anton"
my_name
```

```
[1] "Anton"
```

```
class(my_name)
```

```
[1] "character"
```

or more characters. Notice here that there's a difference between a vector of individual characters and a length-one object of characters.

```
my_name_letters <- c("A","n","t","o","n")
my_name_letters
```

```
[1] "A" "n" "t" "o" "n"
```

```
class(my_name_letters)
```

```
[1] "character"
```

Finally, remember that lower vs. upper case matters in R!

```
my_name2 <- "anton"
my_name == my_name2
```

```
[1] FALSE
```

12.4 What is a function?

Most of what we do in R is executing a function. `read_csv()`, `nrow()`, `ggplot()` .. pretty much anything with a parentheses is a function. And even things like `<-` and `[` are functions as well.

A function is a set of instructions with specified ingredients. It takes an **input**, then **manipulates** it – changes it in some way – and then returns the manipulated product.

One way to see what a function actually does is to enter it without parentheses.

```
# enter this on your console
table
```

You'll see below that the most basic functions are quite complicated internally.

You'll notice that functions contain other functions. *wrapper* functions are functions that “wrap around” existing functions. This sounds redundant, but it's an important feature of programming. If you find yourself repeating a command more than two times, you should make your own function, rather than writing the same type of code.

12.4.1 Write your own function

It's worth remembering the basic structure of a function. You create a new function, call it `my_fun` by this:

```
my_fun <- function() {  
  
}
```

If we wanted to generate a function that computed the number of men in your data, what would that look like?

```
count_men <- function(data) {  
  
  nmen <- sum(data$sex == "Male")  
  
  return(nmen)  
}
```

Then all we need to do is feed this function a dataset

```
count_men(cen10)
```

```
[1] 15220
```

The point of a function is that you can use it again and again without typing up the set of constituent manipulations. So, what if we wanted to figure out the number of men in California?

```
count_men(cen10[cen10$state == "California",])
```

```
[1] 1876
```

Let's go one step further. What if we want to know the proportion of non-whites in a state, just by entering the name of the state? There's multiple ways to do it, but it could look something like this

```
nw_in_state <- function(data, state) {  
  
  s.subset <- data[data$state == state,]  
  total.s <- nrow(s.subset)  
  nw.s <- sum(s.subset$race != "White")  
  
  nw.s / total.s  
}
```

The last line is what gets generated from the function. To be more explicit you can wrap the last line around `return()`. (as in `return(nw.s/total.s)`. `return()` is used when you want to break out of a function in the middle of it and not wait till the last line.

Try it on your favorite state!

```
nw_in_state(cen10, "Massachusetts")
```

```
[1] 0.2040185
```

Checkpoint

1

Try making your own function, `average_age_in_state`, that will give you the average age of people in a given state.

```
# Enter on your own
```

2

Try making your own function, `asians_in_state`, that will give you the number of **Chinese**, **Japanese**, and **Other Asian** or **Pacific Islander** people in a given state.

```
# Enter on your own
```

3

Try making your own function, `'top_10_oldest_cities'`, that will give you the names of cities whose population's average age is top 10 oldest.

```
# Enter on your own
```

12.5 What is a package?

You can think of a package as a suite of functions that other people have already built for you to make your life easier.

```
help(package = "ggplot2")
```

To use a package, you need to do two things: (1) install it, and then (2) load it.

Installing is a one-time thing

```
install.packages("ggplot2")
```

But you need to load each time you start a R instance. So always keep these commands on a script.

```
library(ggplot2)
```

12.6 Conditionals

Sometimes, you want to execute a command only under certain conditions. This is done through the almost universal function, `if()`. Inside the `if` function we enter a logical statement. The line that is adjacent to, or follows, the `if()` statement only gets executed if the statement returns `TRUE`.

For example,

For example,

```
x <- 5
if (x > 0) {
  print("positive number")
} else if (x == 0) {
  print("zero")
} else {
  print("negative number")
}
```

```
[1] "positive number"
```

You can wrap that whole thing in a function

```
is_positive <- function(number) {
  if (number > 0) {
    print("positive number")
  } else if (number == 0) {
    print("zero")
  } else {
    print("negative number")
  }
}

is_positive(5)
```

```
[1] "positive number"
```

```
is_positive(-3)
```

```
[1] "negative number"
```

12.7 For-loops

Loops repeat the same statement, although the statement can be “the same” only in an abstract sense. Use the `for(x in X)` syntax to repeat the subsequent command as many times as there are elements in the right-hand object `X`. Each of these elements will be referred to the left-hand index `x`

First, come up with a vector.

```
fruits <- c("apples", "oranges", "grapes")
```

Now we use the `fruits` vector in a `for` loop.

```
for (fruit in fruits) {  
  print(paste("I love", fruit))  
}
```

```
[1] "I love apples"  
[1] "I love oranges"  
[1] "I love grapes"
```

Here `for()` and `in` must be part of any `for` loop. The right hand side `fruits` must be a thing that exists. Finally the **left-hand** side object is “Pick your favor name.” It is analogous to how we can index a sum with any letter. $\sum_{i=1}^{10} i$ and $\text{sum}_{\{j = 1\}^{10}} j$ are in fact the same thing.

```
for (i in 1:length(fruits)) {  
  print(paste("I love", fruits[i]))  
}
```

```
[1] "I love apples"  
[1] "I love oranges"  
[1] "I love grapes"
```

```

states_of_interest <- c("California", "Massachusetts", "New Hampshire", "Washington")

for( state in states_of_interest){
  state_data <- cen10[cen10$state == state,]
  nmen <- sum(state_data$sex == "Male")

  n <- nrow(state_data)
  men_perc <- round(100*(nmen/n), digits=2)
  print(paste("Percentage of men in",state, "is", men_perc))
}

```

```

[1] "Percentage of men in California is 49.85"
[1] "Percentage of men in Massachusetts is 47.6"
[1] "Percentage of men in New Hampshire is 48.55"
[1] "Percentage of men in Washington is 48.19"

```

Instead of printing, you can store the information in a vector

```

states_of_interest <- c("California", "Massachusetts", "New Hampshire", "Washington")
male_percentages <- c()
iter <- 1

for( state in states_of_interest){
  state_data <- cen10[cen10$state == state,]
  nmen <- sum(state_data$sex == "Male")
  n <- nrow(state_data)
  men_perc <- round(100*(nmen/n), digits=2)

  male_percentages <- c(male_percentages, men_perc)
  names(male_percentages)[iter] <- state
  iter <- iter + 1
}

male_percentages

```

California	Massachusetts	New Hampshire	Washington
49.85	47.60	48.55	48.19

12.8 Nested Loops

What if I want to calculate the population percentage of a race group for all race groups in states of interest? You could probably use tidyverse functions to do this, but let's try using loops!

```
states_of_interest <- c("California", "Massachusetts", "New Hampshire", "Washington")
for (state in states_of_interest) {
  for (race in unique(cen10$race)) {
    race_state_num <- nrow(cen10[cen10$race == race & cen10$state == state, ])
    state_pop <- nrow(cen10[cen10$state == state, ])
    race_perc <- round(100*(race_state_num/(state_pop)), digits=2)
    print(paste("Percentage of ", race , "in", state, "is", race_perc))
  }
}
```

```
[1] "Percentage of  White in California is 57.61"
[1] "Percentage of  Black/Negro in California is 6.72"
[1] "Percentage of  Other race, nec in California is 15.55"
[1] "Percentage of  American Indian or Alaska Native in California is 1.12"
[1] "Percentage of  Chinese in California is 3.75"
[1] "Percentage of  Other Asian or Pacific Islander in California is 9.54"
[1] "Percentage of  Two major races in California is 4.62"
[1] "Percentage of  Three or more major races in California is 0.37"
[1] "Percentage of  Japanese in California is 0.72"
[1] "Percentage of  White in Massachusetts is 79.6"
[1] "Percentage of  Black/Negro in Massachusetts is 5.87"
[1] "Percentage of  Other race, nec in Massachusetts is 4.02"
[1] "Percentage of  American Indian or Alaska Native in Massachusetts is 0.77"
[1] "Percentage of  Chinese in Massachusetts is 2.32"
[1] "Percentage of  Other Asian or Pacific Islander in Massachusetts is 4.33"
[1] "Percentage of  Two major races in Massachusetts is 2.78"
[1] "Percentage of  Three or more major races in Massachusetts is 0"
[1] "Percentage of  Japanese in Massachusetts is 0.31"
[1] "Percentage of  White in New Hampshire is 93.48"
[1] "Percentage of  Black/Negro in New Hampshire is 0.72"
[1] "Percentage of  Other race, nec in New Hampshire is 0.72"
[1] "Percentage of  American Indian or Alaska Native in New Hampshire is 0.72"
[1] "Percentage of  Chinese in New Hampshire is 0.72"
[1] "Percentage of  Other Asian or Pacific Islander in New Hampshire is 2.17"
[1] "Percentage of  Two major races in New Hampshire is 0.72"
```

```
[1] "Percentage of   Three or more major races in New Hampshire is 0"
[1] "Percentage of   Japanese in New Hampshire is 0.72"
[1] "Percentage of   White in Washington is 76.05"
[1] "Percentage of   Black/Negro in Washington is 2.9"
[1] "Percentage of   Other race, nec in Washington is 5.37"
[1] "Percentage of   American Indian or Alaska Native in Washington is 2.03"
[1] "Percentage of   Chinese in Washington is 1.31"
[1] "Percentage of   Other Asian or Pacific Islander in Washington is 6.68"
[1] "Percentage of   Two major races in Washington is 4.79"
[1] "Percentage of   Three or more major races in Washington is 0.29"
[1] "Percentage of   Japanese in Washington is 0.58"
```

Exercises

Exercise 1: Write your own function

Write your own function that makes some task of data analysis simpler. Ideally, it would be a function that helps you do either of the previous tasks in fewer lines of code. You can use the three lines of code that was provided in exercise 1 to wrap that into another function too!

```
# Enter yourself
```

Exercise 2: Using Loops

Using a loop, create a crosstab of sex and race for each state in the set “states_of_interest”

```
states_of_interest <- c("California", "Massachusetts", "New Hampshire", "Washington")
# Enter yourself
```

Exercise 3: Storing information derived within loops in a global dataframe

Recall the following nested loop

```
states_of_interest <- c("California", "Massachusetts", "New Hampshire", "Washington")
for (state in states_of_interest) {
  for (race in unique(cen10$race)) {
    race_state_num <- nrow(cen10[cen10$race == race & cen10$state == state, ])
    state_pop <- nrow(cen10[cen10$state == state, ])
```

```

    race_perc <- round(100*(race_state_num/(state_pop)), digits=2)
    print(paste("Percentage of ", race , "in", state, "is", race_perc))
  }
}

```

```

[1] "Percentage of White in California is 57.61"
[1] "Percentage of Black/Negro in California is 6.72"
[1] "Percentage of Other race, nec in California is 15.55"
[1] "Percentage of American Indian or Alaska Native in California is 1.12"
[1] "Percentage of Chinese in California is 3.75"
[1] "Percentage of Other Asian or Pacific Islander in California is 9.54"
[1] "Percentage of Two major races in California is 4.62"
[1] "Percentage of Three or more major races in California is 0.37"
[1] "Percentage of Japanese in California is 0.72"
[1] "Percentage of White in Massachusetts is 79.6"
[1] "Percentage of Black/Negro in Massachusetts is 5.87"
[1] "Percentage of Other race, nec in Massachusetts is 4.02"
[1] "Percentage of American Indian or Alaska Native in Massachusetts is 0.77"
[1] "Percentage of Chinese in Massachusetts is 2.32"
[1] "Percentage of Other Asian or Pacific Islander in Massachusetts is 4.33"
[1] "Percentage of Two major races in Massachusetts is 2.78"
[1] "Percentage of Three or more major races in Massachusetts is 0"
[1] "Percentage of Japanese in Massachusetts is 0.31"
[1] "Percentage of White in New Hampshire is 93.48"
[1] "Percentage of Black/Negro in New Hampshire is 0.72"
[1] "Percentage of Other race, nec in New Hampshire is 0.72"
[1] "Percentage of American Indian or Alaska Native in New Hampshire is 0.72"
[1] "Percentage of Chinese in New Hampshire is 0.72"
[1] "Percentage of Other Asian or Pacific Islander in New Hampshire is 2.17"
[1] "Percentage of Two major races in New Hampshire is 0.72"
[1] "Percentage of Three or more major races in New Hampshire is 0"
[1] "Percentage of Japanese in New Hampshire is 0.72"
[1] "Percentage of White in Washington is 76.05"
[1] "Percentage of Black/Negro in Washington is 2.9"
[1] "Percentage of Other race, nec in Washington is 5.37"
[1] "Percentage of American Indian or Alaska Native in Washington is 2.03"
[1] "Percentage of Chinese in Washington is 1.31"
[1] "Percentage of Other Asian or Pacific Islander in Washington is 6.68"
[1] "Percentage of Two major races in Washington is 4.79"
[1] "Percentage of Three or more major races in Washington is 0.29"
[1] "Percentage of Japanese in Washington is 0.58"

```

Instead of printing the percentage of each race in each state, create a dataframe, and store all that information in that dataframe. (Hint: look at how I stored information about male percentage in each state of interest in a vector.)

13 Joins and Merges, Wide and Long¹

Motivation

The “Democratic Peace” is one of the most widely discussed propositions in political science, covering the fields of International Relations and Comparative Politics, with insights to domestic politics of democracies (e.g. American Politics). The one-sentence idea is that democracies do not fight with each other. There have been much theoretical debate – for example in earlier work, [Oneal and Russett \(1999\)](#) argue that the democratic peace is not due to the hegemony of strong democracies like the U.S. and attempt to distinguish between realist and what they call Kantian propositions (e.g. democratic governance, international organizations)².

An empirical demonstration of the democratic peace is also a good example of a **Time Series Cross Sectional** (or panel) dataset, where the same units (in this case countries) are observed repeatedly for multiple time periods. Experience in assembling and analyzing a TSCS dataset will prepare you for any future research in this area.

Where are we? Where are we headed?

Up till now, you should have covered:

- R basic programming
- Counting.
- Visualization.
- Objects and Classes.
- Matrix algebra in R
- Functions.

Today you will work on your own, but feel free to ask a fellow classmate nearby or the instructor. The objective for this session is to get more experience using R, but in the process (a) test a prominent theory in the political science literature and (b) explore related ideas of interest to you.

¹Module originally written by Shiro Kuriwaki, Connor Jerzak, and Yon Soo Park

²[The Kantian Peace: The Pacific Benefits of Democracy, Interdependence, and International Organizations, 1885-1992. *World Politics* 52\(1\):1-37](#)

13.1 Setting up

```
library(dplyr)
library(tidyr)
library(readr)
library(ggplot2)
```

13.2 Create a project directory

First start a directory for this project. This can be done manually or through RStudio's Project feature (File > New Project...)

Directories is the computer science / programming name for folders. While advice about how to structure your working directories might strike you as petty, we believe that starting from some well-tested guides will go a long way in improving the quality and efficiency of your work.

Chapter 4 of Gentzkow and Shapiro's memo, [Code and Data for the Social Scientist](#) provides a good template.

13.3 Data Sources

Most projects you do will start with downloading data from elsewhere. For this task, you'll probably want to track down and download the following:

- **Correlates of war dataset (COW):** Find and download the Militarized Interstate Disputes (MIDs) data from the Correlates of War website: <http://www.correlatesofwar.org/data-sets>. Or a dyad-version on dataverse: <https://dataverse.harvard.edu/dataset.xhtml?persistentId=hdl:1902.1/11489>
- **PRIO Data on Armed Conflict:** Find and download the Uppsala Conflict Data Program (UCDP) and PRIO dyad-year data on armed conflict (<https://www.prio.org>) or this link to the flat csv file (<http://ucdp.uu.se/downloads/dyadic/ucdp-dyadic-171.csv>).
- **Polity:** The Polity data can be downloaded from their website (<http://www.systemicpeace.org/inscrdata.html>). Look for the newest version of the time series that has the widest coverage.

13.4 Example with 2 Datasets

Let's read in a sample dataset.

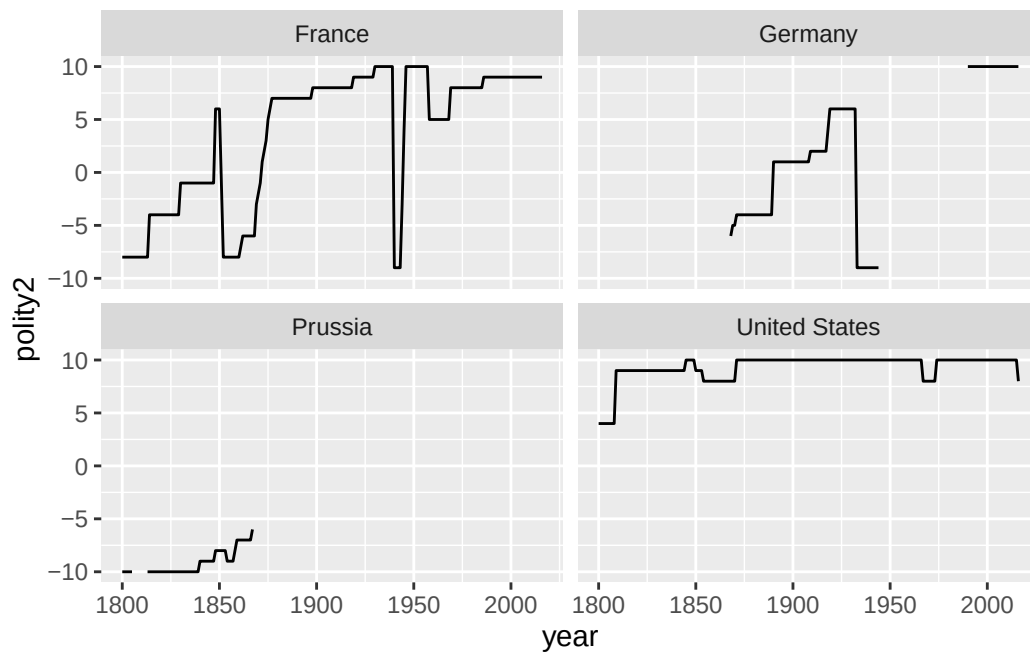
```
polity <- read_csv("data/input/sample_polity.csv")
mid <- read_csv("data/input/sample_mid.csv")
```

What does polity look like?

```
unique(polity$country)
```

```
[1] "France"          "Prussia"         "Germany"         "United States"
```

```
ggplot(polity, aes(x = year, y = polity2)) +
  facet_wrap(~ country) +
  geom_line()
```



```
head(polity)
```

```
# A tibble: 6 x 5
  scode ccode country  year polity2
  <chr> <dbl> <chr>    <dbl>    <dbl>
1 FRN    220 France   1800      -8
2 FRN    220 France   1801      -8
3 FRN    220 France   1802      -8
4 FRN    220 France   1803      -8
5 FRN    220 France   1804      -8
6 FRN    220 France   1805      -8
```

MID is a dataset that captures a **dispute** for a given country and year.

```
mid
```

```
# A tibble: 6,132 x 5
  ccode polity_code dispute StYear EndYear
  <dbl> <chr>          <dbl> <dbl>    <dbl>
1   200 UKG             1  1902    1903
2     2 USA             1  1902    1903
3   345 YGS             1  1913    1913
4   300 <NA>            1  1913    1913
5   339 ALB             1  1946    1946
6   200 UKG             1  1946    1946
7   200 UKG             1  1951    1952
8   651 EGY             1  1951    1952
9   630 IRN             1  1856    1857
10  200 UKG             1  1856    1857
# i 6,122 more rows
```

13.5 Loops

Notice that in the `mid` data, we have a start of a dispute vs. an end of a dispute. In order to combine this into the `polity` data, we want a way to give each of the interval years a row.

There are many ways to do this, but one is a loop. We go through one row at a time, and then for each we make a new dataset. that has **year** as a sequence of each year. A lengthy loop like this is typically slow, and you'd want to recast the task so you can do things with functions. But, a loop is a good place to start.

```
mid_year_by_year <- data_frame(ccode = numeric(),
                               year = numeric(),
                               dispute = numeric())
```

Warning: `data\_frame()` was deprecated in tibble 1.1.0.
i Please use `tibble()` instead.

```
for(i in 1:nrow(mid)) {
  x <- data_frame(ccode = mid$ccode[i], ## row i's country
                 year = mid$StYear[i]:mid$EndYear[i], ## sequence of years for dispute in row i
                 dispute = 1)
  mid_year_by_year <- rbind(mid_year_by_year, x)
}

head(mid_year_by_year)
```

```
# A tibble: 6 x 3
  ccode  year dispute
  <dbl> <int>   <dbl>
1   200  1902       1
2   200  1903       1
3     2  1902       1
4     2  1903       1
5   345  1913       1
6   300  1913       1
```

13.6 Merging

We want to combine these two datasets by merging. Base-R has a function called `merge`. `dplyr` has several types of joins (the same thing). Those names are based on SQL syntax.

a

x1	x2
A	1
B	2
C	3

+
b

x1	x3
A	T
B	F
D	T

=

Mutating Joins

x1	x2	x3
A	1	T
B	2	F
C	3	NA

x1	x3	x2
A	T	1
B	F	2
D	T	NA

x1	x2	x3
A	1	T
B	2	F

x1	x2	x3
A	1	T
B	2	F
C	3	NA
D	NA	T

dplyr::left_join(a, b, by = "x1")
Join matching rows from b to a.

dplyr::right_join(a, b, by = "x1")
Join matching rows from a to b.

dplyr::inner_join(a, b, by = "x1")
Join data. Retain only rows in both sets.

dplyr::full_join(a, b, by = "x1")
Join data. Retain all values, all rows.

Here we can do a `left_join` matching rows from `mid` to `polity`. We want to keep the rows in `polity` that do not match in `mid`, and label them as non-disputes.

```
p_m <- left_join(polity,
                 distinct(mid_year_by_year),
                 by = c("ccode", "year"))

head(p_m)
```

```
# A tibble: 6 x 6
  scode ccode country  year polity2 dispute
  <chr> <dbl> <chr>    <dbl>   <dbl>   <dbl>
1 FRN    220 France   1800     -8      NA
2 FRN    220 France   1801     -8      NA
3 FRN    220 France   1802     -8      NA
4 FRN    220 France   1803     -8      NA
5 FRN    220 France   1804     -8      NA
6 FRN    220 France   1805     -8      NA
```

Replace `dispute = NA` rows with a zero.

```
p_m$dispute[is.na(p_m$dispute)] <- 0
```

Reshape the dataset long to wide

```
p_m_wide <- pivot_wider(p_m,
  id_cols = c(scode, ccode, country),
  names_from = year,
  values_from = polity2)

select(p_m_wide, 1:10)
```

```
# A tibble: 4 x 10
  scode ccode country `1800` `1801` `1802` `1803` `1804` `1805` `1806`
  <chr> <dbl> <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 FRN    220 France      -8     -8     -8     -8     -8     -8     -8
2 GMY    255 Prussia    -10    -10    -10    -10    -10    -10    NA
3 GMY    255 Germany     NA     NA     NA     NA     NA     NA     NA
4 USA      2 United States  4      4      4      4      4      4      4
```

13.7 Main Project

Try building a panel that would be useful in answering the Democratic Peace Question, perhaps in these steps.

Task 1: Data Input and Standardization

Often, files we need are saved in the `.xls` or `xlsx` format. It is possible to read these files directly into R, but experience suggests that this process is slower than converting them first to `.csv` format and reading them in as `.csv` files.

`readxl/readr/haven` packages(<https://github.com/tidyverse/tidyverse>) is constantly expanding to capture more file types. In day 1, we used the package `readxl`, using the `read_excel()` function.

Task 2: Data Merging

We will use data to test a version of the Democratic Peace Thesis (DPS). Democracies are said to go to war less because the leaders who wage wars are accountable to voters who have to bear the costs of war. Are democracies less likely to engage in militarized interstate disputes?

To start, let's download and merge some data.

- Load in the Militarized Interstate Dispute (MID) files. Militarized interstate disputes are hostile action between two formally recognized states. Examples of this would be threats to use force, threats to declare war, beginning war, fortifying a border with troops, and so on.
- Find a way to **merge** the Polity IV dataset and the MID data. This process can be a bit tricky.
- An *advanced* version of this task would be to download the dyadic form of the data and try merging that with polity.

Task 3: Tabulations and Visualization

1. Calculate the mean Polity2 score by year. Plot the result. Use graphical indicators of your choosing to show where key events fall in this timeline (such as 1914, 1929, 1939, 1989, 2008). Speculate on why the behavior from 1800 to 1920 seems to be qualitatively different than behavior afterwards.
2. Do the same but only among state-years that were involved in a MID. Plot this line together with your results from 1.
3. Do the same but only among state years that were *not* involved in a MID.
4. Arrive at a tentative conclusion for how well the Democratic Peace argument seems to hold up in this dataset. Visualize this conclusion.

14 Functionals and Optimization

14.1 More Tidyverse/Data Cleaning

For our main example in this session, we'll take a look at the the 2022 Cooperative Election Survey (CES). The `ces22.dta` file contains a subset of questions asked to respondents in the 2022 wave of this annual survey stored in Stata's `dta` format.

```
ces <- haven::read_dta("data/input/ces22_subset.dta")
```

Stata files often contain metadata for each column. In the CES, responses are stored as numeric values that represent one of the discrete choices available to the respondent. As a result, while many of the columns have numeric data types, they should not be treated as numbers. This is something to be careful of as you work with any dataset – always read the codebook.

For example, while we could take the “mean” of the race variable, this is a nonsense quantity.

```
mean(ces$race)
```

```
[1] 1.696817
```

We can take a closer look at how the race variable is stored:

```
head(ces$race)
```

```
<labelled<double>[6]>: Race  
[1] 1 1 1 1 1 7
```

Labels:

value	label
1	White
2	Black
3	Hispanic
4	Asian

```

5   Native American
6   Two or more races
7           Other
8   Middle Eastern
98           skipped
99           not asked

```

And take a closer look at its class

```
class(ces$race)
```

```
[1] "haven_labelled" "vctrs_vctr"      "double"
```

`haven` reads in the Stata metadata and creates a column that has the `haven_labelled` class, which stores both the numeric labels and the metadata which denotes the mapping between the numeric label and its meaning. We'd like to use this to convert the variables to factors that we can work with in R. To do this, we'll be using the `as_factor()` function. This function is "generic" and has a specific implementation for `haven_labelled` objects in the `haven` package

```
head(as_factor(ces$race))
```

```

[1] White White White White White Other
10 Levels: White Black Hispanic Asian Native American ... not asked

```

Or in "tidy" style

```
ces %>% mutate(race = as_factor(race))
```

```

# A tibble: 60,000 x 28
   caseid commonweight commonpostweight gender4 educ  race  hispanic pid3
   <dbl>      <dbl>          <dbl> <dbl+1> <dbl+1> <fct> <dbl+1b> <dbl+1>
1  1.98e9      3.65            3.53    1 [Man]  6 [Pos~ White 2 [No]  1 [Dem~
2  1.98e9      0.780           0.819    1 [Man]  3 [Som~ White 2 [No]  3 [Ind~
3  1.98e9      0.892           0.774    2 [Wom~  5 [4-y~ White 2 [No]  1 [Dem~
4  1.98e9      1.10            1.21     3 [Non~  6 [Pos~ White 2 [No]  4 [Oth~
5  1.98e9      0.543           0.328    1 [Man]  6 [Pos~ White 2 [No]  3 [Ind~
6  1.98e9      0.114           0.0989   1 [Man]  5 [4-y~ Other 2 [No]  3 [Ind~
7  1.98e9      0.899           1.21     2 [Wom~  2 [Hig~ Black 2 [No]  1 [Dem~

```

```

 8  1.98e9      0.633      0.431  1 [Man]  6 [Pos~ White 2 [No]   1 [Dem~
 9  1.98e9      0.900      0.854  2 [Wom~  5 [4-y~ White 2 [No]   1 [Dem~
10  1.98e9      0.725      0.586  1 [Man]  6 [Pos~ White 2 [No]   1 [Dem~
# i 59,990 more rows
# i 20 more variables: pid7 <dbl+lbl>, inputstate <dbl+lbl>,
#   CC22_306 <dbl+lbl>, CC22_320a <dbl+lbl>, CC22_320b <dbl+lbl>,
#   CC22_320c <dbl+lbl>, CC22_320d <dbl+lbl>, CC22_320e <dbl+lbl>,
#   CC22_320f <dbl+lbl>, CC22_320g <dbl+lbl>, CC22_320h <dbl+lbl>,
#   page_CC22_320grid_timing <dbl>, CC22_333 <dbl+lbl>, CC22_333a <dbl+lbl>,
#   CC22_333b <dbl+lbl>, CC22_333c <dbl+lbl>, CC22_333d <dbl+lbl>, ...

```

But we'd like to do this for every column in our dataset (except for columns that are numeric). We could manually go through every single column and convert it to a factor, or we could use some tidyverse tools.

The `across()` function allows us to select columns using the same custom selection syntax as in `select()`. We can combine this with the `mutate` function to mutate every column that we want to transform. Here, we want to select the columns that are of a certain **class** – the “labelled” class.

```
ces <- ces %>% mutate(across(where(is.labelled), as_factor))
```

`across()`, like `select()` is very flexible in how columns can be selected. In addition to selecting by number, you can select by partial string match, numerical sequence, or even regular expressions!

Now all of our columns operate correctly as factors.

```
ces
```

```
# A tibble: 60,000 x 28
```

	caseid	commonweight	commonpostweight	gender4	educ	race	hispanic	pid3	pid7
	<dbl>	<dbl>	<dbl>	<fct>	<fct>	<fct>	<fct>	<fct>	<fct>
1	1.98e9	3.65	3.53	Man	Post~	White	No	Demo~	Stro~
2	1.98e9	0.780	0.819	Man	Some~	White	No	Inde~	Lean~
3	1.98e9	0.892	0.774	Woman	4-ye~	White	No	Demo~	Stro~
4	1.98e9	1.10	1.21	Non-bi~	Post~	White	No	Other	Inde~
5	1.98e9	0.543	0.328	Man	Post~	White	No	Inde~	Inde~
6	1.98e9	0.114	0.0989	Man	4-ye~	Other	No	Inde~	Inde~
7	1.98e9	0.899	1.21	Woman	High~	Black	No	Demo~	Not ~
8	1.98e9	0.633	0.431	Man	Post~	White	No	Demo~	Not ~
9	1.98e9	0.900	0.854	Woman	4-ye~	White	No	Demo~	Stro~
10	1.98e9	0.725	0.586	Man	Post~	White	No	Demo~	Not ~

```
# i 59,990 more rows
# i 19 more variables: inputstate <fct>, CC22_306 <fct>, CC22_320a <fct>,
#   CC22_320b <fct>, CC22_320c <fct>, CC22_320d <fct>, CC22_320e <fct>,
#   CC22_320f <fct>, CC22_320g <fct>, CC22_320h <fct>,
#   page_CC22_320grid_timing <dbl>, CC22_333 <fct>, CC22_333a <fct>,
#   CC22_333b <fct>, CC22_333c <fct>, CC22_333d <fct>, CC22_333e <fct>,
#   page_CC22_333_timing <dbl>, page_CC22_333grid_timing <dbl>
```

We can do some more steps to make our data easier to work with. First, we can rename columns to be more informative. For example, `CC22_306` is a question on vaccination status. Let's `rename()` that column to make it easier to reference

```
ces <- ces %>% rename(vaccinated = CC22_306) #New aname = old name
```

`vaccinated` has a number of different levels

```
table(ces$vaccinated)
```

```

I am fully vaccinated and have received at least one booster shot
338
I am fully vaccinated but have not received a booster shot
103
I am partially vaccinated (I have received the first of two shots for either Pfizer or Moderna)
13
I am not vaccinated at all
134
skipping
not asked
```

Let's recode this categorical variable to a binary indicator for whether a respondent is fully vaccinated. Note that two of these categories have "fully vaccinated". We want to code these as a value of 1 and the rest as 0. To do this we'll use the `case_when()` function inside of a `mutate()`

```
ces <- ces %>% mutate(fullyvax = case_when(str_detect(vaccinated, "fully vaccinated") ~ 1,
!str_detect(vaccinated, "fully vaccinated") ~ 0,
is.na(vaccinated) ~ NA_real_))
```

How many respondents are fully vaccinated?

```
table(ces$fullyvax)
```

```
0      1  
15423 44424
```

CC22_320 contains approval ratings of various political institutions. Let's pull the three big ones (President, Legislature, Supreme Court) and create indicators for whether the respondent approves or disapproves of that institution.

```
ces <- ces %>% mutate(approvePresident = case_when(str_detect(CC22_320a, "disapprove") ~ 0,  
                                                    str_detect(CC22_320a, "approve") ~ 1,  
                                                    str_detect(CC22_320a, "Not sure") ~ 0),  
                    approveCongress = case_when(str_detect(CC22_320b, "disapprove") ~ 0,  
                                                  str_detect(CC22_320b, "approve") ~ 1,  
                                                  str_detect(CC22_320b, "Not sure") ~ 0),  
                    approveCourt = case_when(str_detect(CC22_320c, "disapprove") ~ 0,  
                                               str_detect(CC22_320c, "approve") ~ 1,  
                                               str_detect(CC22_320c, "Not sure") ~ 0))
```

Let's pull those three columns, party ID and the survey weights and analyze them further.

```
ces_approval <- ces %>% select(commonweight, pid3, approvePresident, approveCongress, approveCourt)
```

Functionals

Many functions in R act on functions as input. You've already seen a lot of **tidyverse** functions that do this. For example, the **summarize()** function takes as input a dataset and a function or functions describing what operations should be done on the dataset. Above, we used **mutate** which took the functional arguments **across** and **as_factor**.

R is a *functional programming* language in that functions are “first-class citizens” and can be treated as any other data type. They can be given a name, passed as inputs to other functions, and returned as output.

This allows programs to be written in terms of compositions of functions. In fact, this is one of the principles of the **tidy** project as outlined by Hadley Wickham, and many of the constructs provided by the tidyverse encourage you to work this way. See the [tidy manifesto](#) for more.

14.1.1 Replacing for loops

R programmers tend to favor functional replacements for `for` loops. Rather than iterate over elements of a vector or list and execute some operation for each iteration, you can use one of the `apply` (Base-R) or `map` (tidyverse) functions.

To illustrate, let's take a look at our approval rating data from before. How would we write a `for`-loop to calculate the (weighted) mean for each column?

```
approval_data <- ces_approval %>% select(approvePresident, approveCongress, approveCourt)
for_time <- proc.time() # Store current time -- used for timing speed of function
approval <- rep(NA, 3)
names(approval) <- c("approvePresident", "approveCongress", "approveCourt")
for (name in names(approval)){
  approval[name] <- weighted.mean(approval_data[[name]], ces_approval$commonweight, na.rm=TRUE)
}
print(proc.time() - for_time)
```

```
user  system elapsed
0.008   0.000   0.009
```

```
print(approval)
```

```
approvePresident  approveCongress  approveCourt
           0.4131196           0.2635909           0.3600206
```

This is pretty terrible looking. We've already seen the `summarize()` function work for this task, but let's illustrate another operation in base-R, the `apply` function. `apply()` operates on matrices and applies a function to each row or column.

```
for_time <- proc.time() # Store current time -- used for timing speed of function
approval <- apply(approval_data, 2, function(x) weighted.mean(x, ces_approval$commonweight, na.rm=TRUE))
print(proc.time() - for_time)
```

```
user  system elapsed
0.005   0.000   0.006
```

```
print(approval)
```

approvePresident	approveCongress	approveCourt
0.4131196	0.2635909	0.3600206

Note that there aren't really substantial speed benefits. Rather, the code just looks cleaner. There are some common functions that **are** faster, but they have been specifically optimized for speed. Consider `colMeans` or `rowMeans` for simple (unweighted) means.

Also note how we defined an “in-line” function within the `apply()` call. This is sometimes called a “lambda” or “anonymous” function. We won't be able to call it outside of the `apply()` call since it has no name, but there's not ever a reason why we would want to.

`sapply()` works on generic lists while `tapply()` can also apply a function based on the value of an index.

However, if you are using `tidyverse`, you probably should prefer the equivalent “generic” version of `apply()` – the `map` family of functions. They vary primarily in how they return their output. `map()` always returns a list while other forms (like `map_dbl()` or `map_chr()`) will force this list to be a vector of the specified type.

```
approval_data %>% map_dbl(function(x) weighted.mean(x, ces_approval$commonweight, na.rm=T))
```

approvePresident	approveCongress	approveCourt
0.4131196	0.2635909	0.3600206

You can use `map()` on a vector to iterate over a function repeatedly

```
map(1:10, function(x) sample(1:100, 1)) # Sample a number from 1:100 ten times.
```

```
[[1]]
[1] 58

[[2]]
[1] 25

[[3]]
[1] 100

[[4]]
[1] 3

[[5]]
```

```
[1] 12
```

```
[[6]]  
[1] 49
```

```
[[7]]  
[1] 89
```

```
[[8]]  
[1] 65
```

```
[[9]]  
[1] 24
```

```
[[10]]  
[1] 7
```

Note that when applying functions to grouped columns of a data frame, you should probably still be using `summarize()` for most cases. `group_map()` has more flexibility, but can be a bit more difficult to implement.

```
ces_approval %>% group_by(pid3) %>% summarize_at(vars(contains("approve")), ~ weighted.mean
```

```
# A tibble: 5 x 4  
  pid3      approvePresident approveCongress approveCourt  
  <fct>          <dbl>          <dbl>          <dbl>  
1 Democrat      0.837            0.507            0.205  
2 Republican    0.0646           0.113            0.607  
3 Independent   0.338            0.185            0.346  
4 Other         0.270            0.121            0.343  
5 Not sure      0.248            0.150            0.170
```

14.2 Excerise: Optimization

In this section, we'll illustrate another valuable use of functional programming – optimization routines. Many tasks require finding maxima or minima of functions. Most functions of interest are very difficult to analyze analytically and rarely does a **closed-form solution** for the optima exist. However, a large number of methods exist that allow you to find a numerical solution to this problem.

Consider the following function.

$$f(x_1, x_2) = -x_1^2 + 2x_1 - 2x_2^2 + 3x_2 + x_1x_2 + 2$$

Let's find its maximum using numerical optimization

```
polynom <- function(x){
  return(-x[1]^2 + 2*x[1] - 2*x[2]^2 + 3*x[2] + x[1]*x[2] + 2)
}
```

We will use an implementation of the **BFGS** algorithm. This is an extension of the classic Newton-Raphson method. To find the maximum of a function, this approach starts with an initial guess x_n . Then, each subsequent step updates x_n by taking steps in the direction of the gradient, scaled by the Hessian. In a single dimension, the update step is:

$$x_{n+1} = x_n + \frac{f'(x_n)}{f''(x_n)}$$

For minimization, we replace the plus with a minus (as we want to move **away** from the gradient).

BFGS is implemented alongside many other algorithms in the **optim** function that is part of Base-R. **optim** takes as input a set of initial parameters, the name of the function to be optimized. A function that returns the gradient can also be specified, but this is optional. If not provided, **optim** routines that use the gradient will approximate it using a finite difference method (evaluating the function at small changes in the inputs).

```
max_1 <- optim(c(0,0), polynom, method = "BFGS", hessian=T, control=list(fnscale=-1))
```

Note that we have set **hessian=T** to return an evaluation of the hessian matrix at the critical point. We have also set this as a **maximization** problem by using the **fnscale** argument in **control**. By default, **optim()** minimizes the function. **fnscale** flips the function so that minimization is maximization of the original function.

Let's see the output

```
max_1
```

```
$par
```

```
[1] 1.571269 1.142798
```

```
$value
```

```
[1] 5.285714
```

```
$counts
function gradient
      10      6
```

```
$convergence
[1] 0
```

```
$message
NULL
```

```
$hessian
      [,1] [,2]
[1,]   -2    1
[2,]    1   -4
```

The x solution is stored in `$par`, the value at the maximum is stored in `$value`. `$convergence` describes whether the optimization routine converged or not (0 = success). We can confirm that this is a maximum by showing that the Hessian is negative definite and that all the eigenvalues of the Hessian are negative.

```
eigen(max_1$hessian)
```

```
eigen() decomposition
$values
[1] -1.585786 -4.414214
```

```
$vectors
      [,1]      [,2]
[1,] -0.9238795 -0.3826834
[2,] -0.3826834  0.9238795
```

Note that starting values matter. Choosing a starting point far away from the true solution means that more steps are required to reach convergence. Here, the function is well-behaved enough that even choosing $x_0 = \{1000, -100\}$ doesn't increase the number of steps too much, but some functions can be very poorly behaved (near-zero gradients) for some inputs.

```
max_2 <- optim(c(1000,-100), polynom, method = "BFGS", hessian=T, control=list(fnscale=-1))
max_2
```

```
$par  
[1] 1.571428 1.142848
```

```
$value  
[1] 5.285714
```

```
$counts  
function gradient  
      25      10
```

```
$convergence  
[1] 0
```

```
$message  
NULL
```

```
$hessian  
      [,1] [,2]  
[1,]    -2    1  
[2,]     1   -4
```

Challenge problem:

Consider the following function.

```
myfun <- function(x, mu=5, sigma=10){  
  return(((x*sigma*sqrt(2*pi))^-1)*exp(-(log(x) - mu)^2/(2*sigma^2)))  
}
```

Using `optim`, find the maximum of `myfun` for parameters `mu=5` and `sigma=10`.

Hint: `myfun` is defined only over the positive reals, but `optim` assumes the inputs are unbounded (at least for BFGS). Try to transform the inputs such that `optim` will work (you can do this with a lambda function).

Now, find the maximum for parameters `mu=7` and `sigma=5`

Challenge Problem 2:

Write a function that returns the gradient of `polynom`. Pass this function as an argument to `optim`. Compare the speed of the optimizer when the gradient is known in closed form vs. when it is approximated.

15 Simulation¹

Motivation: Simulation as an Analytical Tool

An increasing amount of political science contributions now include a simulation.

- [Axelrod \(1977\)](#) demonstrated via simulation how atomized individuals evolve to be grouped in similar clusters or countries, a model of culture.²
- [Chen and Rodden \(2013\)](#) argued in a 2013 article that the vote-seat inequality in U.S. elections that is often attributed to intentional partisan gerrymandering can actually be attributed to simply the reality of “human geography” – Democratic voters tend to be concentrated in smaller areas. Put another way, no feasible form of gerrymandering could spread out Democratic voters in such a way to equalize their vote-seat translation effectiveness. After demonstrating the empirical pattern of human geography, they advance their key claim by simulating thousands of redistricting plans and record the vote-seat ratio.³
- [Gary King, James Honaker, and multiple other authors](#) propose a way to analyze missing data with a method of multiple imputation, which uses a lot of simulation from a researcher’s observed dataset.⁴ (Software: Amelia⁵)

Statistical methods also incorporate simulation:

- The bootstrap: a statistical method for estimating uncertainty around some parameter by re-sampling observations.
- Bagging: a method for improving machine learning predictions by re-sampling observations, storing the estimate across many re-samples, and averaging these estimates to form the final estimate. A variance reduction technique.
- Statistical reasoning: if you are trying to understand a quantitative problem, a wonderful first-step to understand the problem better is to simulate it! The analytical solution is often very hard (or impossible), but the simulation is often much easier :-)

¹Module originally written by Connor Jerzak and Shiro Kuriwaki

²Axelrod, Robert. 1997. “The Dissemination of Culture.” *Journal of Conflict Resolution* 41(2): 203–26.

³Chen, Jowei, and Jonathan Rodden. “Unintentional Gerrymandering: Political Geography and Electoral Bias in Legislatures. *Quarterly Journal of Political Science*, 8:239-269”

⁴King, Gary, et al. “Analyzing Incomplete Political Science Data: An Alternative Algorithm for Multiple Imputation”. *American Political Science Review*, 95: 49-69.

⁵James Honaker, Gary King, Matthew Blackwell (2011). Amelia II: A Program for Missing Data. *Journal of Statistical Software*, 45(7), 1-47.

Where are we? Where are we headed?

Up till now, you should have covered:

- R basics
- Visualization
- Matrices and vectors
- Functions, objects, loops
- Joining real data

In this module, we will start to work with generating data within R, from thin air, as it were. Doing simulation also strengthens your understanding of Probability (Section [@ref{probability}](#)).

Check your Understanding

- What does the `sample()` function do?
- What does `runif()` stand for?
- What is a `seed`?
- What is a Monte Carlo?

Check if you have an idea of how you might code the following tasks:

- Simulate 100 rolls of a die
- Simulate one random ordering of 25 numbers
- Simulate 100 values of white noise (uniform random variables)
- Generate a “bootstrap” sample of an existing dataset

We’re going to learn about this today!

15.1 Pick a sample, any sample

15.2 The `sample()` function

The core functions for coding up stochastic data revolves around several key functions, so we will simply review them here.

Suppose you have a vector of values `x` and from it you want to randomly sample a sample of length `size`. For this, use the `sample` function

```
sample(x = 1:10, size = 5)
```

```
[1] 6 9 2 1 10
```

There are two subtypes of sampling – with and without replacement.

1. Sampling without replacement (`replace = FALSE`) means once an element of `x` is chosen, it will not be considered again:

```
sample(x = 1:10, size = 10, replace = FALSE) ## no number appears more than once
```

```
[1] 1 4 2 9 5 6 7 3 10 8
```

2. Sampling with replacement (`replace = TRUE`) means that even if an element of `x` is chosen, it is put back in the pool and may be chosen again.

```
sample(x = 1:10, size = 10, replace = TRUE) ## any number can appear more than once
```

```
[1] 10 1 7 10 4 1 7 5 2 2
```

It follows then that you cannot sample without replacement a sample that is larger than the pool.

```
sample(x = 1:10, size = 10, replace = FALSE)
```

```
[1] 9 6 1 10 8 5 4 7 3 2
```

So far, every element in `x` has had an equal probability of being chosen. In some application, we want a sampling scheme where some elements are more likely to be chosen than others. The argument `prob` handles this.

For example, this simulates 20 fair coin tosses (each outcome is equally likely to happen)

```
sample(c("Head", "Tail"), size = 20, prob = c(.5, .5), replace = TRUE)
```

```
[1] "Tail" "Head" "Head" "Tail" "Head" "Tail" "Head" "Head" "Tail" "Tail"
[11] "Tail" "Tail" "Tail" "Head" "Head" "Head" "Tail" "Tail" "Tail" "Tail"
```

But this simulates 20 biased coin tosses, where say the probability of Tails is 4 times more likely than the number of Heads

```
sample(c("Head", "Tail"), size = 20, prob = c(0.2, 0.8), replace = TRUE)
```

```
[1] "Tail" "Tail" "Tail" "Tail" "Tail" "Tail" "Tail" "Tail" "Tail" "Tail"
[11] "Tail" "Tail" "Tail" "Head" "Head" "Tail" "Tail" "Tail" "Head" "Tail"
```

15.2.1 Sampling rows from a dataframe

In tidyverse, there is a convenience function to sample rows randomly: `slice_sample()`

For example, load the dataset on cars, `mtcars`, which has 32 observations.

```
mtcars
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
Dodge Challenger	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	2
AMC Javelin	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2
Camaro Z28	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	4
Pontiac Firebird	19.2	8	400.0	175	3.08	3.845	17.05	0	0	3	2
Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1

Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2
Ford Pantera L	15.8	8	351.0	264	4.22	3.170	14.50	0	1	5	4
Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.50	0	1	5	6
Maserati Bora	15.0	8	301.0	335	3.54	3.570	14.60	0	1	5	8
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2

sample_n picks a user-specified number of rows from the dataset:

```
mtcars[sample(1:ncol(mtcars), size=3, replace=F),]
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2

```
slice_sample(mtcars, n=3)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4

Sometimes you want a X percent sample of your dataset. In this case use the prop argument to slice_sample

```
mtcars %>% slice_sample(prop=.5)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.50	0	1	5	6
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2
Maserati Bora	15.0	8	301.0	335	3.54	3.570	14.60	0	1	5	8
Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4

Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Pontiac Firebird	19.2	8	400.0	175	3.08	3.845	17.05	0	0	3	2
Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1

A common task in statistics is to generate “bootstrap” samples of your data. This involves sampling a new dataset of equivalent size (**n** rows) but **with replacement** (so some observations are dropped; others might appear 2 or 3 times). You can do this with `slice_sample` as well

```
mtcars %>% slice_sample(prop=1, replace=T)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Mazda RX4...3	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Merc 280...4	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
Fiat 128...6	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2
Porsche 914-2...8	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
AMC Javelin...9	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2
Chrysler Imperial...10	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Dodge Challenger	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	2
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Porsche 914-2...15	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.50	0	1	5	6
Merc 450SL...17	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
Fiat 128...18	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Merc 450SL...19	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
Porsche 914-2...20	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
AMC Javelin...21	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2
Mazda RX4...22	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Chrysler Imperial...23	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Mazda RX4...25	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4

Chrysler Imperial...	26	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Merc 230		22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Ford Pantera L		15.8	8	351.0	264	4.22	3.170	14.50	0	1	5	4
Merc 280...	29	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Toyota Corona...	30	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
Toyota Corona...	31	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
Maserati Bora		15.0	8	301.0	335	3.54	3.570	14.60	0	1	5	8

As a side-note, these functions have very practical uses for any type of data analysis:

- Inspecting your dataset: using `head()` all the same time and looking over the first few rows might lead you to ignore any issues that end up in the bottom for whatever reason.
- Testing your analysis with a small sample: If running analyses on a dataset takes more than a handful of seconds, change your dataset upstream to a fraction of the size so the rest of the code runs in less than a second. Once verifying your analysis code runs, then re-do it with your full dataset (by simply removing the `sample_n / sample_frac` line of code in the beginning). While three seconds may not sound like much, they accumulate and eat up time.

15.3 Random numbers from specific distributions

`rbinom()`

`rbinom` builds upon `sample` as a tool to help you answer the question – what is the *total number of successes* I would get if I sampled a binary (Bernoulli) result from a test with `size` number of trials each, with a event-wise probability of `prob`. The first argument `n` asks me how many such numbers I want.

For example, I want to know how many Heads I would get if I flipped a fair coin 100 times.

```
rbinom(n = 100, size = 1, prob = 0.1)
```

```
[1] 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 1 0 0 0 0
[38] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
[75] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0
```

Now imagine this I wanted to do this experiment 10 times, which would require I flip the coin $10 \times 100 = 1000$ times! Helpfully, we can do this in one line

```
rbinom(n = 10, size = 100, prob = 0.5)
```

```
[1] 46 41 50 50 59 53 57 49 51 56
```

`runif()`

`runif` also simulates a stochastic scheme where each event has equal probability of getting chosen like `sample`, but is a continuous rather than discrete system. We will cover this more in the next math module.

The intuition to emphasize here is that one can generate potentially infinite amounts (size `n`) of noise that is a essentially random

```
runif(n = 5, min=0, max=1)
```

```
[1] 0.01051054 0.57140770 0.35731914 0.36474572 0.28763365
```

`rnorm()`

`rnorm` is also a continuous distribution, but draws from a Normal distribution – perhaps the most important distribution in statistics. It runs the same way as `runif`

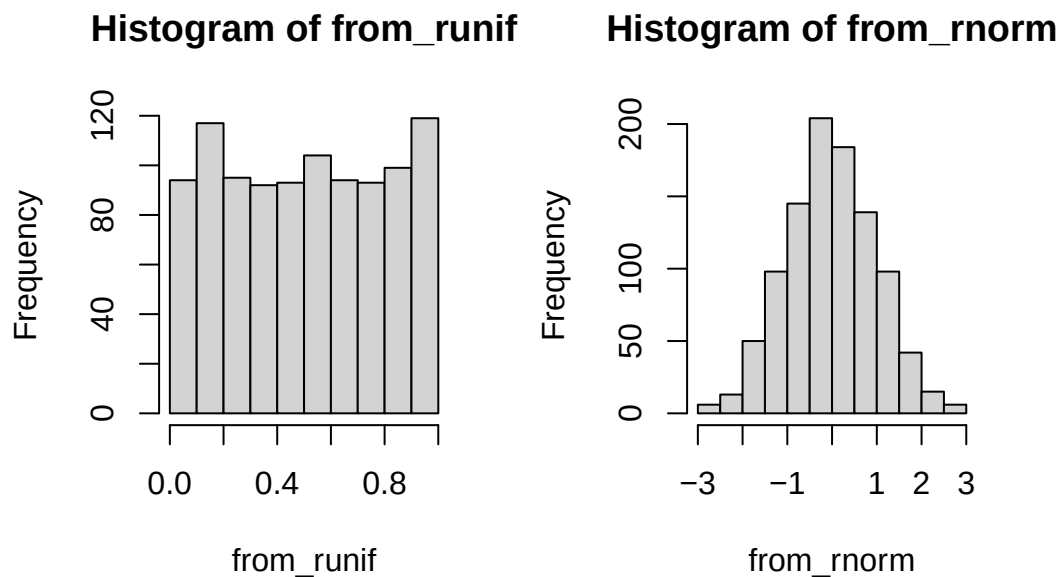
```
rnorm(n=5, mean=0, sd=2)
```

```
[1] 1.9375519 -0.9540334 -0.7534709 -1.4081971 3.5492358
```

To better visualize the difference between the output of `runif` and `rnorm`, let's generate lots of each and plot a histogram.

```
from_runif <- runif(n = 1000)
from_rnorm <- rnorm(n = 1000)

par(mfrow = c(1, 2)) ## base-R parameter for two plots at once
hist(from_runif)
hist(from_rnorm)
```

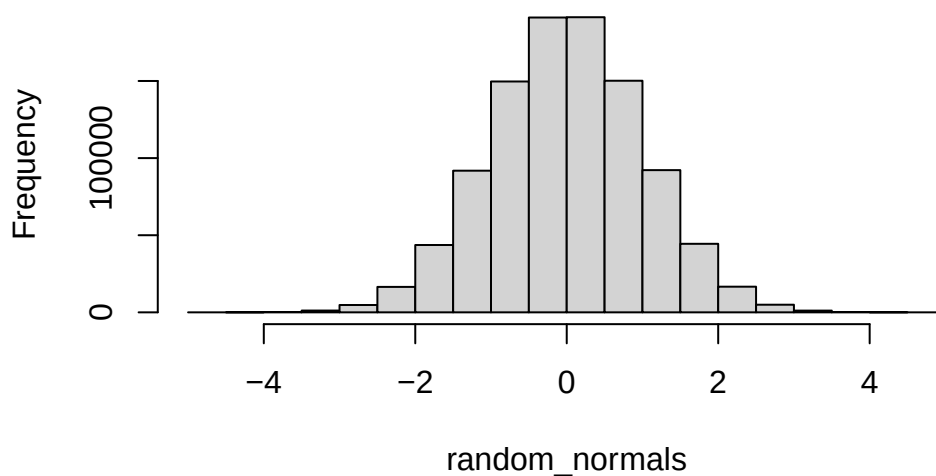


15.4 r, p, and d

Each distribution can do more than generate random numbers (the prefix `r`). We can compute the cumulative probability by the function `pbinom()`, `punif()`, and `pnorm()`. Also the density – the value of the PDF – by `dbinom()`, `dunif()` and `dnorm()`.

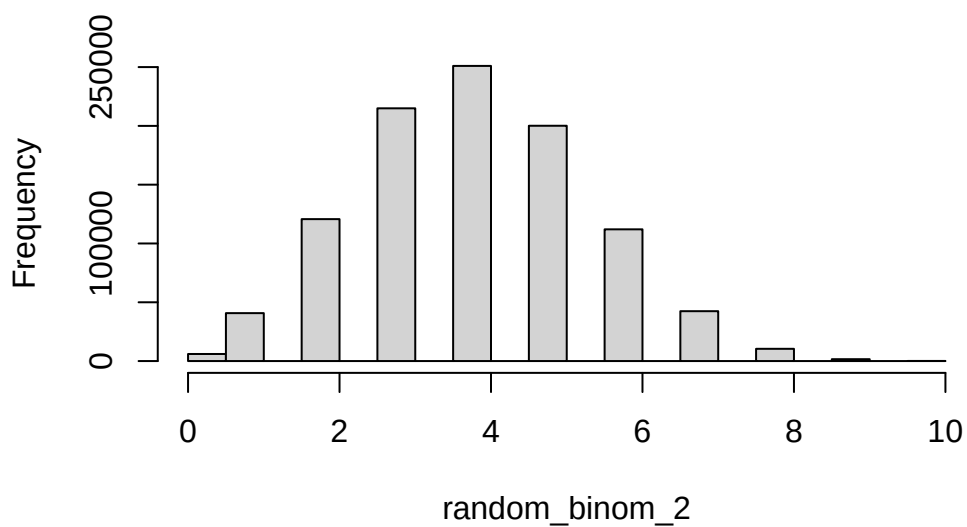
```
random_normals <- rnorm(n=1000000, mean=0, sd=1)
hist(random_normals)
```

Histogram of random_normals



```
random_binom_2 <- qbinom(runif(n=1000000), size=10, prob=.4)
hist(random_binom_2)
```

Histogram of random_binom_2



15.5 set.seed()

R doesn't have the ability to generate truly random numbers! Random numbers are actually very hard to generate. (Think: flipping a coin $\rightarrow$ can be perfectly predicted if I know wind

speed, the angle the coin is flipped, etc.). Some people use random noise in the atmosphere or random behavior in quantum systems to generate “truly” (?) random numbers. Conversely, R uses deterministic algorithms which take as an input a “seed” and which then perform a series of operations to generate a sequence of random-seeming numbers (that is, numbers whose sequence is sufficiently hard to predict).

Let’s think about this another way. Sampling is a stochastic process, so every time you run `sample()` or `runif()` you are bound to get a different output (because different random seeds are used). This is intentional in some cases but you might want to avoid it in others. For example, you might want to diagnose a coding discrepancy by setting the random number generator to give the same number each time. To do this, use the function `set.seed()`.

In the function goes any number. When you run a sample function in the same command as a preceding `set.seed()`, the sampling function will always give you the same sequence of numbers. In a sense, the sampler is no longer random (in the sense of unpredictable to use; remember: it never was “truly” random in the first place)

```
set.seed(02138)
runif(n = 10)
```

```
[1] 0.51236144 0.61530551 0.37451441 0.43541258 0.21166530 0.17812129
[7] 0.04420775 0.45567854 0.88718264 0.06970056
```

The random number generator should give you the exact same sequence of numbers if you precede the function by the same seed,

```
set.seed(02138)
runif(n = 10)
```

```
[1] 0.51236144 0.61530551 0.37451441 0.43541258 0.21166530 0.17812129
[7] 0.04420775 0.45567854 0.88718264 0.06970056
```

15.6 Calculating an expectation using Monte Carlo

We can use repeated, independent draws from a random process in order to approximate its mean (or variance, or higher-order moments). The intuition comes from the law of large numbers. As we obtain repeated independent and identically distributed realizations from a random variable and take their average, this average should get closer and closer to the true expected value. The longer we run the simulation, the better the approximation.

Crucially, we can do this for essentially any random process that we can repeatedly obtain independent draws from. All we need to do is be able to implement the process in code. This can often be easier than trying to calculate an expectation analytically.

Consider the problem of calculating the expectation of the maximum of two independent 20-sided die rolls. We could construct a function that took the maximum of two independent draws from a 1-20 vector. Doing this a large number of times and taking the average allows us to approximate the true expectation of this new random variable without having to do any math!

```
radvantage <- function(sides=20, k=2){  
  return(max(sample.int(sides, k, replace = T)))  
}  
  
# Take the mean of 100,000 rolls with "advantage"  
set.seed(60639)  
mean(map_vec(1:1e5, radvantage))
```

```
[1] 33379
```

Next, consider the “birthday problem” or the probability that among k randomly chosen people, at least two people will have the same birthday. Recall that the expected value of an indicator random variable is equal to the probability that the indicator is equal to 1 (so we can calculate probabilities using Monte Carlo by getting draws from an indicator variable for the event of interest).

We could construct a function that simulates one hypothetical set of k people. Assume no leap years and uniformly distributed birthdays.

```
birthday <- function(k){  
  # For each person, sample a birthday  
  bdays <- map_vec(1:k, function(x) sample.int(365, 1))  
  # If any birthdays are duplicated, the length of unique(x) will be shorter than the length of x  
  return(as.numeric(length(bdays) != length(unique(bdays))))  
}  
  
set.seed(60639)  
# For a room of 23 people  
mean(map_vec(1:1e4, function(x) birthday(23)))
```

```
[1] 0.5156
```

```
# For a room of 90 people
mean(map_vec(1:1e4, function(x) birthday(60)))
```

```
[1] 0.9953
```

Notably, it is still important that the expectation of the random variable **exists**. Consider the *Cauchy* distribution which has no finite mean. Suppose I take a large number of independent draws from this distribution and average them. I get wildly different results across different runs!

```
set.seed(60639)
mean(rcauchy(1e5)) # Average of 100,000 draws from a Cauchy(0, 1)
```

```
[1] 7.696414
```

```
mean(rcauchy(1e5)) # Another average of 100,000 draws from a Cauchy(0, 1) -- wildly different
```

```
[1] 0.8749143
```

Exercise

Baccarat

Monte Carlo simulations are frequently used to analyze expected outcomes in casino games and calculate the “house edge” or the percent of a player’s winnings that they can expect to lose on a particular bet. One casino game that is known to have a very **low** house edge is Baccarat. In this exercise, you will implement a function that simulates a game of Baccarat. Using this function, you will calculate the expected value for each of the three types of bets in the game. You will then simulate how these expected values might change if the casino set different payouts for some of the bets.

First, a brief primer on baccarat. Baccarat is a table game in which two hands of playing cards – the “player” hand and the “banker” hand – are drawn and then compared against one another. The hand containing the highest score wins the round. The most common version played in modern casinos is called “**punto banco**” baccarat and essentially plays itself according to a fixed set of card drawing rules. The names “player” and “banker” are simply labels for each of the two hands.

Before each round of play, bettors can place one of three bets: that the “player” hand will win, that the “banker” hand will win, or that the hands will “tie” and have the same score.

Then, the round begins. The game is played by drawing cards from a “shoe” of cards containing multiple decks of standard playing cards (typically six or eight). The shoe is shuffled to randomize the order. **Two** cards are dealt to the *player hand* and **two** cards are dealt to the *banker hand*. We will ignore common casino practice of “burning” some cards from the top of the deck to discourage card counting as this does not impact the simulation.

The player hand may then be dealt a third card depending on the value of the initial two cards.

The banker hand may also receive a third card depending on the value of the banker’s initial two cards as well as the value of the player’s third card if they drew one. The rules for determining whether either hand receives a third card are below. After all cards are dealt, the value of the hands is calculated.

Face cards and 10s are worth zero points, aces are worth one point, and all other cards are worth their printed number (2-9 points). The value of a hand is the ones digit of the sum of the point values of the cards in that hand. So a hand consisting of a 2 and a 5 would be worth 7 points, while a hand consisting of a 4, 5 and 3 would be worth 2 points ($4+5+3 = 12$, $12 \bmod 10 = 2$). In other words, hands are valued at their point sum modulo 10.

The hand with the highest value is declared the winner. If the hands are equally valued, the result is a “tie”

The rules for determining whether the player or banker hand receives a third card in a given round are somewhat complex:

First, if **either** the player or the banker has a hand valued at 8 or 9, then no third cards are drawn, the round ends, and a winner is declared based on the value of the initial two-card hands.

Second, if neither hand is an 8 or 9, the game decides whether to give the “player hand” a third card. If the player’s initial hand value is 5 or less, then they are given a third card. If the player’s initial hand value is 6 or 7, they “stand”

Third, the game decides whether to give the “banker hand” a third card. If the player did not receive a third card, the banker acts by the same rule as the player (draw if 5 or less, stand if 6 or 7). If the player did receive a third card, then the decision to draw depends on both the value of the banker’s current hand and the value of the **third card drawn** by the player.

- If the banker’s hand is valued at 2 or less, they always draw a third card irrespective of what the player was dealt.
- If the banker’s hand is valued at 3, they draw a third card unless the player’s third card is an 8.

- If the banker's hand is valued at 4, they draw a third card if the player's third card is between 2 and 7 (inclusive)
- If the banker's hand is valued at 5, they draw a third card if the player's third card is between 4 and 7 (inclusive)
- If the banker's hand is valued at 6, they draw a third card only if the player's third card is a 6 or 7.
- If the banker's hand is valued at 7, they do not draw a card

The Wikipedia page for Baccarat has a nice table that summarizes the “hit/stand” decision for the banker.

After the cards are dealt and the winner determined for the round, bets are paid.

- If the bettor bet on “player” and the “player” hand wins, they keep the amount wagered and receive an equal amount from the casino (pays 1-to-1).
- If the bettor bet on “banker” and the “banker” hand wins, they keep the amount wagered and receive 95% of their wager from the casino (pays 19-to-20).
- If the bettor bet on either “player” or “banker” and the result is a tie, they keep the amount wagered but receive no additional money from the casino (push).
- If the bettor bet on “tie” and the result is a tie, they keep the amount wagered and receive 8 times their wagered amount from the casino (pays 8-to-1)
- If the bettor bet on “player” and the “banker” wins or if they bet on “banker” and the “player” wins, they lose the amount wagered.
- If the bettor bet on “tie” and either “banker” or “player” wins, they lose the amount wagered.

15.6.1 Part 1 (-)

Implement, in code, a function that simulates a single round of play (ignore the betting process for now). Assume the casino is playing with a shoe of six decks of standard playing cards and that this shoe is refreshed after every round.

Below is an outline for the sorts of functions you should implement to break this problem down into smaller component parts along with some hints.

```
# This function should take as input some number of decks and generate a shoe of cards in
# HINT: You don't need to store the actual cards, just their values
# HINT 2: Read the documentation for the 'rep' function
gen_deck <- function(){
}
}
```

```
# This function should take as input some vector that represents a hand of cards and return
# HINT: In R, the modulo operator is %%
value_hand <- function(){

}
```

```
# This function should take as input the banker's and the player's hands and return the outcome
determine_winner <- function(){

}
```

```
# This is your main function. It should return (at a minimum) the outcome of the round of play
# HINT: Write out each phase of the round in plain language in the comments. Then try to implement
# HINT 2: You'll likely need to use a lot of conditional statements to implement the draw
play_round <- function(){

}
```

15.6.2 Part 2 (-)

Now implement a function that takes as input a bettor's chosen outcome and their wager. The function should then play a round of baccarat and return the amount that the bettor receives from the casino. There are a few equally valid ways to implement “winnings.” For the purposes of this problem, you should have the function return 0 if the player loses their entire wager.

```
# This function should take as input a choice and a wager. You can choose to have it wrap
standard_payoff <- function(){

}
```

15.6.3 Part 3 (-)

Using a Monte Carlo simulation, calculate the expected return of a wager of 100 dollars on the “player” bet. In other words, if a bettor pays 100 to bet on “player”, what is the amount that they expect to win. Set your seed once to 60639 and run it for 300,000 iterations (try running for fewer iterations as you're testing, but you'll need a decent number of iterations to get the desired numerical precision)

Calculate the “house edge” of the “player” bet. The house edge is the difference between a bettor’s amount wagered and their expected return, divided by the amount wagered. In other words, it’s the amount that a casino expects to keep of a player’s bet.

$$\text{HouseEdge} = \frac{\text{AmountWagered} - \text{ExpectedReturn}}{\text{AmountWagered}}$$

```
set.seed(60639) # Set the seed
# Simulation code here
```

15.6.4 Part 4 (-)

Use a Monte Carlo simulation to calculate the expected return of a wager of 100 dollars on the “banker” bet. Calculate the “house edge” of the “banker” bet. Set the seed to 60640 and run for 300,000 iterations.

```
set.seed(60640) # Set the seed
# Simulation code here
```

Which bet has the lower house edge?

15.6.5 Part 5 (-)

One odd feature of Baccarat is that the “banker” bets pay 19-to-20 rather than the 1-to-1 for player bets. Using a Monte Carlo simulation, show why casinos don’t pay 1-to-1 on the banker (calculate the house edge). You’ll need to write a new payoff function to handle the alternative payout structure. Set the seed to 60641 and run for 300,000 iterations.

```
set.seed(60641) # Set the seed
# Simulation code here
```

15.6.6 Challenge problem (-)

A new version of Baccarat that has become popular in some areas called “EZ-Baccarat” purports to solve the problem you identified in Part 5 while still paying “banker” bets at 1-to-1. It does so by having any round where the banker wins with a hand of 3 cards that totals 7 points be a “push” (bettors keep their wagers but don’t win any additional money) rather than a “win” for the banker hand.

Show, using a Monte Carlo simulation, how this payout structure retains the house edge on “banker” bets. You may have to modify your simulation functions to return more information about the outcome of the round in order to implement a new payoff function.

See the [state regulatory documents](#) for more info about this version of the game.

```
set.seed(60642) # Set the seed
```